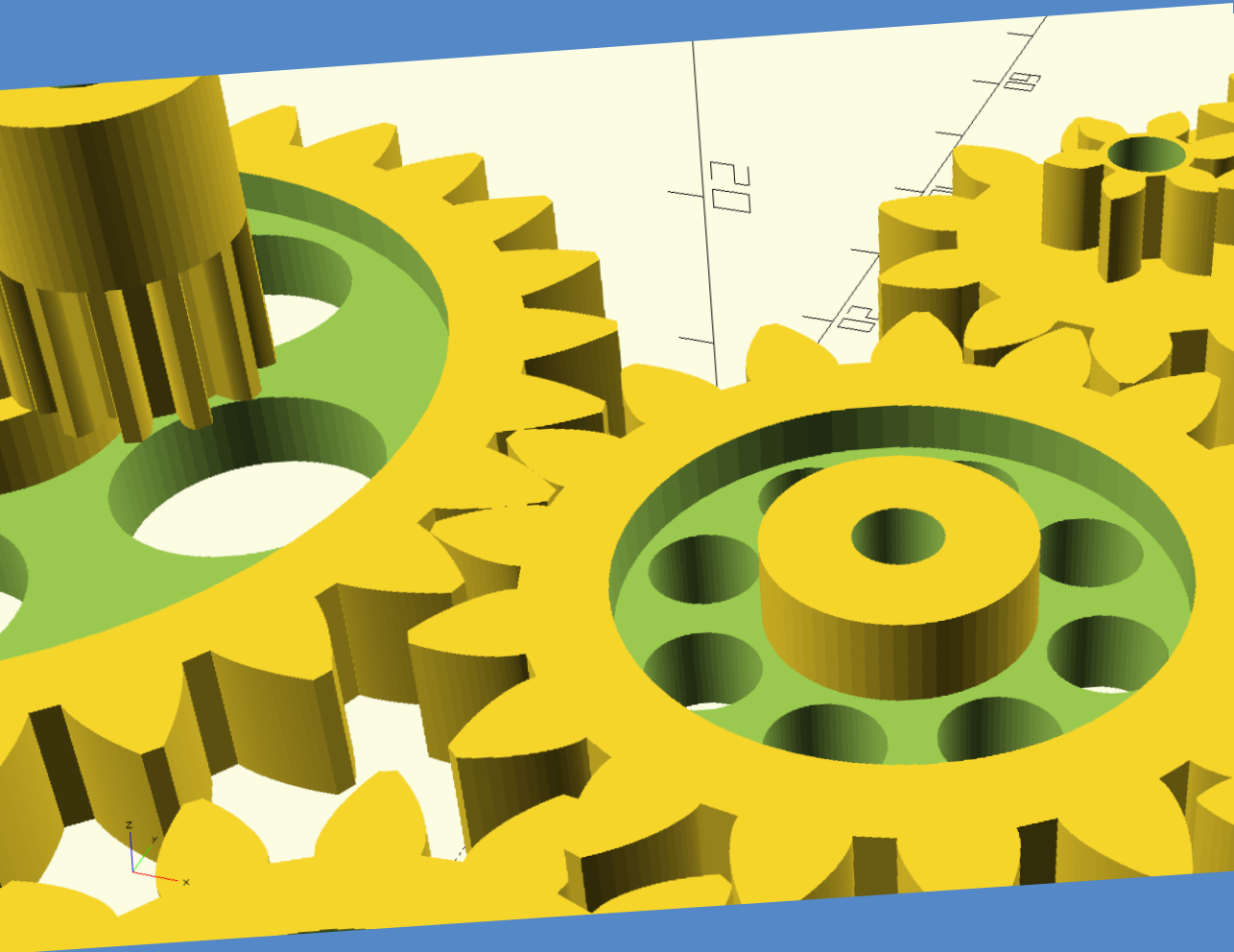


Technical Modeling with OpenSCAD

Create Models for 3D Printing, CNC Milling,
Process Communication and Documentation



Tam Hanna

Technical Modeling with OpenSCAD

**Create Models for 3D Printing, CNC Milling,
Process Communication and Documentation**



Tam HANNA

● This is an Elektor Publication. Elektor is the media brand of
Elektor International Media B.V.

PO Box 11

NL-6114-ZG Susteren, The Netherlands

Phone: +31 46 4389444

● All rights reserved. No part of this book may be reproduced in any material form, including photocopying, or storing in any medium by electronic means and whether or not transiently or incidentally to some other use of this publication, without the written permission of the copyright holder except in accordance with the provisions of the Copyright Designs and Patents Act 1988 or under the terms of a licence issued by the Copyright Licensing Agency Ltd., 90 Tottenham Court Road, London, England W1P 9HE. Applications for the copyright holder's permission to reproduce any part of the publication should be addressed to the publishers.

● British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library

ISBN 978-1-907920-99-8

EBOOK 978-3-89576-394-6

EPUB 978-3-89576-395-3

© Copyright 2020: Elektor International Media B.V.

Prepress Production: D-Vision, Julian van den Berg



Elektor is part of EIM, the world's leading source of essential technical information and electronics products for pro engineers, electronics designers, and the companies seeking to engage them. Each day, our international team develops and delivers high-quality content - via a variety of media channels (including magazines, video, digital media, and social media) in several languages - relating to electronics design and DIY electronics. www.elektormagazine.com

LEARN > DESIGN > SHARE

Chapter 1 • Why OpenSCAD?	9
1.1 What do we need?	12
1.2 Who am I?	13
Chapter 2 • Installing OpenSCAD.	15
2.1 Installing OpenSCAD: Linux, compiled package.	15
2.2 Installing OpenSCAD: Linux, compilation.	16
2.3 Installing OpenSCAD: Windows, finished binary	22
2.4 Excursus: Nightlies	23
2.5 Conclusion	24
Chapter 3 • User interface and first experiments.	25
3.1 OpenSCAD start screen	25
3.2 Manipulating the Viewport	29
3.3 More Settings	31
3.4 Conclusion	32
Chapter 4 • Create, combine and move 3D objects	33
4.1 Hands hovering over the building surface	33
4.2 Create cuboids: data types and more.	36
4.3 Sphere and cylinder.	39
4.4 Translation operators move objects	42
4.5 Rotation operators rotate objects	44
4.6 Color parts of objects.	47
4.7 Linking translation and rotation.	49
4.8 Combine 3D objects smartly	51
4.9 Outlook: Realisations, polyhedra, and projections	54
Chapter 5 • Realize and understand (round) objects	55
5.1 Edges and corners.	55
5.2 Antipattern: Construct with \$fn.	58
5.3 Excursus: 3D print pipeline.	58
5.4 3D printing models	60
5.5 Holes, for the first	60
5.6 Conclusion	63
Chapter 6 • OpenSCAD as 2D modeling tool	64

6.1 Theory of construction in the two-dimensional domain	66
6.2 Creating Ellipses	68
6.3 Linear extrusion	70
6.4 Worked Example: Coat rack a la Tam	71
6.5 linear_extrude with advanced parameterization.	74
6.6 Create rotationally symmetrical objects	78
6.7 Fully parametric construction from point clouds.	81
6.7 Holes in polygons	85
6.8 Worked Example: Board holder	86
6.9 What next?	91
Chapter 7 • OpenSCAD as a dynamically reconfigurable modeling system	92
7.1 Variable in OpenSCAD	92
7.2 Modules create geometry	96
7.3 Selections react to parameter states	101
7.4 Selections, modules, children	102
7.5 Reject parameters with Assert.	103
7.6 Duplicate geometry with "for"	105
7.7 Functions and calculations	110
7.8 Processing lists	111
7.9 Tools for troubleshooting	115
7.10 Conclusion	119
Chapter 8 • Texts, projections and bump mapping	120
8.1 Render texts	120
8.2 A question of alignment	122
8.3 Add and manage fonts	124
8.4 Exporting 2D Snapshots	125
8.5 Import geometry item maps	129
8.6 Conclusion	134
Chapter 9 • Advanced 3D objects	135
9.1 Create three-dimensional polygons	135
9.2 Excursus: Soap Dispenser, Part 1	138
9.3 Polygon alignment in three-dimensional space	141

9.4 Combination with Minkowski	143
9.5 Effect of the Minkowski operator on boreholes.	148
9.6 Excursus: Soap dispenser, part 2.	150
9.7 The Hull operator.	152
9.8 Conclusion	155
Chapter 10 • MCAD - technical primitives for OpenSCAD	156
10.1 Providing the library.	156
10.2 Generation of gears, analysis of the library structure.	157
10.3 Conjuring screws and nuts	158
10.4 Lego emulation	161
10.5 Generate outlines of stepper motors	162
10.6 Conclusion	164
Chapter 11 • Value-added OpenSCAD	165
11.1 Model to measure	165
11.2 Controls of the Customizer	171
11.3 Custom model, second	172
11.4 OpenSCAD without App	174
11.5 OpenSCAD meets Python	176
11.6 Conclusion	180
Appendix • Quo vadis?	181
Contact the development team	181
Contact the author	182
Read more	183
Index	183

Chapter 1 • Why OpenSCAD?

When you think of a 3D design program, you instinctively think of AutoCAD, Rhino, Google SketchUp, or - for the oldies - 3D Studio Max. There's no question that great designs have been and will be created with all these products.

But there is no question that designers and coders have radically different thinking processes - the best proof for this is the definition of the Coder Colors shown in figure one, which stands for such an ugly color scheme that it can only come from the brain of a software developer.



Figure 1-1. The definition of the (now somewhat forgotten) term Coder Colors is not very charming. (Picture source: <http://www.pouet.net/topic.php?which=5540&page=1>)

Funnily enough, the engineering way of thinking is not exclusively found among computer scientists. The author works as a technical consultant for the American fashion company Icy Beats LLC (see <http://www.bopsync.com/>), and had a similar experience with the owner. The lady is an excellent and experienced fashion designer but doesn't get along with 3-D modeling software any more than the author does.

The reason for this is that both the fashion designer and the engineer "work with their hands" and therefore naturally think in hand movements and hand actions.

OpenSCAD differs from the systems mentioned in the introduction in that it uses a "step-by-step" approach. Since a picture often says more than 1000 words, figure two shows a "cut out" cuboid next to the source code intended for its generation. Please do not analyze the subtleties of the code at this point - the only important thing to note is that the cuboid is created by "subtracting" two cuboids.

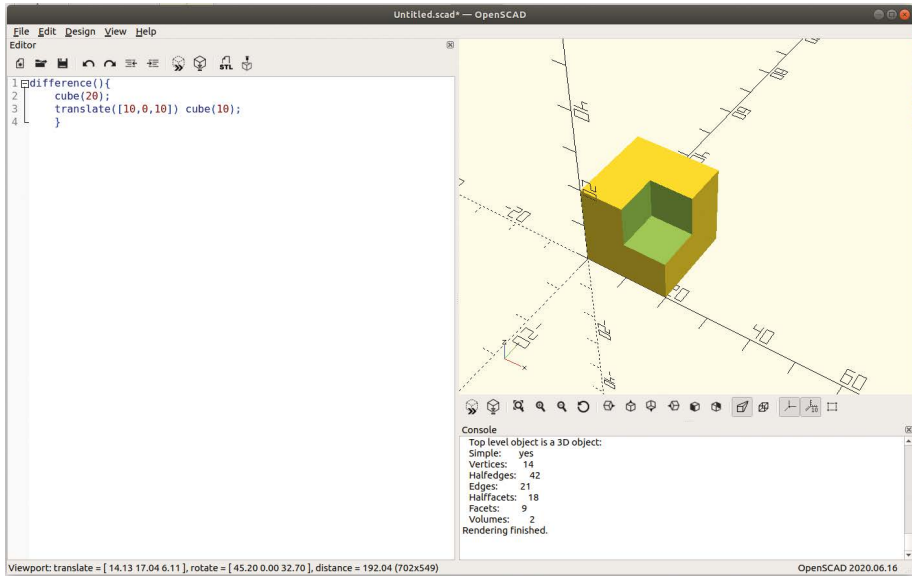


Figure 1-2. A picture is worth a thousand words.

In addition to the engineer-friendly approach, OpenSCAD benefits from the fact that the language implements a kind of object orientation. The best examples of this are shown in figures three, four, and five – they show real parts of the underground habitat of yours truly.

The bottom part of the "brackets" holding the dispenser in place was taken from identical code. The only difference between the two parts was that they were parameterized differently – OpenSCAD took care of generating the rest.



Figure 1-3. No bunker is complete if its users lack soap and lotion dispensers.



Figure 1-4. Naturally, this also applies to the bathroom.



Figure 1-5. ...sadly, black and white dispensers were only available in different sizes.

If you always wanted to design "engineering" parts, you will find OpenSCAD a great modeling system. This textbook will introduce you to the basics and advanced application scenarios. I will focus on the actual work with OpenSCAD, and peripherally on interacting with 3D printers. I explicitly do not want to give an introduction to "mechanical engineering" - if you are looking for that, find a list of books on different competence levels in the appendix.

1.1 What do we need?

The author demands little from the esteemed reader - spatial awareness is helpful. Safe handling of a caliper gauge is recommended; those who are familiar with a cordless screwdriver and/or a drilling station also have an easier life. Knowledge of electronics is explicitly not required!

From a technical point of view, we do not need much either. OpenSCAD is not demanding, the program works on slow and fast computers. The author illustrates how to set up the program under Windows and Linux; under MacOS, analogous work can be done. If you don't want to go through the effort of compiling the program yourself, older versions are available as ready-made packages.

If you buy a workstation explicitly for OpenSCAD, pay attention to extremely high single-thread performance. Figure 1-6 shows that the majority of the author's eight-core workstation is bored when working with OpenSCAD, while one core is under full steam.

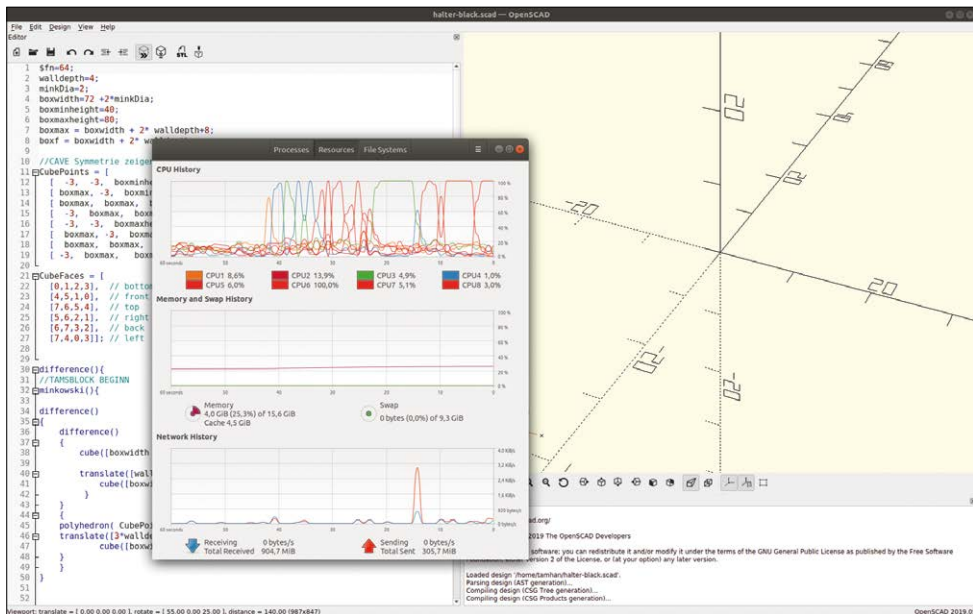


Figure 1-6. OpenSCAD no speak parallelisation.

So back to the human factors: OpenSCAD is a programming language. The author of these lines assumes that you have worked with C, Java, Javascript, or Pascal - other languages, whether object-oriented or not, are also suitable.

Otherwise, all you need is a desire to design - the author asks you to at least "skim" the work as a whole to get a grip on the functions contained in OpenSCAD. Learning the syntax by heart is not very useful outside the academic world - there is no reason why you should not use templates, this book, and the cheat sheet (<https://www.openscad.org/cheatsheet/>) when working with OpenSCAD.

If you can do one, it's easier to learn another

Experience has shown that a person who is familiar with a programming language can familiarize himself with (most) others relatively quickly. The author can also confirm this from his life experience - he started his career with Pascal, then learned C on the side and has had no major problems with more exotic languages like Python.

If you find yourself confronted with an older version of OpenSCAD, the situation is not too bad. The OpenSCAD developer team was close to a political party that will certainly not be given significant governmental responsibility in Austria for the next ten years and therefore does not have a great deal of money to distribute. As a result - logically, but also somehow, fortunately - a very slow development of the language means that the ecosystem is stable. Harder wars are fought over the question of which 3D printer is suited to the needs of a company or individual. The author of these lines uses two Renkforce RF100 V2 - a small, compact, ready-made device that can be purchased from Conrad for about 175 euros - which (especially with retrofitted component cooling, see the author's YouTube channel) works well and produces parts up to $12 \times 12 \times 12$ cm in size.

If you have another 3D printer and can handle it with confidence, this is no obstacle. We conduct our experiments with CURA because the author always uses CURA. If you use a different slicer and can already handle it, this is not a problem.

If you don't have a 3D printer and you can't find the space (and maintenance time) for such a hangar queen, use the services of model joineries instead. Almost every shop which sells 3D printers accepts print orders and - assuming a friendly approach - is very happy to perform "mercenary printing services" for paying customers.

1.2 Who am I?

Many moons ago, yours truly took to electrical engineering as a way to improve his life. Since then, technical progress gave rich gifts - digital oscilloscopes and 32-bit processors were once unaffordable, but now can be bought even on a limited budget. Colour displays are now so inexpensive that yours truly's firm installs them in humidors (!!!) to the delight of all (see picture).



Figure 1-7. A color display in the cigar box: accessible for everyone, thanks to HygroSage.

While advances in all areas of technology solved the electronics problem, the question of realizing custom "in-house" mechanical parts remained open - a wonderful control system for a drone is of no use, even in the presence of civil war and buyers, if you cannot build the actual drone based on it.

When the author was still on active duty, each part required a long and bumpy journey in a ZAZ-965 Zaporizhzhets of the motor pool to talk to the model joinery. A small spar evolved into a task that demanded highly qualified technical personnel.

3D printers have meanwhile answered the "technical" part of this question - you can get admittedly small, but problem-free devices as mentioned in the introduction for less than 200 Euro while in Germany. Imports from China can severely undercut this price.

Sadly, the question of the theoretical design part remained open even as 3D printers became better and cheaper. If you are looking for a way to design mechanical parts, be it housings or brackets, you will find OpenSCAD convenient. Yours truly wouldn't want to do without it anymore - whether furnishing his property, preparing proof of concepts for court hearings, or designing a replacement button for the beloved LeCroy oscilloscope of a French electronics engineer: OpenSCAD is a product that gives pleasure time and time again.

Enough talk: let's now get going in this spirit! And may your designs - whether civil or military - always work to your (and their end user's) utmost satisfaction.

Chapter 2 • Installing OpenSCAD

Given that you are still reading, yours truly is free to assume that you are sufficiently convinced of the benefits of OpenSCAD. To start our experiments, we need a version of the software.

OpenSCAD is open source like many other "novel" engineering programs. This means that you can view the product's source code and correct any errors found yourself. On the other hand, however, this also means that there are several ways to get our hands onto a runnable binary.

This chapter looks at the deployment under Windows and Linux. For those who want to work under macOS, there are various instructions on the Internet about both compilation and use of provided packages.

2.1 Installing OpenSCAD: Linux, compiled package

Anyone who has spent some time with Linux will certainly know commands like apt-get. They query "package source servers" for a package, and proceed to download and install it afterwards.

Since most distributors perform extensive checks on the packages listed in their package sources, the packages contained there are often not particularly up-to-date - in the case of Ubuntu, for example, the server will provide you with an OpenSCAD version dated from 2015. Quarrels between the OpenSCAD and Ubuntu teams ensure that some releases of Ubuntu have to make do without precompiled binaries.

If you want to work with a reasonably up-to-date OpenSCAD, the first step is to visit the URL <https://www.openscad.org/downloads.html>. There, scroll down to the Other Linux section. Then click on the link shown in figure one to download a file of about 35 MB.

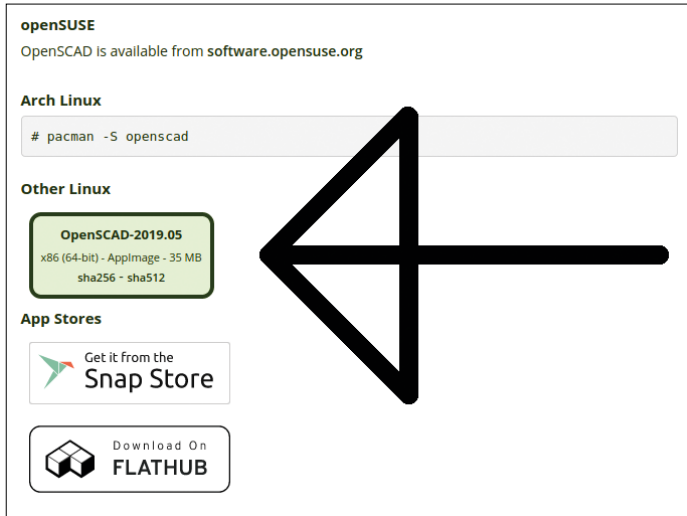


Figure 2-1. This link leads to happiness.

Please note that "finished" OpenSCAD packages for Linux are only available for 64-bit versions of the operating system. If you absolutely must or want to work with a 32-bit system, you have to do a manual compilation as discussed below.

AppImage files are a "new type of" packaging program which packages Linux applications and the libraries belonging to them. OpenSCAD is started by entering the following two commands:

```
tamhan@TAMHAN18:~/Downloads$ chmod +x OpenSCAD-2019.05-x86_64.AppImage
tamhan@TAMHAN18:~/Downloads$ sudo ./OpenSCAD-2019.05-x86_64.AppImage
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
. . .
```

The first call adds the executable attribute to the file, while the subsequent command instructs to start OpenSCAD. The program reacts by displaying the OpenSCAD start screen. Depending on the distribution, there is the possibility of integrating the pre-built OpenSCAD binary into the program starter - as yours truly prefers to work on the command line level and as distributions change permanently, please refer to the documentation of the respective system. Alternatively, you can also work through the compilation instructions discussed in the following step, which integrate OpenSCAD "completely" into your system.

2.2 Installing OpenSCAD: Linux, compilation.

OpenSCAD version 2019.05, which we downloaded as a finished package, was functional, but is not particularly up-to-date. If you compile OpenSCAD from source, you firstly get the latest version, and secondly can instruct the compiler to provide an experimental version of the OpenSCAD language. This lets you benefit from additional features that cannot be accessed "as is". Annoyingly, OpenSCAD consists of a whole group of components, which is why compilation requires a little work.

The component zoo

If you always wanted to know which libraries work in the background of OpenSCAD, https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Building_OpenSCAD_from_Sources is a highly recommended source.

Like almost all other open-source projects of "larger" dimensions, OpenSCAD development is managed using the git version control system. Our first task is to make sure that git is installed on your workstation:

```
tamhan@TAMHAN18:~/Downloads$ sudo apt-get install git
```

The next step makes the git utility download the latest version of the source code from the repository. Don't be surprised that the author enters this command in the home directory - virtually all git repositories create a new directory during the download process:

```
tamhan@TAMHAN18:~$ cd ~/
tamhan@TAMHAN18:~$ git clone https://github.com/openscad/openscad.git
Cloning into 'openscad'...
. . .
Receiving objects: 100% (64355/64355), 62.52 MiB | 5.11 MiB/s, done.
Resolving deltas: 100% (45989/45989), done.
```

The command downloads the most recent version - this is "disadvantageous" in that during development, non-working or non-compilable versions are sometimes uploaded. In this case, it is recommendable to delete the created directory and then use the branch function to download one of the releases available at <https://github.com/openscad/openscad/tree/master/releases>.

Be that as it may, the next step is to return to the OpenSCAD working directory:

```
tamhan@TAMHAN18:~$ cd openscad/
```

Git repositories have long been capable of mapping relationships between repositories – if a component is based on a library, the component developer can set up a symbolic link to the library's Git repository. This is disadvantageous for us in that the following two commands must be entered within the OpenSCAD folder to provide a fully-fledged source code environment:

```
tamhan@TAMHAN18:~/openscad$ git submodule init
Submodule 'libraries/MCAD' (https://github.com/openscad/MCAD.git) registered
for path 'libraries/MCAD'
tamhan@TAMHAN18:~/openscad$ git submodule update
. . .
Cloning into '/home/tamhan/openscad/libraries/MCAD'...
Submodule path 'libraries/MCAD': checked out
'a7be3d623669d635b7249a327cfce5796ea200b3'
```

The explicit reference to the nested relationships in git repositories is important because git refuseniks like to use the download button shown in Figure 2-2. Unfortunately, its use is not appropriate because the archive it provides does not include the linked repositories.

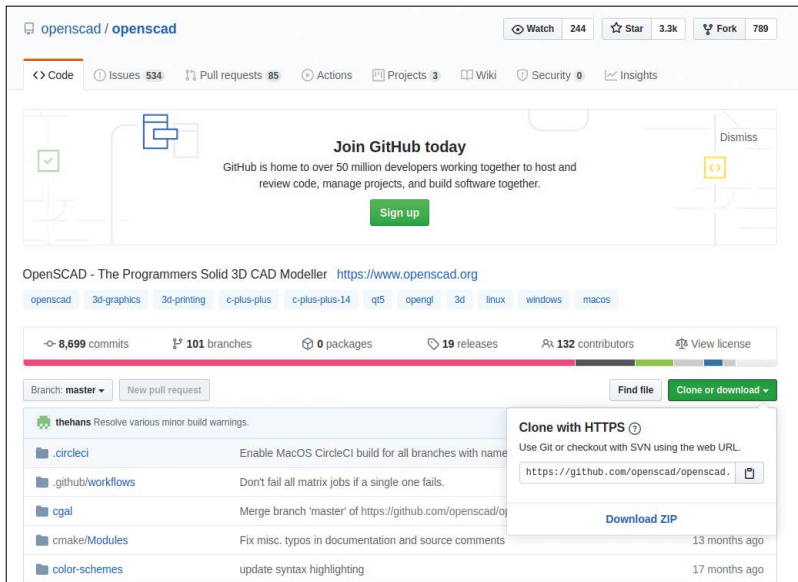


Figure 2-2. Git Refugees, beware: this button promises disaster.

The next step provides additional dependencies:

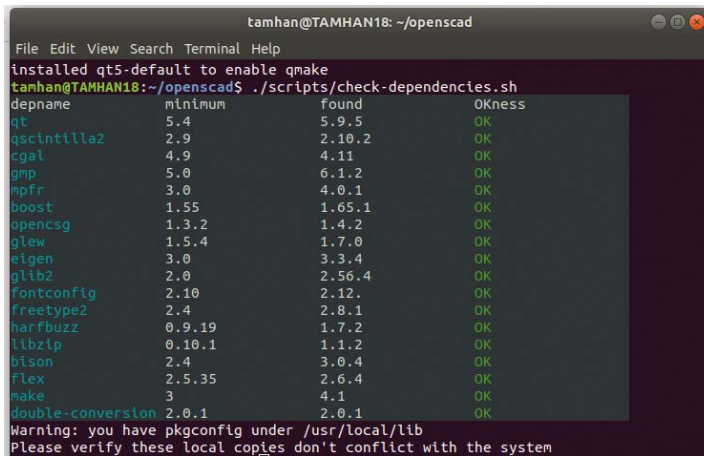
```
tamhan@TAMHAN18:~/openscad$ sudo ./scripts/uni-get-dependencies.sh
. . .
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
installed qt5-default to enable qmake
```

The OpenSCAD development team supports us with a script which automatically downloads the necessary elements using package managers and other tools on somewhat current releases of Ubuntu. If problems occur during this process, you can find help at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Building_on_Linux/UNIX#Installing_dependencies.

At this point, a reboot of the system is recommended. Afterwards, the existence of the necessary prerequisites can be checked by the following command:

```
tamhan@TAMHAN18:~/openscad$ ./scripts/check-dependencies.sh
```

On the author's workstation, its processing leads to the result shown in Figure 2-3.



```
tamhan@TAMHAN18: ~/opencscad
File Edit View Search Terminal Help
installed qt5-default to enable qmake
tamhan@TAMHAN18:~/opencscad$ ./scripts/check-dependencies.sh
depname      minimum      found      OKness
qt           5.4          5.9.5      OK
qscintilla2  2.9          2.10.2     OK
cgal         4.9          4.11       OK
gmp          5.0          6.1.2      OK
mpfr         3.0          4.0.1      OK
boost       1.55         1.65.1      OK
opencsg      1.3.2        1.4.2       OK
glew         1.5.4        1.7.0       OK
eigen        3.0          3.3.4       OK
glib2        2.0          2.56.4      OK
fontconfig   2.10         2.12        OK
freetype2    2.4          2.8.1       OK
harfbuzz     0.9.19       1.7.2       OK
libzip       0.10.1       1.1.2       OK
libbson      2.4          3.0.4       OK
flex         2.5.35       2.6.4       OK
make         3            4.1         OK
double-conversion 2.0.1       2.0.1       OK
Warning: you have pkgconfig under /usr/local/lib
Please verify these local copies don't conflict with the system
```

Figure 2-3. The necessary dependencies are ready for compilation.

Be that as it may, the next step is to run the qmake command-line tool. Its task is to bring the source code into a structure that is "processable" by the compiler and provide various script files and other elements that the compiler will use during its work. If you wish to compile a "normal" version of OpenSCAD, the following input is sufficient:

```
tamhan@TAMHAN18:~/opencscad$ qmake
Info: creating stash file /home/tamhan/opencscad/.qmake.stash
Project MESSAGE: If you're building a development binary, consider adding
CONFIG+=experimental
. . .
to the PKG_CONFIG_PATH environment variable
No package ,lib3MF' found
Project MESSAGE: 3MF Import/Export disabled
```

Don't be surprised if the tool throws a group of status outputs during the processing of the qmake command, referring to the absence of various import/export libraries. Unless qmake throws a really serious error, everything will work fine.

If you compile OpenSCAD to take advantage of modern or experimental features, the call to qmake will look slightly different. In this case, you have to provide an additional attribute, which is added via the config command line parameter:

```
tamhan@TAMHAN18:~/opencscad$ qmake CONFIG+=experimental
. . .
```

Be that as it may, we can instruct a compilation of our application at this point. Passing the -B parameter instructs make to eliminate all existing Makefiles. This is important if you switch between compiling a normal and an experimental version of OpenSCAD:

```
tamhan@TAMHAN18:~/opencscad$ make -B
. . .
```

Due to the considerable codebase, it is recommended to parallelise the actual compilation triggered by make as much as possible. To do this, you have to pass the parameter `-j` – when sitting on an eight-core workstation, enter the following command:

```
tamhan@TAMHAN18:~/opencscad$ make -B -j 8
. . .
418 translated messages, 24 fuzzy translations, 7 untranslated messages.
itstool missing, won't apply translations to opencscad.appdata.xml
tamhan@TAMHAN18:~/opencscad$
```

As shown in Figure 2-4, make sometimes throws errors during execution, indicating problems with the translation files. However, these are irrelevant to the functioning of OpenSCAD.

```
boost_regex -lGLEW -lopencsg -lmpfr -lgmp -L/usr/lib/x86_64-linux-gnu -lqscintilla2_qt5 -lQt5PrintS
lX11 -lpthread
/home/tamhan/opencscad/scripts/translation-make.sh'
Compiling language files (CWD = /home/tamhan/opencscad)...
msgfmt -c -v -o ./locale/cs/LC_MESSAGES/opencscad.mo ./locale/cs.po
248 translated messages, 51 fuzzy translations, 150 untranslated messages.
msgfmt -c -v -o ./locale/de/LC_MESSAGES/opencscad.mo ./locale/de.po
449 translated messages.
msgfmt -c -v -o ./locale/es/LC_MESSAGES/opencscad.mo ./locale/es.po
223 translated messages, 91 fuzzy translations, 135 untranslated messages.
msgfmt -c -v -o ./locale/fr/LC_MESSAGES/opencscad.mo ./locale/fr.po
406 translated messages, 26 fuzzy translations, 17 untranslated messages.
msgfmt -c -v -o ./locale/hy/LC_MESSAGES/opencscad.mo ./locale/hy.po
408 translated messages, 24 fuzzy translations, 17 untranslated messages.
msgfmt -c -v -o ./locale/pl/LC_MESSAGES/opencscad.mo ./locale/pl.po
377 translated messages, 33 fuzzy translations, 39 untranslated messages.
msgfmt -c -v -o ./locale/ru/LC_MESSAGES/opencscad.mo ./locale/ru.po
410 translated messages, 23 fuzzy translations, 16 untranslated messages.
msgfmt -c -v -o ./locale/uk/LC_MESSAGES/opencscad.mo ./locale/uk.po
299 translated messages, 42 fuzzy translations, 108 untranslated messages.
msgfmt -c -v -o ./locale/zh_CN/LC_MESSAGES/opencscad.mo ./locale/zh_CN.po
379 translated messages, 24 fuzzy translations, 46 untranslated messages.
msgfmt -c -v -o ./locale/zh_TW/LC_MESSAGES/opencscad.mo ./locale/zh_TW.po
418 translated messages, 24 fuzzy translations, 7 untranslated messages.
itstool missing, won't apply translations to opencscad.appdata.xml
tamhan@TAMHAN18:~/opencscad$
```

BILD 2-4. The make run is quite communicative.

Why qmake and make?

OpenSCAD is based on the C++ cross-platform framework QT. It not only offers developers libraries and a GUI stack, but also extends the C++ language with functions such as a signal slot system.

To be able to process applications equipped with these attributes with the usual compiler of the operating system, an additional step is required. qmake takes care of building the Makefiles in a way that instructs Qt's helper infrastructure to be activated. After the qmake command has been processed, a Makefile is found in the project directory, which is responsible for the actual compilation using the operating system's compiler.

After successfully running the command, the compiler has created an executable binary file. In the interests of consistency, it is recommended to clean up two "frequent" storage locations for OpenSCAD binaries:

```
tamhan@TAMHAN18:~/opencscad$ rm /usr/local/lib/opencscad
rm: cannot remove '/usr/local/lib/opencscad': No such file or directory
tamhan@TAMHAN18:~/opencscad$ rm /usr/local/share/opencscad
rm: cannot remove '/usr/local/share/opencscad': No such file or directory
```

If you get the message "No such file or directory", the file was not present. If you get a permission error instead, you have to add `sudo` to remove any lingering remnants of OpenSCAD binaries.

If you are working under an operating system derived from Debian - Ubuntu is explicitly included - you have to download the `checkinstall` tool at this point. It takes care of putting the binary into a package that can be processed by the Debian package management system:

```
tamhan@TAMHAN18:~/opencscad$ sudo apt-get install checkinstall
Reading package lists... Done
. . .
```

If you don't work with Debian, just type `sudo make install` at this point and enjoy the results. For Debian users, the process continues as follows:

```
tamhan@TAMHAN18:~/opencscad$ sudo checkinstall -D make install
checkinstall 1.6.2, Copyright 2009 Felipe Eduardo Sanchez Diaz Duran
This software is released under the GNU GPL.
```

For the creation of Debian packages, various information called metadata is required. In the first step, the product asks if it should create a default parameter set. This is confirmed by typing `Y` and then pressing `Enter`:

```
The package documentation directory ./doc-pak does not exist.
Should I create a default set of package docs? [y]: y
```

In the next step, the program gives you the option of adjusting some settings. For us this is not necessary, simply press `Enter` to accept the default settings:

```
End your description with an empty line or EOF.
>>
```

```
*****
*** Debian package creation selected ***
*****
. . .
```

13 - Replaces: []

Enter a number to change any of them or press ENTER to continue:

The reward for the effort is the screen shown in Figure 2-5, which informs about the successful installation - the fact that the software throws some error messages during the processing "is part of the good tone under Unix".

```
Copying files to the temporary directory...tar: .0: Cannot stat: No such file or directory
tar: Exiting with failure status due to previous errors
OK
Stripping ELF binaries and libraries...OK
Compressing man pages...OK
Building file list...OK
Building Debian package...OK
Installing Debian package...OK
Erasing temporary files...OK
Writing backup package...OK
OK
Deleting temp dir...OK

*****
Done. The new package has been installed and saved to
/home/tamhan/openscad/openscad_20200616-1_and64.deb
You can remove it from your system anytime using:

    dpkg -r openscad
*****
tamhan@TAMHAN18:~/openscad$
```

Figure 2-5. The installation was successful.

If you look carefully at the output, you'll also notice why we've put in the effort with the package. If you would like to banish OpenSCAD from your workstation at some point, simply enter `dpkg -r openscad`.

2.3 Installing OpenSCAD: Windows, finished binary

Compiling unixoid programs under Windows is generally a hairy task, since Windows development, which is usually done under Visual Studio, requires a completely different workflow and toolchain. Fortunately, there are comparatively recent packages for Windows at <https://www.openscad.org/downloads.html>.

Just scroll down until you see the box shown in Figure 2-6. The OpenSCAD development team offers pre-compiled binary files for both 32-bit and 64-bit variants of Windows.

Windows

System requirements: Windows XP or newer on x86 32/64 bit

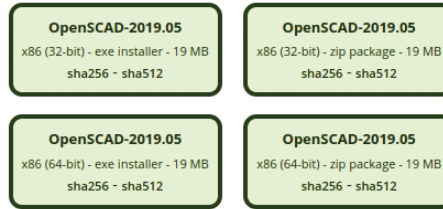


Figure 2-6. OpenSCAD is available for Windows in four different versions.

We decide on a version marked with the string exe installer, which fits the bitness of our operating system. After that, an installation wizard starts, which even integrates the program into the start menu.

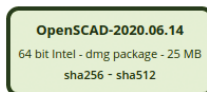
2.4 Excursus: Nightlies

Anyone scrolling further down on the OpenSCAD website will find the header shown in Figure 2-7. It offers precompiled binaries compiled from the "latest" contents of the GitHub repository.

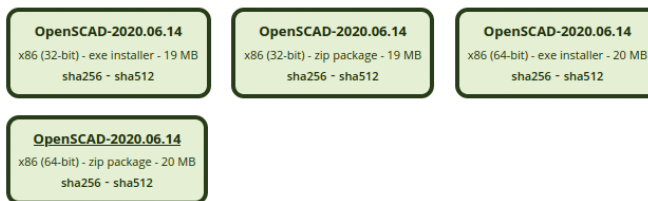
Development Snapshots

Development snapshots are built irregularly. If you want access to a more recent development snapshot, please contact the [mailing list](#).

macOS



Windows



Linux - AppImage

Please try the automatically built snapshots first if you are running a supported distribution (see below), the following AppImage builds are still experimental.

The ARM 64-bit (aarch64) AppImage is built and tested only for **Raspberry PI OS 64-bit** (which may still be in beta test).



Figure 2-7. In the Development Snapshots section, you can find ready-to-use packages with beta versions.

Since the OpenSCAD developers have been compiling their Linux images in cooperation with Suse for some time now, there is a special convenience variant for Linux users. The URLs shown in Figure 2-8 allow the inclusion of a package source containing daily updated packages for different distributions.

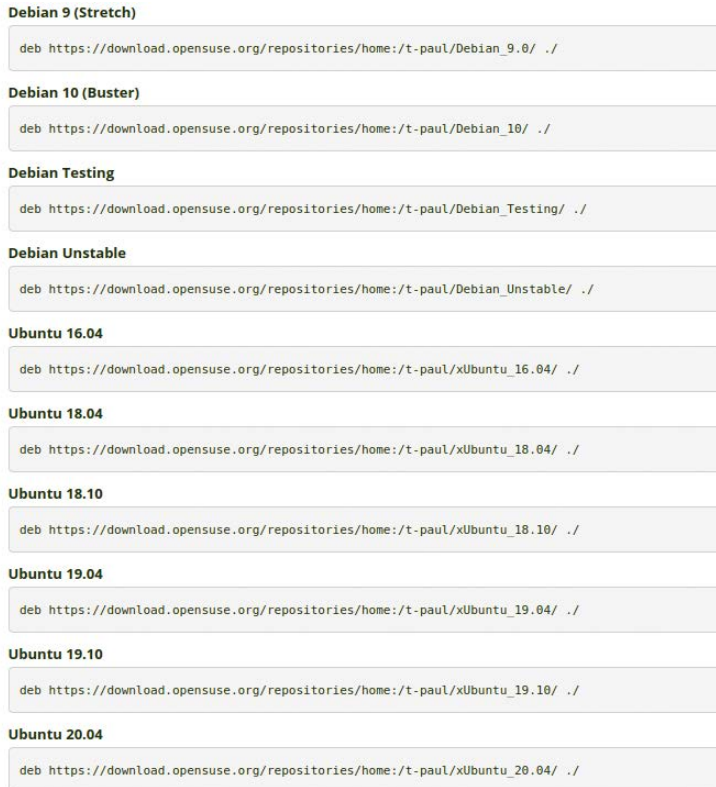


Figure 2-8. Under the header Linux - Distribution Packages you can find turnkey new images of test versions.

Please note, however, that the binary contained in them is named `openscad-nightly`. Therefore a slightly different call is required at program start:

```
tamhan@TAMHAN18:~/openscad$ ./openscad-nightly
```

2.5 Conclusion

Be that as it may: at this point you should have OpenSCAD installed on your computer and see the OpenSCAD start screen in front of you. This means we are ready to take our first steps into the world of 3D modeling.

If you can't get OpenSCAD to work at all, you can use a browser-based variant instead. Its handling is discussed in detail in Chapter 11 - most of the code discussed here can also be used with it. With that in mind, let's go - we'll read each other in the next chapter.

Chapter 3 • User interface and first experiments.

After the installation of OpenSCAD, we have arrived at the point where we can conduct our first experiments. This chapter wants to familiarise you with the user interface

The created objects are not complicated - advanced composite objects will be created in the next chapter. Nevertheless, there is already a lot to do here, so let's get to work.

3.1 OpenSCAD start screen

The first official action is to start OpenSCAD either from the Start menu or from the console. You will then see the start screen shown in figure one.

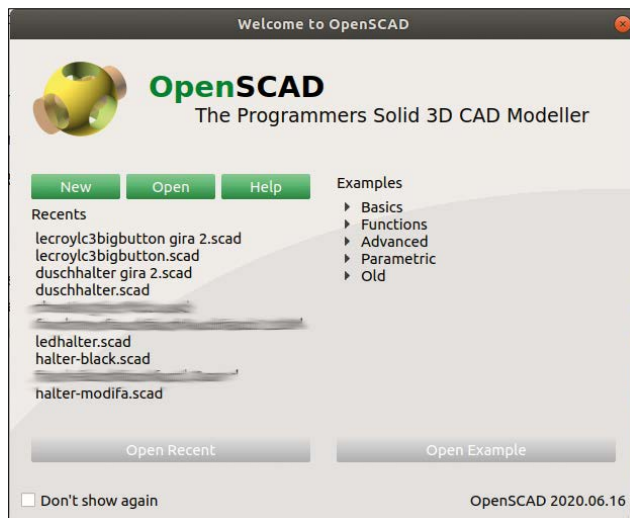


Figure 3-1. The OpenSCAD start screen serves as an introduction to the world of the modeling program.

The three green buttons at the top left allow direct access to commonly used functions. By clicking New, you can start working on a completely new OpenSCAD project, while the Open button starts a CommonDialog for loading existing files. The Recents field - partially censored in this screenshot for data security reasons – presents a list of OpenSCAD files that you used in the past and which can be quickly loaded again.

The Examples tree offers a group of ready-made program examples that illustrate both basic and advanced aspects of OpenSCAD. You can also deactivate the start screen using the checkbox shown at the bottom right - OpenSCAD then starts in editor mode mode.

Since we assume that your OpenSCAD is a virgin installation, click the New button to start the actual editor. It appears as shown in Figure 3-2.

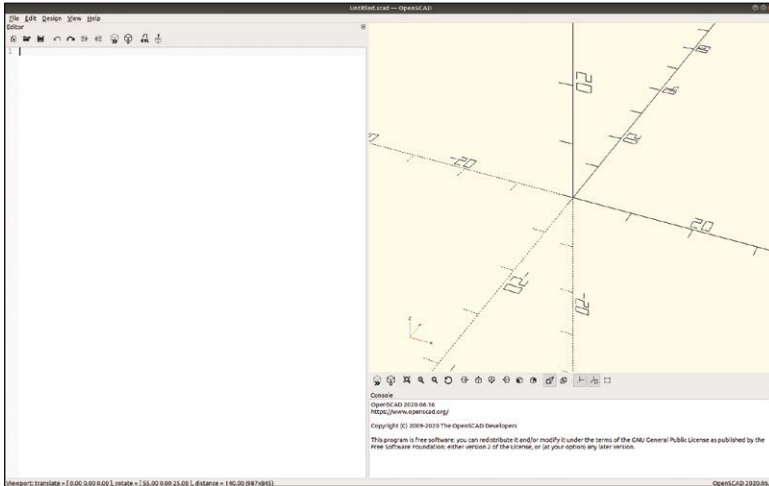


Figure 3-2. The OpenSCAD Editor allows for editing and rendering of the models.

The empty white area on the far left is the editor window in which we place the following code:

```
cylinder(12, 20, 10, center= true);
```

At the moment, accept the code as god-given - the only interesting thing is that we create an object. In OpenSCAD, objects are generated by calling the respective generator function; analogous to C or Java, a semicolon is placed at the end of each statement.

In the course of our work with OpenSCAD, we will get to know other commands, some of which do not require a semicolon. What is more interesting is to obtain a three-dimensional representation of the object. To do this, click the preview symbol highlighted in Figure 3-3.

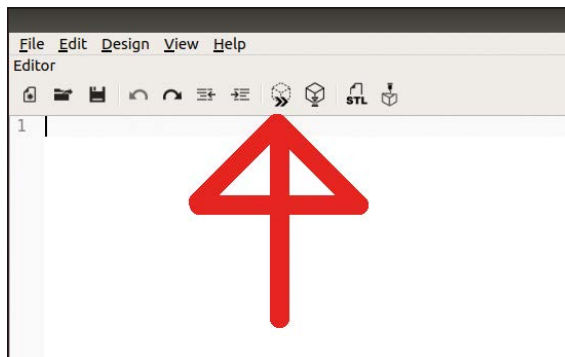


Figure 3-3. This toolbar icon commands a fast rendering of the object in the editor window.

A preview of the cylinder will then appear in the right-hand side preview window; figure four shows the rendering output on yours truly's box.

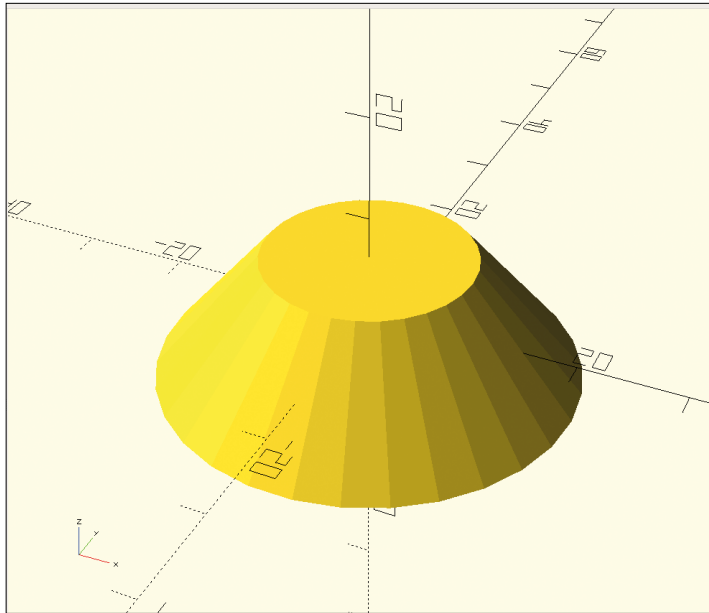


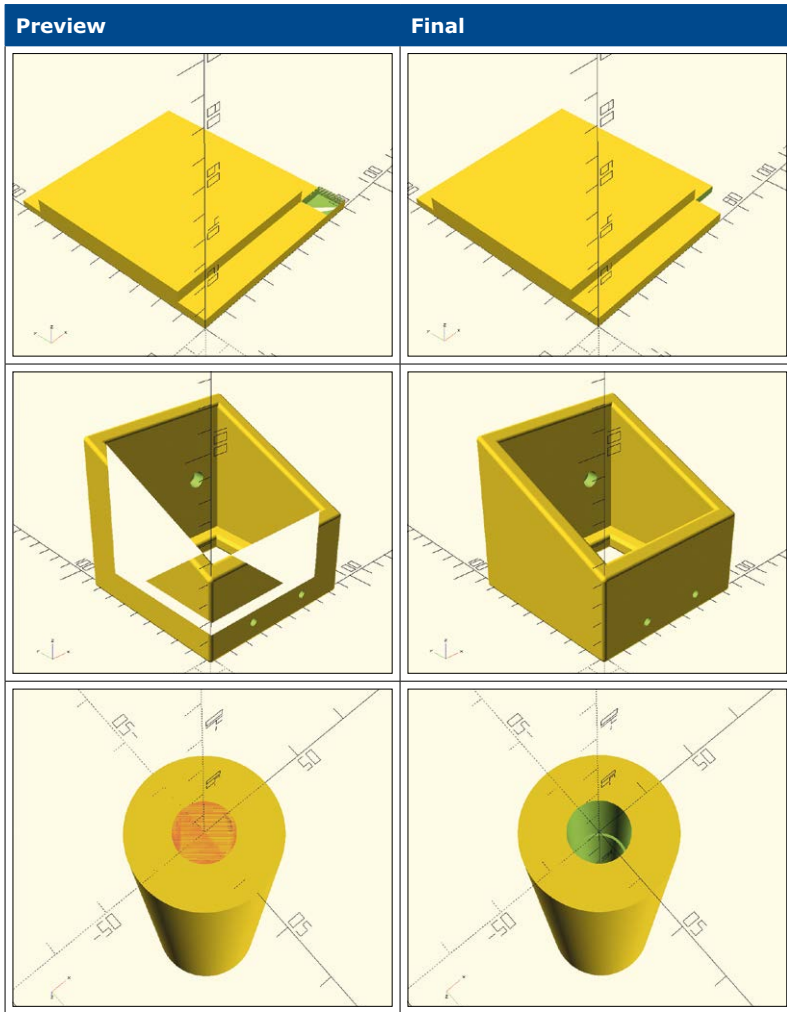
Figure 3-4. The object has appeared in the rendering preview.

Do not worry about jaggedness for now

If you look carefully at a "round" object like the cylinder shown here, you will notice that the edges are "stepped". This is not a problem at the moment and will be discussed later - if you want a smoother rendering immediately, see the **5.5 Holes, for the first** section for background information.

The division between preview and final production rendering is important in that OpenSCAD uses hardware acceleration for the preview. This leads to a much faster update of the display, but the results depend on the behaviour of your graphics accelerator and the corresponding driver and are therefore not consistent from system to system or from OpenSCAD version to OpenSCAD version.

What at first sight sounds like a semantic quibble, turns out to be a highly explosive problem in practice. Although preview rendering is much faster, it is not very accurate on many workstations, with cutouts being an especially hairy problem. What this means in practice can be illustrated in the table which compares some preview and normal renderings. Some very complex geometries cause the preview renderer to crash altogether.



From a certain degree of model complexity, it is therefore always advisable to create only final renderings. However, the point at which this changeover should take place is "hairy" - if you have worked with modeling in OpenSCAD for some time, experience shows that you will soon find the correct point.

With that all talk about final renderings, the question is how to make one. The answer lies in the symbol shown in Figure 3-5 - it commands a rendering to be performed exclusively on the main processor of your workstation. This is a "slower" process, but one which always produces consistent results. For this reason, the OpenSCAD development team also presupposes its use when you want to export an OpenSCAD file as an .STL file for a 3D printer.

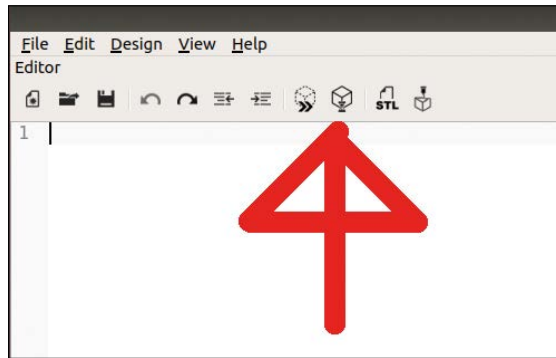


Figure 3-5. This button commands a rendering using the main processor.

The other toolbar icons displayed above the editor window are generally self-explanatory. For example, you can save your code in a .scad file, which can be reloaded using the Open command.

Incidentally, OpenSCAD uses ordinary text files as saving format - if you are dissatisfied with this editor for any reason, you can of course use another product.

3.2 Manipulating the Viewport

In the author's screenshot shown above, the generated object fits "perfectly" into the display area. In practice, this is rarely the case, which is why OpenSCAD users should get to grips with the manipulation of the drawing area as quickly as possible.

The most important control element is the coordinate cross, which is normally displayed on the left, and which provides information about the current orientation of the model. Click the left mouse button in the drawing area and keep the mouse button pressed to "rotate" the model by dragging and dropping.

If you want to move the model as in a translation operation, use the right mouse button. To zoom in and out, owners of a three-button mouse use the mouse wheel - those who, like the author, work with a Logitech MarbleMouse use the symbols displayed below instead, which functionally behave similarly. The OpenSCAD development team also provides a group of "advanced" mouse options, which are presented as shown in figure six.

Action	Icons	Description
rotating the view		Dragging with the left mouse button rotates the view along the axes of the viewing area. It preserves the vertical axis' direction.
		Dragging with the left mouse button when the shift key is pressed rotates the view along the vertical axis and the axis pointing towards the user.
moving the viewing area		Dragging with the right mouse button moves the viewing area.
		using the scroll wheel
		dragging with the middle mouse button
zooming		dragging with the right or middle mouse button and the shift key pressed
		the keys  and 
rotation reset		Rotation can be reset using the shortcut  +0.
movement reset		Movement can be reset using the shortcut  +P.

Figure 3-6. The OpenSCAD development team leaves the user free to choose the control method.

Advanced input devices.

When it comes to 3D modeling, people like to use joysticks, game controllers, or dedicated 3D mice. OpenSCAD also supports these, but the integration is somewhat labour-intensive. If you are interested in using such a device, you will find extensive information at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Input_Devices.

In practice, developers and users are quick to pick out the zoom function - just play around a bit to learn the "special features". More interesting are the additional symbols that are graphically highlighted in figure 3-7.

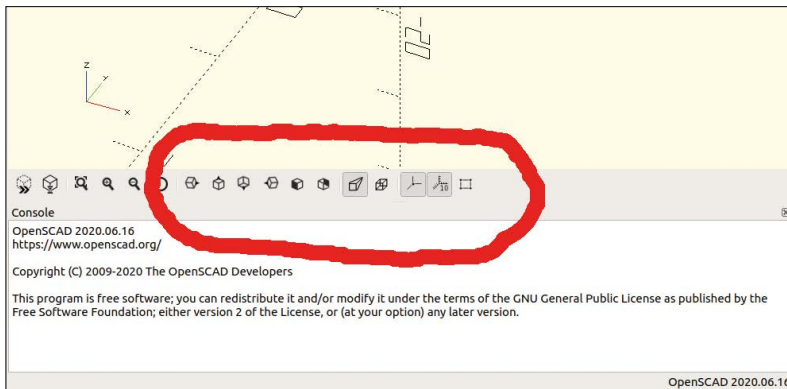


Figure 3-7. OpenSCAD contains some additional navigation options.

The first six icons allow you to load "defined" views of the model. The next two symbols switch between the display modes - the author generally prefers to work with perspective display, but those who prefer orthogonal display can activate it here.

The next two symbols allow the coordinate axes and the corresponding dimensions to be shown or hidden. Using the "last" symbol, a "combination mode" can be activated which displays model surfaces and associated edges together.

Finally, the View menu offers the option to select Wireframe to activate a wireframe view - it displays only the "defining" edges of the model, as shown in Figure 3-8.

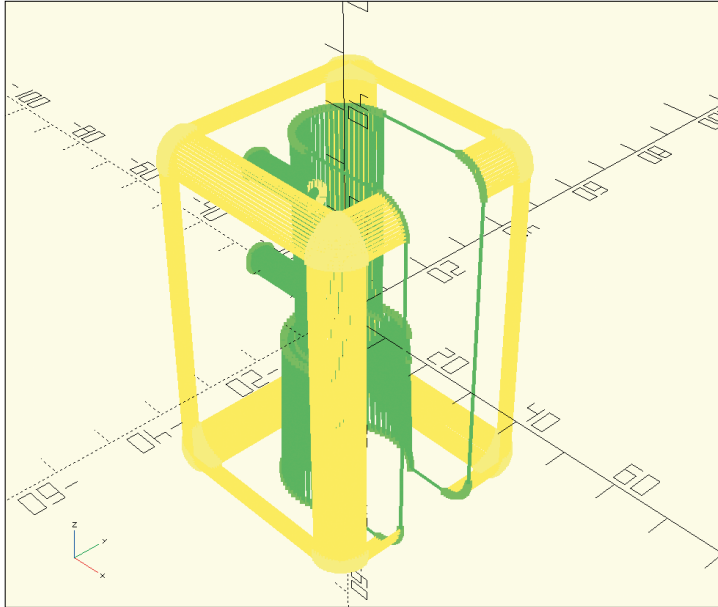


Figure 3-8. The wireframe display is sometimes useful.

Note, however, that it is not always available in conjunction with the view. For example, after requesting a preview rendering, the system is always in Surface mode; previews cannot be calculated in Wireframe mode.

3.3 More Settings

There is hardly a program whose development team doesn't get into trouble when selecting default settings - the best way to circumvent this problem is to offer a preference dialog, which in the case of OpenSCAD can be found under Edit → Preferences.

Before clicking on this window, however, take a look at the Increase Font Size and Decrease Font Size options, which allow you to increase or decrease the size of the font used in the Text Editor.

Be that as it may, clicking the Preferences menu will open a window where you can make various settings. By far the most interesting one is in the 3D View section, where you can select a group of different color schemes for the preview.

In the Editor field, you will find advanced" options which allow you to select, for example, the font to be used for code display. In the Features tab, you can activate various experimental options - compilation of an experimental OpenSCAD binary file is a prerequisite for this.

The remaining settings are best left as they are. If you would like more information about them, you will find a useful entry point at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/The_OpenSCAD_User_Interface.

3.4 Conclusion

Our small experiments with the cylinder used here may not be particularly complicated. However, we have gained valuable experience in working with the OpenSCAD user interface. In the next chapter, we will create "basic" three-dimensional objects and learn more about the algorithms contained in OpenSCAD.

Chapter 4 • Create, combine and move 3D objects

In the last few chapters, we used the OpenSCAD language to generate rectangles and pyramids. In this chapter, we want to take a closer look at these geometric primitives and learn more about how OpenSCAD does its work.

4.1 Hands hovering over the building surface

To understand how OpenSCAD works, let's take a closer look at the cube shown in figure one.

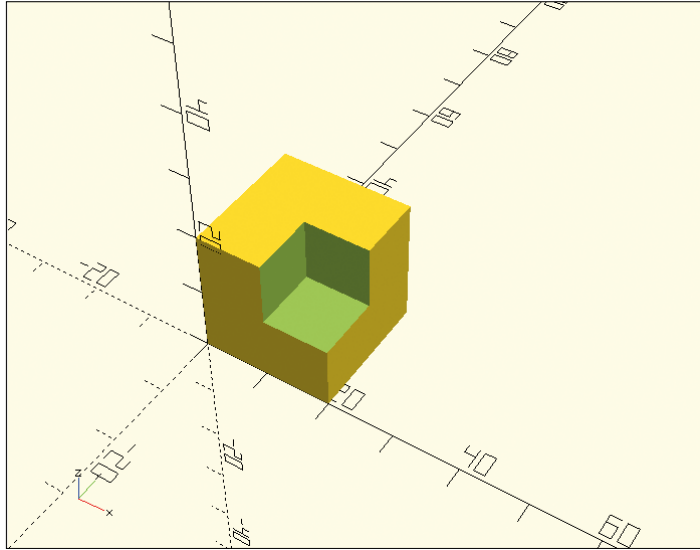


Figure 4-1. A simple object, but one that creates much joy.

The following code is required for its production:

```
difference(){
  cube(20);
  translate([10,0,10])
    cube(10);
}
```

A good analogon involves imagining how a CNC milling machine would manufacture the cuboid. In the first step the machine operator would load a "full" cuboid, which would be represented by the following code and would look like shown in figure two:

```
cube(20);
```

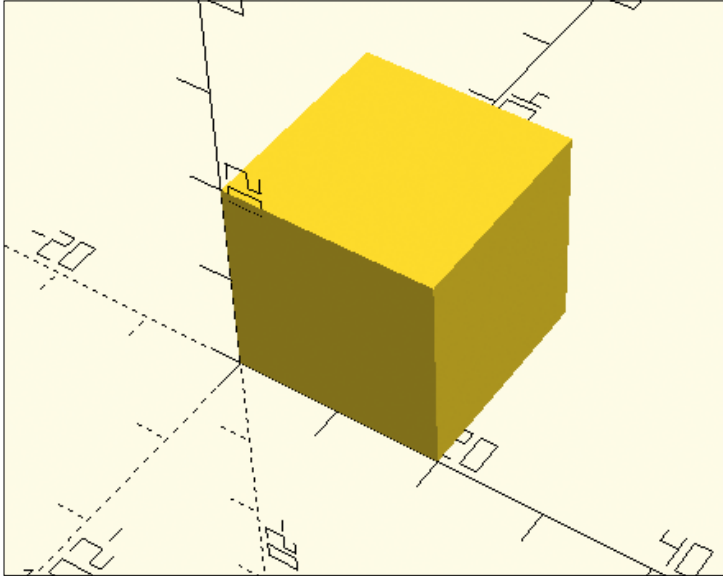


Figure 4-2. A full cuboid serves as the starting point.

Next, a "cut operation" would be performed, which would remove another cube from the upper right corner. The creation of this right cube could theoretically be imagined as shown in figure 4-3 and 4-4. In the first step, a cube is created in the origin, which is then moved to its destination by a translation command.

Trans-what?

The movement of an object along a vector in three-dimensional space is also known as translation - a term to remember as a 3D modeler.

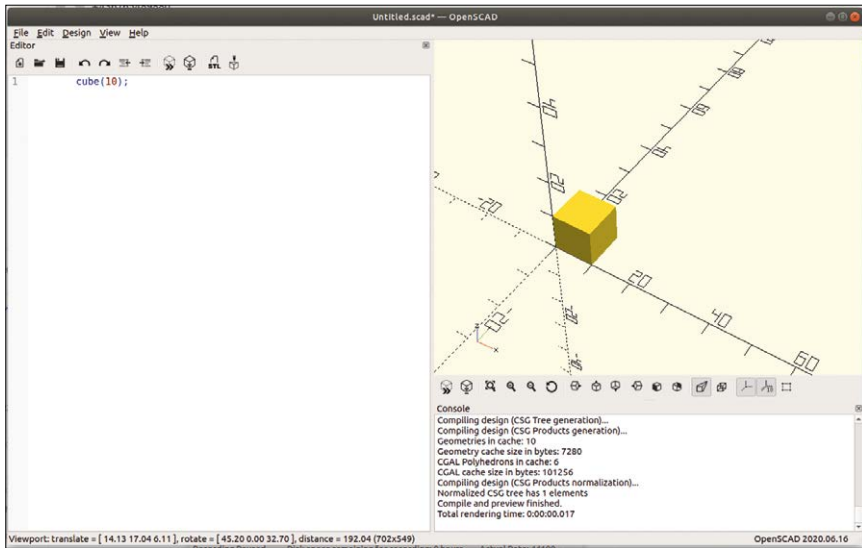


Figure 4-3. `cube(10);` takes care of the creation of the cube...

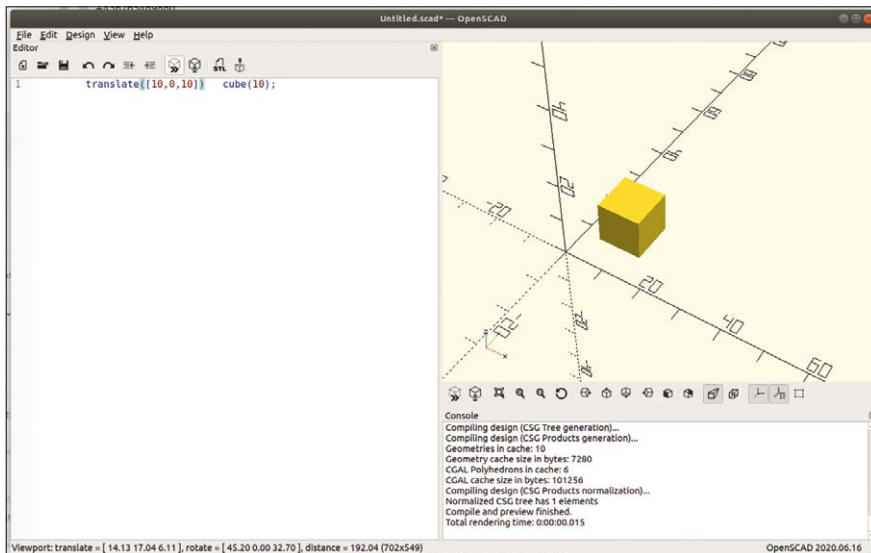


Figure 4-4. ...which is then pushed to its destination via `translate([10,0,10])`.

In OpenSCAD, the difference operator is then required to complete the final "subtraction" which the CNC machine would perform with its end mill. In short, OpenSCAD mirrors the processes a real-world machine would take to end up with a finished element – with the only difference being that it can always re-add material which was removed before.

An alternative mental model involves imagining the OpenSCAD engine as two hands hovering over the three-dimensional construction surface composed of tiny rectangles.

One hand can animate one of the "minimal voxels" by touching it, while a touch of the other hand ensures that the voxel becomes "empty". The OpenSCAD programmer creates a series of commands, which are run to spawn a voxel cloud, which is finally exported to the target format.

If you're looking for an even more rustic approach to the subject, you might also think of the Pokémon shown in Figure five. It has two hands with which it interacts with the building surface by creating and removing elements.



Figure 4-5. Far-fetched, but a useful analogue: Alpollo (© Pokémon · Nintendo · Creatures · GAME FREAK; via https://www.pokewiki.de/Datei:Sugimori_093.png).

4.2 Create cuboids: data types and more.

In the course of recent experiments we have repeatedly used the Cube function. Let us take a closer look at it - we find the following two declarations in the official syntax list available at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Primitive_Solids#cube:

```
cube(size = [x,y,z], center = true/false);
cube(size = x ,      center = true/false);
```

OpenSCAD is full of methods that accept multiple parameter sets. This is - analogous to the overloading process found in C++ - a way to let you invoke a generator with a parameter set best suited to the situation at hand.

Let us start with the variant `cube(size = [x,y,z], center = true/false)`, which takes two named parameters, namely `size` and `center`. The brackets around the `size` parameter inform us that this parameter is expected as a vector.

OpenSCAD differs from other programming languages in that there is a dedicated vector representation. A vector is but a collection of "numbers", which - see figure six - are regarded as a "single data element", analogous to a classical array or a mathematical vector.

$$val = \left(\frac{\frac{X}{Y}}{Z} \right)$$

Figure 4-6. Vectors combine numerical values – such as three coordinates describing a point in 3D space - into an easy-to-handle structure

The vector passed to size is 3 values high and determines the extent of the box in the direction of the X, Y, and Z axes. Parameter number two, center, is declared false by default (see figure seven).

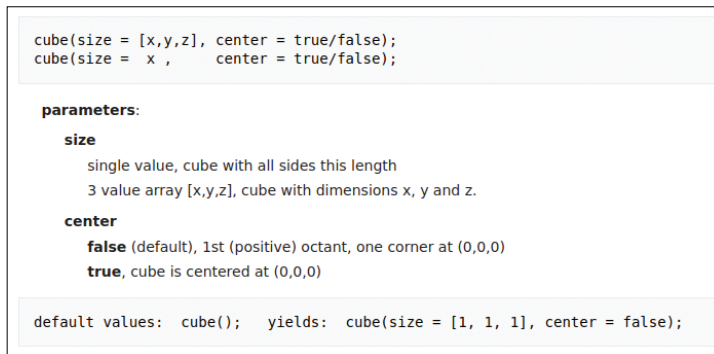


Figure 4-7. The OpenSCAD documentation often places the default values below the BNF description of the commands at hand.

Center defines where the "center of mass" of the created cuboids should be located. This is also a topic that can be illustrated most easily by two illustrations. Please note that the position of the cuboids relative to the coordinate origin changes depending on the Center value.

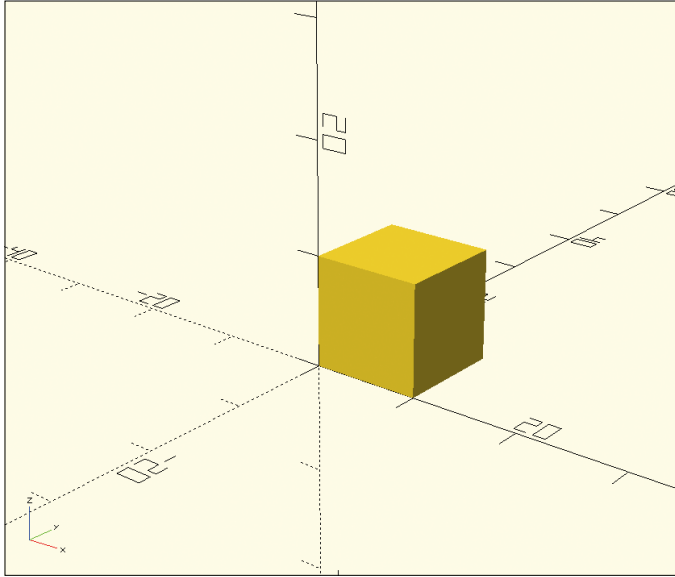


Figure 4-8. `cube([10,10,10], false);` and ...

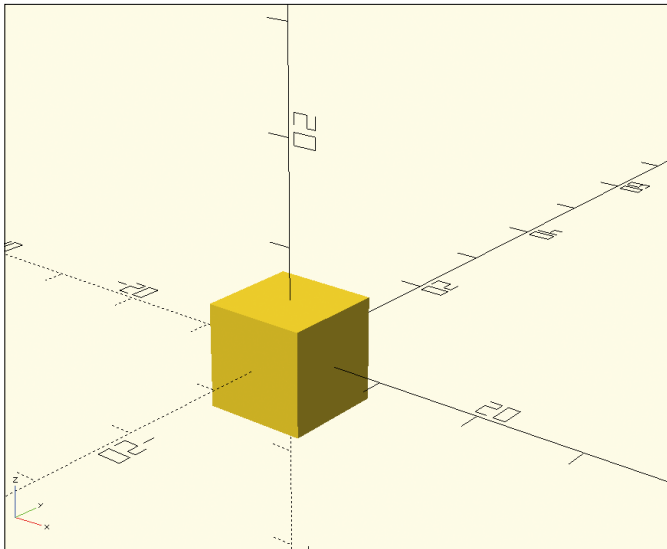


Figure 4-9. `cube([10,10,10], true);`

The simpler version of the cube command takes a numeric value in the size field, which is applied to the X, Y, and Z-axis. If you want to create a cube measuring $10 \times 10 \times 10$ units, you can do so using one of the following methods. Both statements would produce the same cube:

```
cube([10,10,10]);
cube(10);
```

By the way: if you want to ensure that two elements produce identical results, simply place them one below another in a file and command a rendering. Then take turns switching off one of them while commanding rendering processes - if the preview window does not change, you have an identical result (for simple geometries).

This, of course, is no longer the case if the elements differ from each other internally. In this case, it is advisable to perform comparisons via the difference operator which we will get to know later in this chapter.

Units?

OpenSCAD coordinates are - per se - dimensionless. We will deal with this question in more detail in chapter five - for the moment, let us assume that ten is equal to a "distance of ten units on the screen".

4.3 Sphere and cylinder

Those who are comfortable working with cuboids can realize a nice number of "technical" components - in the author's company, many commercially used objects consist exclusively of rectangles. In practice, one often wishes for something round - be it a sphere to balance something; or be it a cylinder that serves as a location for a screw.

Although there are a few things to consider when working with cylinders and spheres, which I will introduce to you in chapter five, we will work with the basic generator functions here. Analogous to the cuboid used above, a generator function spawns spheres or cylinders and must be terminated with a semicolon. In the documentation, we find the declarations shown in figure 4-10 for the sphere; a BNF form is not found here.

sphere [\[edit \]](#)

Creates a sphere at the origin of the coordinate system. The *r* argument name is optional. To use *d* instead of *r*, *d* must be named.

Parameters

r
Radius. This is the radius of the sphere. The resolution of the sphere is based on the size of the sphere and the \$fa, \$fs and \$fn variables. For more information on these special variables look at: [OpenSCAD_User_Manual/Other_Language_Features](#)

d
Diameter. This is the diameter of the sphere.

\$fa
Fragment angle in degrees

\$fs
Fragment size in mm

\$fn
Resolution

default values: sphere(); yields: sphere(\$fn = 0, \$fa = 12, \$fs = 2, r = 1);

Figure 4-10. The sphere function is declared graphically.

The definition of the sphere function has parameters named *R* and *D*, which are both "numeric". According to the documentation, *R* is the default value, so calling `sphere(10);` will result in a sphere with a radius of 10. If you want to define the diameter instead, you must declare the following:

```
sphere(d=10);
```

Rendering round objects requires significant computing performance, due to which OpenSCAD provides a set of "dollar values" which let developers trade off between performance and rendering quality. We will discuss them soon – for now, let's simply run the following command:

```
sphere(10);
```

You can see the result shown in Figure 4-11 by clicking on the Render button.

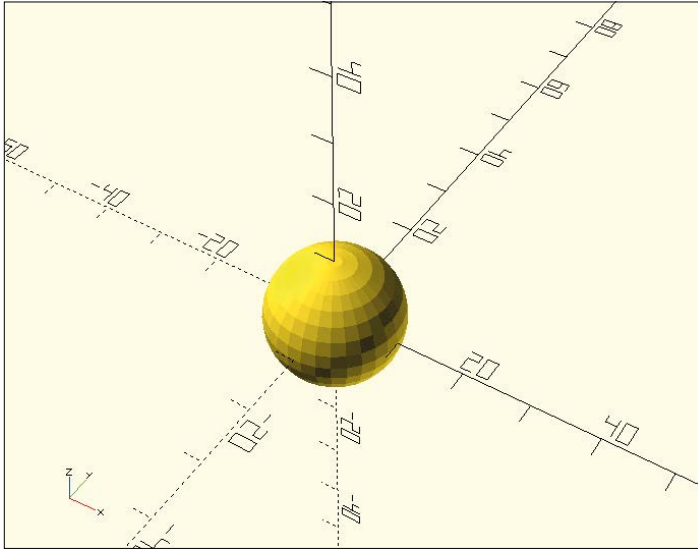


Figure 4-11. Our sphere extends from -10 to +10.

Incidentally, note that the OpenSCAD parser performs a syntax check both when you click Render and when you click Preview. If an error occurs, this is indicated in the code editor by a red circle, as shown in Figure 4-12, and the console window by the output of a red error message.

Note that the contents of the preview window will not be updated or deleted in this case - instead, it will display what was left in the program's memory from the last "valid" rendering run.

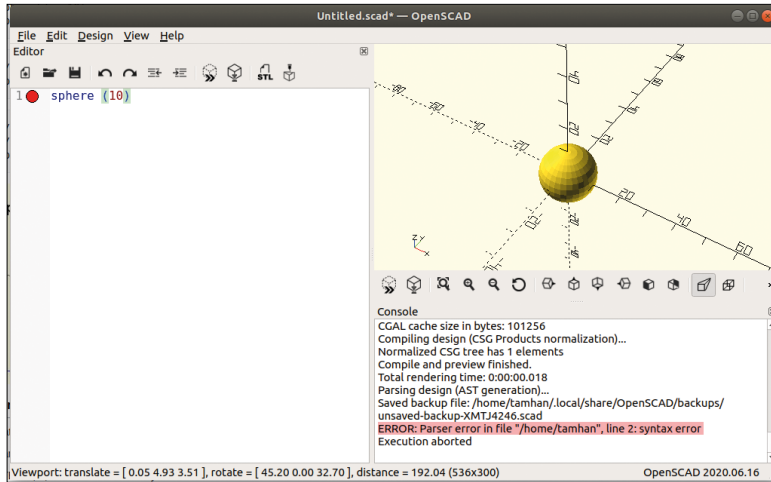


Figure 4-12. If you forget a semicolon, OpenSCAD will sound the alarm!

Next, we want to look at the cylinder function, which is defined according to the following scheme:

```
cylinder(h = height, r1 = BottomRadius, r2 = TopRadius, center = true/
false);
```

In principle, three values are required to create a cylinder - in addition to the height that is normally centered in the Z-axis, the lower and upper radius of the boundary circles must be defined.

If you provide but two values according to the following scheme, you define the height and a radius instead, which is then applied both above and below:

```
cylinder(20, r=5, center = true/false);
```

Please note that using a "single-value" radius requires you to write `r=` as prefix - if this is not done, a "pointed cylinder" is created, because OpenSCAD stores the second value in the slot for R1 and sets R2 to zero. It should be noted that both the `R1>R2` and `R2>R1` states are permissible.

Also, there is a `center` parameter that determines whether the "center of altitude" is located at the coordinate origin - the effect of the operation can be determined most easily using Figures 4-13 and 4-14.

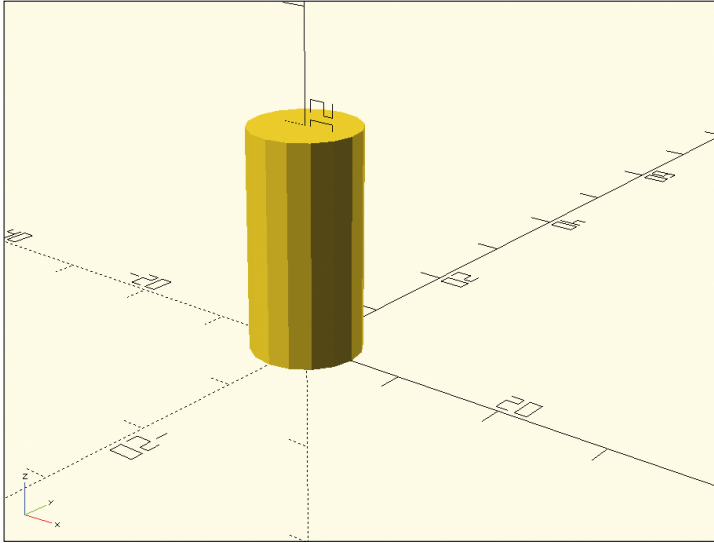


Figure 4-13. `cylinder(20, r=5, center = false);` and

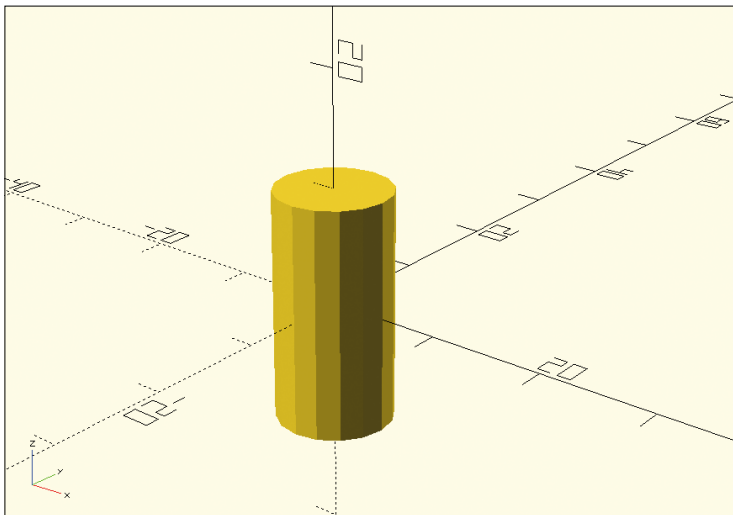
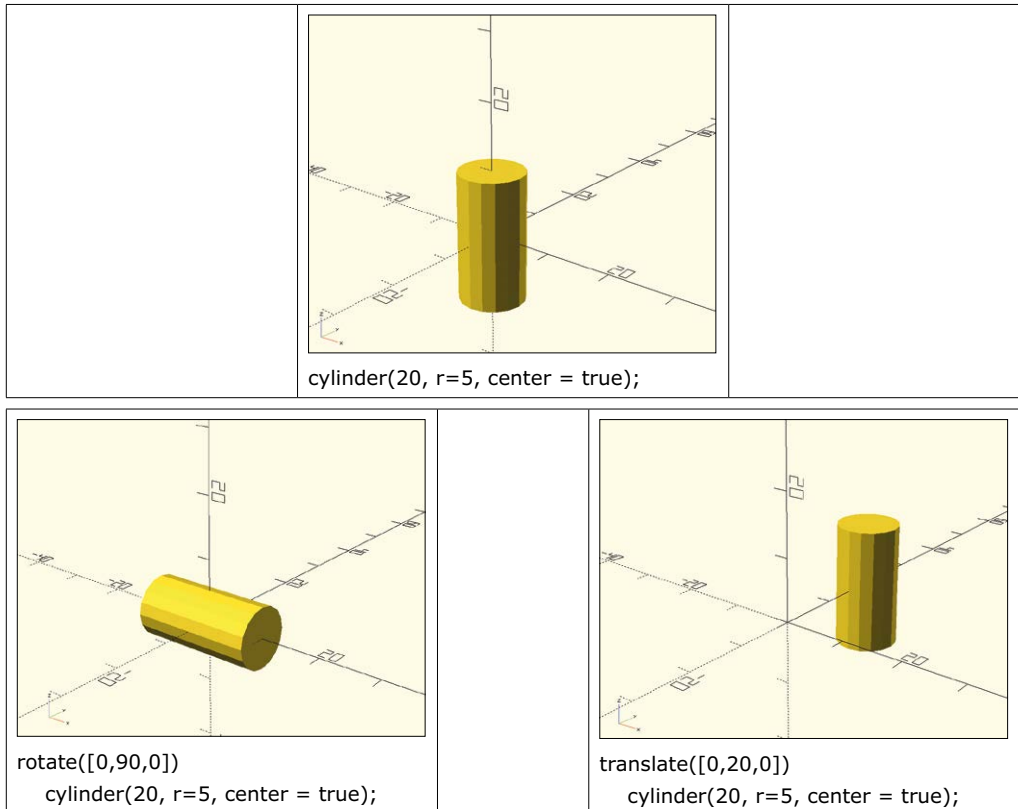


Figure 4-14. `cylinder(20, r=5, center = true);`

4.4 Translation operators move objects

Our previous experiments with the center parameter allowed us to move the objects within a limited range. To realize more complex objects like the cube mentioned above, we have to move elements freely in space.

The OpenSCAD cheatsheet knows a good dozen of transformations, but a good part of them do not play a major role in practice. Translation and rotation are particularly important - the effects of both are shown in the three screenshots.



With this, return to the declaration in BNF form. Let us start with the translation, for which the following forms are envisaged:

```

translate(v = [x, y, z]) { ... }
translate([x, y, z]) { ... }
translate(v = [x, y, z]) object();
translate([x, y, z]) object();

```

Translate is available in four different versions. Let's start with the version that always takes a vector called *v*. It contains the "distance values" by which the object must be moved in the direction of the respective axes.

Incidentally, it is not necessary to prefix the parameter name *v*, as OpenSCAD implies it - in virtually all OpenSCAD files known to the author, Translate simply gets a vector enclosed in square brackets and is "happy" with it.

More interesting is the type of payload passed: in addition to a single object represented by its generator function, you may also combine a group of objects. This is done by using curved brackets, which declare blocks in a fashion similar to Java or C.

Last but not least it should be noted that operators can also be concatenated. If you want to perform two translation operations after each other, code constructed according to the following scheme is perfectly legitimate:

```
translate([9,9,0])
  translate([0,9,0])
    cylinder(20, r=5, center = true);
```

Incidentally, OpenSCAD is uncritical in terms of whitespace - it would also be permissible to arrange the three statements in a single line or to indent and outdent with tabs as desired. Although OpenSCAD is more tolerant than programming languages such as Python, the author's practical experience shows that it is advisable to use "reasonable" indentation. OpenSCAD files have the unpleasant property of becoming very complicated sooner or later. It should be noted that there are two buttons in the toolbar above the editor which allow you to "move" a marked block to the left or right - it is advisable to use these if you need to "reformat" an existing file.

Ellipses are created via distortion

In the interest of flattening the learning curve, the author does not discuss the creation of elliptical bodies at this point. They are created by a variation of the translation, which has different effects on the individual axes. If it is absolutely necessary to create an ellipse at this point, take a look at the relevant sections of chapter six.

A translation moves the object by the amounts specified in the vector - if you call `translate([9,9,0])`, the object moves both along the X and the Y-axis by the value 9. It should be noted that you can also use negative values - in this case, the object moves in the direction of the "negative" parts of the axis, which are displayed as dashed lines in the OpenSCAD preview window.

The prospective OpenSCAD modeller has several procedures at his disposal for finding the correct translation value. It follows from logic that one should have a "real example" of what you seek to implement - in this case, you can use a caliper to find out relevant dimensions.

If for some reason, you do not have one or if it is not sufficiently accurate, nothing prevents you from "playing the problem to death". As long as you do not perform any complicated operations, OpenSCAD rendering runs are very fast - simply adjust the parameters in an iterative process until you get a satisfactory result.

4.5 Rotation operators rotate objects

The next operation is rotation. It - logically - does not move an object, but rotates it "around an axis". When looking for the rotate operator's definition, the following candidates crop up:

```
rotate(a = deg_a, v = [x, y, z]) { ... }
rotate(deg_a, [x, y, z]) { ... }
```

```
rotate(a = [deg_x, deg_y, deg_z]) { ... }
rotate([deg_x, deg_y, deg_z]) { ... }
```

In the interest of a more compact text, we only print the versions with a block payload here. It follows from logic that the command can of course also be applied to discrete objects or the results of another operator – in this aspect, it behaves analogously to the translate operation which we just discussed.

The most commonly used version is `rotate([deg_x, deg_y, deg_z])`, which accepts a trivalent rotation table and executes it according to the following scheme

```
rotate(a=[0,0,az]) rotate(a=[0,ay,0]) rotate(a=[ax,0,0]) {...}
```

The versions with one angle value and one vector allow an "alternative" as notation according to the following scheme:

```
rotate(a=180, v=[0,1,0]) { ... }
```

This command would cause a rotation around the Y-axis, rotating the object 180° - by setting the respective axis value in `v` to 1, you specify that this axis should be affected by the rotation angle passed in via the `A` parameter.

In practice, you almost always encounter the variant with a vector - we present the alternative variant mainly to cover you if you ever catch it "in the wild".

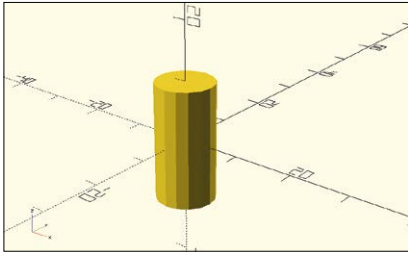
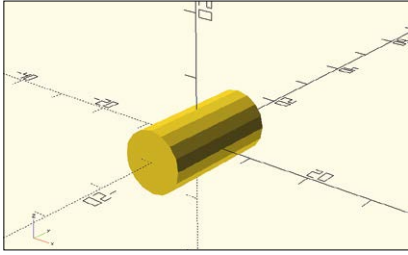
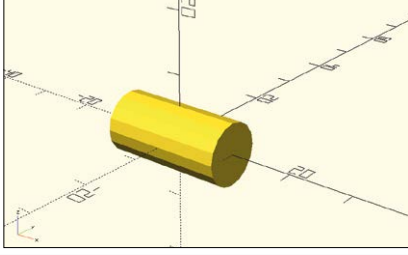
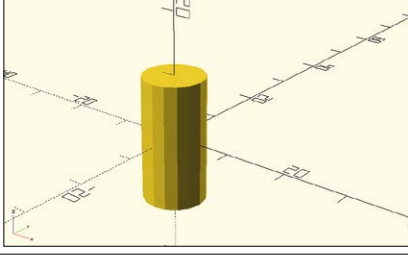
When working with "completely two-dimensional" objects, you will encounter calls according to the scheme `rotate(number)` - but this should not stop us at this time.

More interesting is the question, in which direction a "positive" rotation occurs. Use the Right-Hand Grip Rule to answer this question. The idea behind this is to imagine that the thumb of the right-hand points in the direction of the positive "correlation axis" - the angled fingers then show in which direction a positive rotation occurs (see figure).



Figure 4-15. The rule of the right-handed grip helps to estimate the turning sequences (picture source: https://commons.wikimedia.org/wiki/File:Right-hand_grip_rule.svg).

For the "correlation" between the axes and the positions in the rotation vector, please refer to the table.

<pre>rotate([0,0,0]) cylinder(20, r=5, center = true);</pre>	
<pre>rotate([90,0,0]) cylinder(20, r=5, center = true);</pre>	
<pre>rotate([0,90,0]) cylinder(20, r=5, center = true);</pre>	
<pre>rotate([0,0,90]) cylinder(20, r=5, center = true);</pre>	

Degrees? Yes, degrees!

Although the calculation software serving as the OpenSCAD engine works with Radians internally, the OpenSCAD development team decided - probably in the interest of ease of use - to express rotation and other angles in degrees in the entire scripting language. Proposals for changes in this respect, such as the one at <http://forum.openscad.org/Degrees-instead-of-radians-td7398.html>, have so far been rejected. However, it should be noted that it is not particularly complicated to convert degree values to radians.

4.6 Color parts of objects

The OpenSCAD development team planned a whole group of additional functions that were of little use for technical designs and which - thanks to a change of government - were only partially implemented.

One of these is the ability to assign "colors" to objects. This feature is of little relevance to 3D modelers, since STL files do not have a way to describe the "coloring" of an elemental voxel at the time of printing. However, the feature still has a right to live because it allows "highlighting" parts of a design. A classic example would be the coloring of individual modules to make the overall composition of a system more visible.

Not suitable for dual extruders.

Even though Geeetech's now-legendary failure with the Prusa I3 ProC did immense damage to the reputation of 3D printers with multiple extruders: when bought from a well-known manufacturer, the devices usually work without any problems.

Unfortunately, the color system is not a viable way of marking individual extruders - anyone who owns such a printer model will find more information on how to realize object geometries at <https://hackaday.com/2017/01/20/multiextrusion-and-openscad/> and <https://3dprinting.stackexchange.com/questions/4291/convert-an-stl-model-to-a-two-extruder-model>.

Before introducing the Color operator, we would like to point out that an alternative command that allows "short-term" highlighting of individual model parts will be presented in the section on debugging tools.

Be that as it may, we will focus on the operator for now. Its normal form definition looks like this:

```
color( c = [r, g, b, a] ) { ... }
color( c = [r, g, b], alpha = 1.0 ) { ... }
color( "#hexvalue" ) { ... }
color( "colorname", 1.0 ) { ... }
```

OpenSCAD is very flexible regarding the format in which colour information is to be delivered. In addition to numerical values with and without an alpha value, it is also possible to pass hexadecimal values or even color names known from HTML and summarized in Figure 4-16.

Information only visible in previews.

In most versions of OpenSCAD, "final" models always appear in the colour scheme specified in the Preferences dialog: the colour parameters are only used during preview renderings of the model.

Figure 4-16. Web designers know these color names for sure.

Figure 4-17 shows that the two elements appear on the screen.

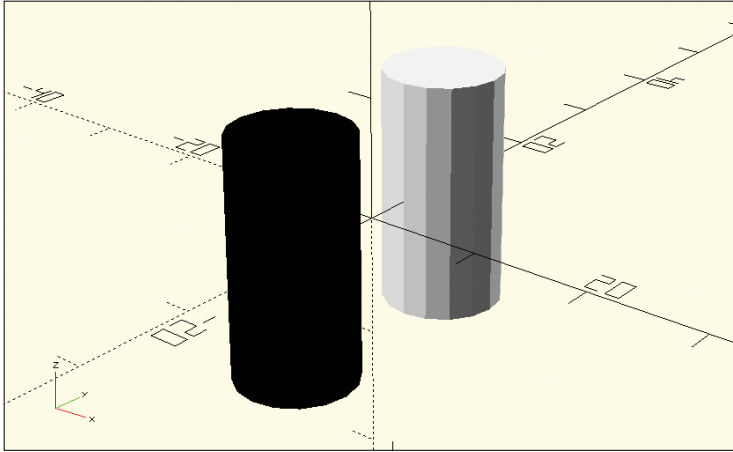


Figure 4-17. Black and white cylinder side by side.

Note that the color operator is commutative. You can place it anywhere in the operator's pipeline, and the result remains the same.

4.7 Linking translation and rotation

This raises the question of whether objects can be rotated and moved at the same time. The answer is yes, as partly stated above. However, the open question is whether translation and rotation are commutative or not.

Remember: an operation is considered commutative if the order of its parameters can be exchanged without affecting the result. A classic example would be multiplication - $a*b$ and $b*a$ return an identical result, so the operation is commutative.

The simplest way to check the behavior is to create two objects as above and swap the order of the operators once per creation operation. If the two are "on top of each other", operations are commutative - if we see two elements, the operation is not commutative. To keep the example "compact", the author would like to anticipate advanced experiments with the description language at this point and declare a module:

```
module tamsDreidel(dia=22)
{
  cylinder(dia, dia, dia, true);
  cylinder(dia*2, dia/9, dia/4, true);
}
```

Modules are "handmade" generator functions that conjure up objects analogous to the generators discussed earlier in this chapter. `tamsDreidel` creates a kind of "spinning top" - it is not about winning design awards, but about realizing a highly visible object.

The following declaration can be written in the next step:

```
color("white")
  translate([0,20,0])
    rotate([45,0,0])
      tamsDreidel(5);
color("grey")
  rotate([45,0,0])
    translate([0,20,0])
      tamsDreidel(5);
```

In addition to swapping the translate and rotate operations, it is important that the author selected grey instead of black for the second element. The reward for the effort is that the contours of the cylinder do not "sink" as much in the monochromatic print. Either way, the result of a program execution then presents itself as shown in Figure 4-18.

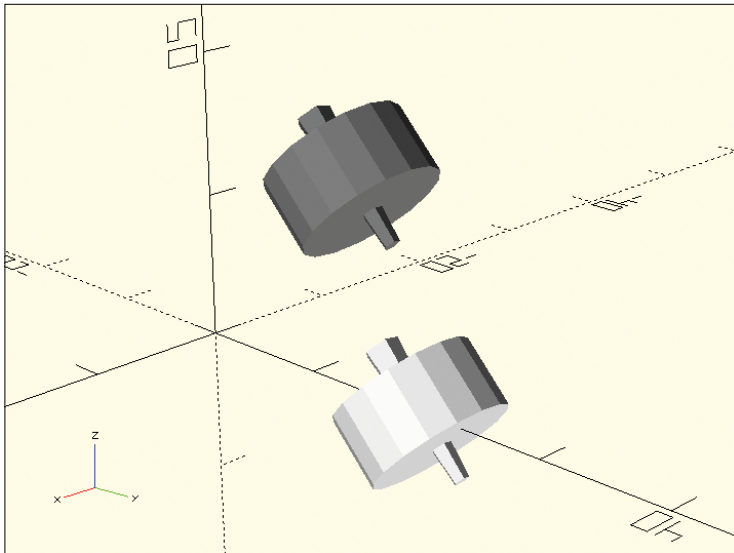


Figure 4-18. Since two dreidels are created, we know that translate and rotate are not commutative.

As we want to understand the problem, we have to look at the coordinate system in the figure. In both cases, the translate operations perform a shift along the X-axis, which is clearly visible in the coordinate system. It follows that the difference must be in the rotation. Instead of having a "global" coordinate system as before, we now assume that every element geometry has another coordinate system - this is called a model coordinate system in the world of computer games.

In a "logically" constructed world, translations would grab the center of the model coordinate system and move it. In the interest of "more efficient" execution of the various geometric operations, this procedure is rarely found in computer systems. Instead, rotation

is always calculated using the origin of the coordinate system of "the entire scene" - in our case this would be the center 0|0|0.

But: since the translation "shifts" the individual vectors, a rotation performed after a translation sees a completely different object, which - logically - behaves differently when subjected to rotation.

For the actual resolution of "nested" operations, OpenSCAD uses the right-before-left method. An "inner" operation is carried out before the "outer" one is executed - this can be checked using the two objects. The white object lies on the Y-axis - let us now look at it again, as shown in Figure 4-19.

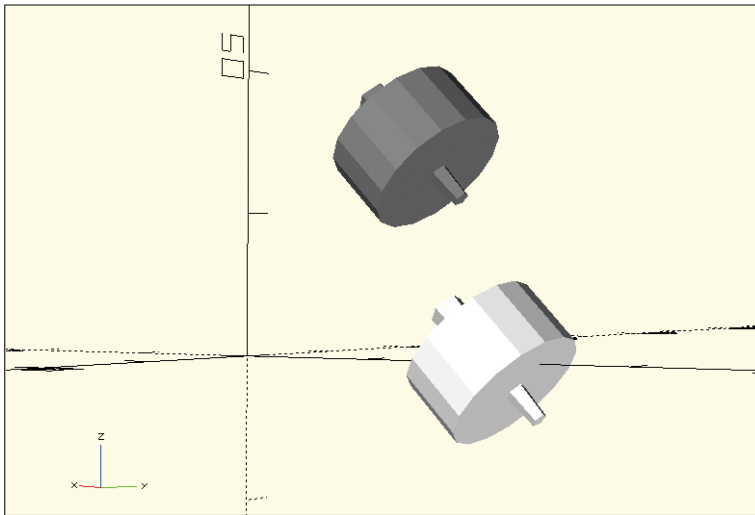


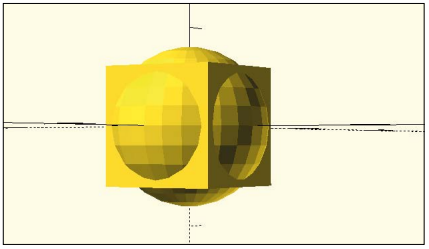
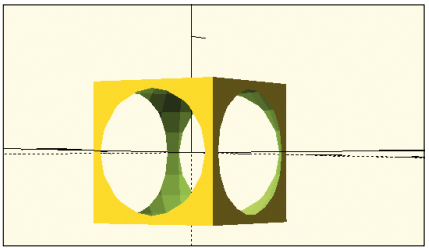
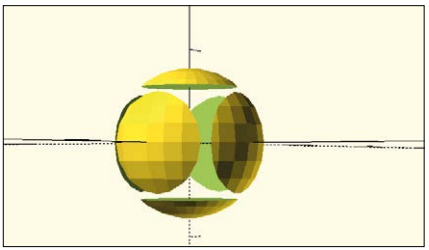
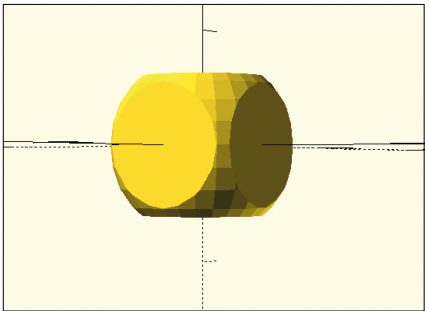
Figure 4-19. Changing the perspective helps us understand the operator concatenation.

This is because the dreidel is rotated in the first step, and only then is shifted. The other object is moved first and rotated afterwards - since the distance on the Y-axis is also rotated when the rotate operation is applied, it is "levitated" upwards like on a thread.

4.8 Combine 3D objects smartly

Summoning several objects gives you a summing object - every elemental voxel touched by at least one of the objects appears as set. In practice, this is not always desirable - if we want to create a screw hole, a subtraction of the cylinder from the rest of the body would be necessary.

OpenSCAD has a total of four different operations in the "Combinations" area - the procedure previously accepted as the "standard" is called Union. As so often in the world of OpenSCAD, the easiest way to understand this problem is to look at some results - the table shown here illustrates the four operations using the combination of a cube and a sphere. The difference operator must appear twice because it implements a non-commutative operation (namely subtraction).

<pre>union() { cube(12, center=true); sphere(8); }</pre>	
<pre>difference() { cube(12, center=true); sphere(8); }</pre>	
<pre>difference() { sphere(8); cube(12, center=true); }</pre>	
<pre>intersection() { sphere(8); cube(12, center=true); }</pre>	

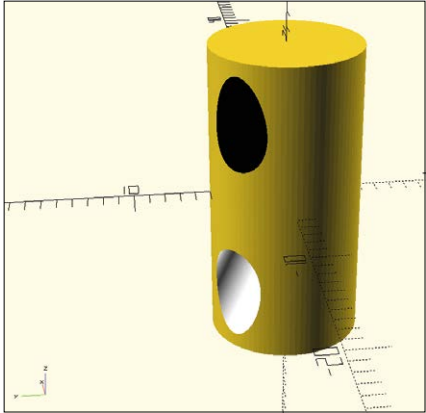
The behavior of the union operator is simple - it "combines" the geometric objects by an addition. This means that any voxel touched by at least one of the two objects is assumed to be touched or set in the output element.

The difference operator works the other way round - it first conjures the first object passed and then subtracts the "following" one. This means that only those elements remain set that are set by the first, but not by the second object.

Finally, there is the intersection operator, which only sets those voxels that are marked as to be set by both the first and the second object.

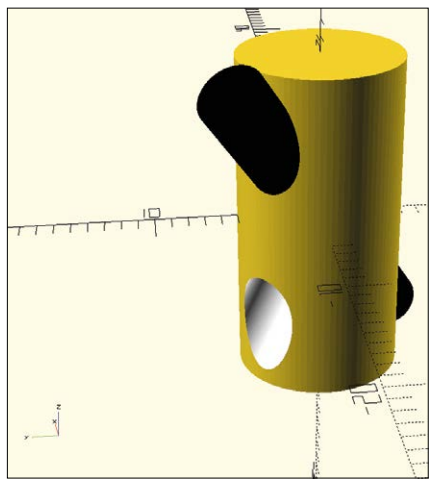
We have already noted above that the "stupid" juxtaposition of objects in many cases leads to OpenSCAD combining them. To demonstrate the problems, we would like to use an example provided by the OpenSCAD development team, which the author has refined a little.

In principle, the file consists of three cylinders with different rotations, which are accommodated in a difference operator. The table shows the three different possible combinations - pay particular attention to whether the white and black cylinders represent a hole or "an extension" of the existing geometry.

<pre> \$fn=90; difference(){ //Object A cylinder(r=5,h=20,center=true); //Object B rotate([00,140,-45]) color("black") cylinder(r=2,h=25,center=true); //Object C rotate([00,40,-50]) color("white") cylinder(r=2,h=30,center=true); } </pre>	
<pre> \$fn=90; difference(){ { //Object A cylinder(r=5,h=20,center=true); //Object B rotate([00,140,-45]) color("black") cylinder(r=2,h=25,center=true); } //Object C rotate([00,40,-50]) color("white") cylinder(r=2,h=30,center=true); } </pre>	No changes

```
$fn=90;
difference(){
  union(){
    //Object A
    cylinder(r=5,h=20,center=true);

    //Object B
    rotate([00,140,-45]) color("black")
    cylinder(r=2,h=25,center=true);
  }
  //Object C
  rotate([00,40,-50]) color("white")
  cylinder(r=2,h=30,center=true);
}
```



There are several interesting things here: firstly, the difference operator can also accept multiple objects. In this case, the first is assumed to be a positive form, from which all subsequent ones are subtracted. Inserting an empty block, which is placed in curved brackets, does not change this behavior at the time of printing. A change in behaviour is only achieved when we additionally use the union operator.

The sometimes senile author recalls that this behavior was different in the past. For the sake of consistency alone, it is, more than advisable to use a union operator when trying to combine the results of a group of primitives or module calls for another operator.

4.9 Outlook: Realisations, polyhedra, and projections

Congratulations - after working through this chapter, you have the tools to create basic elements with OpenSCAD. For example, the object built in the context of the following Worked Example has been working in the author's company for years to the satisfaction of all those involved.

In the field of 3D objects, there is another type of body called a polyhedron. We will look at it in a subsequent chapter due to its complexity. Before that, however, we will look into the question of how three-dimensional objects can be realised and why round elements look so "jagged"!

Chapter 5 • Realize and understand (round) objects

In this chapter, we won't be getting to know any new features, but instead we'll look at objects and how they interact with 3D printers.

5.1 Edges and corners

In the last chapters, we had to deal with round objects again and again. Depending on their size, they appeared more or less strongly angular. The reason for this peculiar behaviour lies in the underlying rendering engine of OpenSCAD, which approximates objects using a "line trail".

The reasoning behind this procedure is that calculations against two-dimensional program headers are easier to perform than if OpenSCAD had to permanently perform a "full-fledged" calculation against a circle.

To help you understand the thought processes behind this, we would like to tell you a little joke that the author was told in a first-class lounge many years ago - at that time, yours truly was still but a cadet:

A mathematician and an electronics engineer sit together in a first-class lounge. They are joined by a chubby little chick, who tells them they should approach her. However, the approach has to be gradual - each step reduces the distance by half.

While the mathematician despairs because of this situation, the electronics engineer decides that 0.001mm distance is more than close enough - good enough is good enough after all.

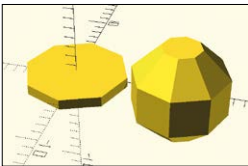
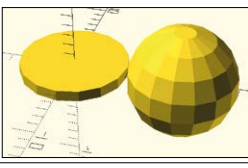
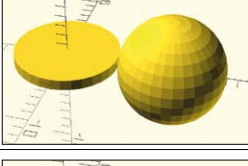
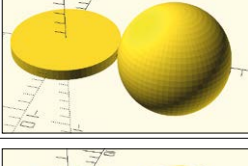
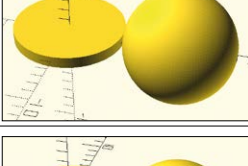
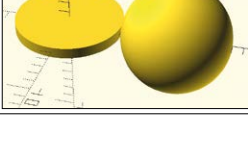
The point of this little story is that it makes no sense to demand "excessively high" rendering accuracy. Being too accurate does little except waste CPU cycles, after all.

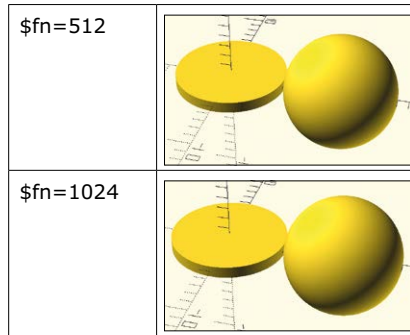
OpenSCAD solves this problem in that it lets the designer of the model determine the behaviour via a group of variables. Specifically, there are three different variables, which I will briefly introduce to you in the table. In practice, you almost always work with \$fn, so the (somewhat confusing) descriptions of the other variables are not overly important.

\$fa	Minimal Angle behind an individual fragment. The maximal number of segments in a circle is then determined via $360/\$fa$.
\$fs	Minimal size of an individual fragment.
\$fn	Overwrites both \$fs and \$fa, thereby allowing you to directly set the number of segments that are to be used in a circle. Best numerical stability can be achieved if the value passed in is divisible by four.

In practice, there is no need to bother with this - the author simply uses the value `fn` to command a model-wide specification of rendering accuracy. The following table uses the following example program to create a table demonstrating the impact of `$fn` on an example rendering:

```
$fn=8;  
circle(5);  
translate([10,0,0])  
  sphere(5);
```

\$fn=8	
\$fn=16	
\$fn=32	
\$fn=64	
\$fn=128	
\$fn=256	

**Preview rendering only!**

2D and 3D objects must not be mixed in one file - if you do this, you will get an error along the following lines:

```
WARNING: Mixing 2D and 3D objects is not supported, in file , line 3
WARNING: Ignoring 3D child object for 2D operation, in file , line 3
```

However, this only occurs during final rendering - more about the causes and remedies follows in the next chapter.

Circle is a 2D drawing method, which we will get to know in the next chapter - the author asks you to consider the function as "God-given" at the moment. Note that setting a value of `fn` too high is also unproductive.

The higher the setting, the more time passes in the rendering process - Our table shows some measurements the author made on a (not very) complicated model. The information provided was taken from the OpenSCAD console and was not verified by multiple processing.

GIMS sends greetings!

The generation of microbenchmarks is an immensely complex process. OpenSCAD makes our work more difficult in that it creates caches in the background that are used in successive render runs. When generating this table, the author deleted these caches before each run via Design → Flush caches.

\$fn=8	0:00:02.571
\$fn=16	0:00:06.089
\$fn=32	0:00:18.814
\$fn=64	0:01:06.317
\$fn=128	0:04:18.864
\$fn=256	0:17:24.232

If you need a higher accuracy for parts of the model, some functions also offer the possibility of locally overwriting the three parameters mentioned in the table.

It should be noted that there is a "funny" trick - while "working" on the model, set a lower accuracy value, which you increase before the last rendering process.

At this point, the author can't resist a little anticipation of the following explanations. The "change" of the variable value can also be done by the following command, that you place at the top of the file instead of the "numerical" assignment to `fn`:

```
$fn = $preview ? 16 : 64;
```

If you adopt the snippet one-to-one, preview renderings will henceforth use the value 16, while a "full" rendering will use 64.

5.2 Antipattern: Construct with `$fn`

Looking at the objects shown in the tables above inspires a dangerous idea - by setting the variable `$fn` accordingly, you can create polygonal cylindrical objects. For example, if you want an octagonal column, you would simply set the value to eight.

The procedure works fine in theory - the author emphasizes the word in theory. From the point of view of the OpenSCAD development team, this is an extreme anti-pattern.

The reason for this is that the "alignment" of the individual surfaces is not guaranteed: the surfaces can "rotate" from version to version.

In the interests of consistency, it is therefore recommended that such elements be implemented using the 2D drawing functions discussed in the following chapters - with them, alignment is guaranteed.

Anyone who realises geometries using `$fn` must take care to specify the OpenSCAD version used for generation and also the platform used and must carry out a check of the results for compatibility before or after each rendering run. This becomes especially important if newly generated elements are to be used in conjunction with already existing files that were rendered or even printed "before".

5.3 Excursus: 3D print pipeline

At the risk that 3D printing-savvy readers might start yawning at this point, the author cannot refuse "basic" explanations about 3D printing. Roughly speaking, the pipeline presents itself as shown in Figure 5-1.

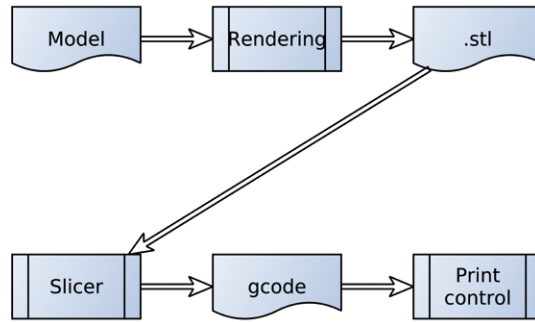


Figure 5-1. Model data passes several preparative steps before the printing process can commence.

On the far left, we find the three-dimensional model that the designer creates. We use OpenSCAD as a matter of course in this textbook - but there is nothing to stop us from imagining 3D Studio MAX or AutoCAD at this point.

In any case, the first task of the modeling program is to convert the object into a .STL file. The abbreviation stands for stereolithography; an industry-standard file format that describes the "spread" of a 3D object in space. Think of it as a three-dimensional array that lists all filled voxels of the model.

Up to the STL file, the workflow is identical between machines and vendors - no matter if you use a 3D printer or a CNC-milling machine. The data becomes hardware-specific in the next step, which is called Slicer in the case of the 3D printer.

Why Slicer?

The term slicer is derived from the act of slicing things up. The program divides the geometry in the STL file into "layers", which the 3D printer realizes.

The slicer takes care of converting the information to a format "understandable" to the 3D printer - in the consumer sector, the resulting files are usually called .gcode. These files describe the individual steps to be performed by the controller inside the printer - think moving the head from A to B, etc pp.

The importance of the slicer should not be underestimated - it is responsible for generating, for example, brims and supports that make the printing of complex models possible. A badly set-up slicer can (and will) lead to print results that stay far behind what your printer hardware would be able to offer.

Last but not least, this .gcode file must be brought to the printer somehow. Depending on the configuration of the printer, the actual process is different - the author likes to print "one-off" affairs using his workstation. Objects that need to be printed several times are moved to an SD card as a .gcode file and then read indirectly by the printer software via the dialog shown in figure 5-2.



Figure 5-2. gcode files can be printed without a workstation.

Remote control recommended!

If your 3D printer control is done via a separate computer, install a remote maintenance system like VNC. Operation from the main workstation is much more convenient.

It should be noted that the energy costs in Hungary are extremely low: if you live in an "energy expensive" country or if you want to save operating hours of a workstation that is difficult to replace, it is advisable to take the detour via an SD card. The two methods don't differ in terms of speed - in hour-long test prints by the author, the difference in speed usually was less than one minute.

5.4 3D printing models

After this short mental intermezzo, it should be clear that our first job involves converting the OpenSCAD file to an STL file.

In itself, this task is not too complicated - next to the rendering symbol, you will find the sign STL, which exports an STL file when clicked. The only important thing is to make sure that you ran a full rendering job before exporting. Incidentally, a preview rendering is not sufficient for this, which OpenSCAD confirms with the following error message:

ERROR: Nothing to export! Try rendering first (press F6).

In this context, it is important not to forget to always command a rendering after changes in the code before exporting the .STL file - otherwise, stale data can be exported. This is also a good opportunity to cache the .STL file or upload it to a version control system.

5.5 Holes, for the first

One of the "most frequent" tasks in the life of the OpenSCAD model designer is the realization of holes that accommodate screws or bolts. We will address this topic several times in this book - let us do a little reflection first, though.

A cylinder, which is intended as a "receiving surface" for a screw, is also simulated by a line train. However, the engine has several ways to place it as shown in Figure 5-3. Lines can be placed either completely inside, "in the middle" or completely outside - the example with six edges illustrates the problem clearly.

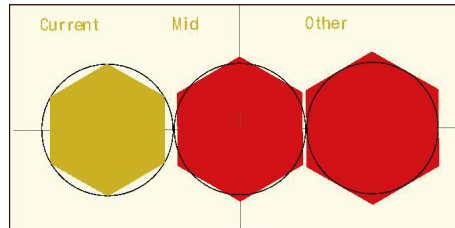


Figure 5-3. The devil hides in the detail (Picture source: <https://bit.ly/2E5GUWY>).

The easiest way to illustrate the behaviour implemented in OpenSCAD is to print out the following object:

```
$fn=8;
difference(){
  cube([50,50,3]);

  translate([25,25,-1])
    cylinder(30,40/2,40/2);
}
```

This program creates a small cube structure that serves as a "positive shape". A cylinder is then removed from it, which leads to the creation of a plate with a comparatively large hole. By setting `$fn=8`; we instruct the OpenSCAD engine to use eight surfaces to calculate the hole.

The height of the subtracted cylinder is set to 30, which appears much too high when looking at the size of the cube. In the author's experience, it has been recommended to allow for a certain margin of overhang in the case of negative shapes to guarantee reliable punching through the surfaces. We will get to know the reasons for this in the following chapter on Minkowski.

In the next step, we command a rendering as usual and export the file to the STL format. At this point, we notice a difficult question - our OpenSCAD models were, so far, "dimensionless". From the slicer's point of view, an STL file can be enlarged and reduced at will. In practice, it is advisable to assume a value of one as 1 mm - in our case, we would have the declaration `cube(10)`, which would then result in a cuboid approx. 10 mm wide.

Why approximately?

3D printers based on the FDM principle are not completely accurate. It is recommended to print a test cube before producing very accurate parts. This information can be stored in the slicer or directly in the firmware of the printer, thereby compensating "mechanical" inaccuracy by software.

At this point, we command a printout and annoy the resulting object with a sliding ear as shown in figures 5-4 and 5-5.



Figure 5-4. The difference between the smallest...



Figure 5-5. ...and the greatest distance is clearly visible.

When looking at the results, keep in mind that our cylinder should have a radius of 20; that is, a diameter of 40. The calculation of the values shows that the "elements" are completely inside the cylinder.

If you explicitly can't have this, you can use a group of "prefabricated" modules. Further information on this can be found at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/undersized_circular_objects.

However, since "productive" printouts are usually done with high FN values, the practical impact of this problem is rather small – post-processing by sanding with a file usually solves the problem anyway.

In the interest of "fun", the author printed the model several times with different values for FN - the table shows the measured values.

fn	Minimal	Maximal
8	36.84	39.07
16	38.64	39.56
32	39.52	39.69

Attentive readers will miss measured values for FN greater than 32 - this is not laziness on the part of the author, but was because in our 5cm object, edges were hardly visible at 32. Logic dictates that the situation would be different with a 50cm large object - as a rule of thumb, larger round objects must be calculated with higher FN values...

5.6 Conclusion

After these excursions, we could sit back and say that we have completed our work with OpenSCAD. But that is not the case. In practice, there are a whole range of objects that are very difficult to describe "three-dimensionally".

Analogous to the work of the electronics engineer switching between modulation, spectral, and time domains, there is a completely different way of constructing elements. In the next chapter, we will look at it – stay tuned lest you miss out on a completely different approach which can be highly efficient at times.

Chapter 6 • OpenSCAD as 2D modeling tool

The reason why a 3D program should also be used to "capture" two-dimensional objects appears to be arguable at first. For motivation, we want to start with the decorative object shown in figure 6-1. It was taken from the OpenSCAD documentation and describes the problem at hand very well.

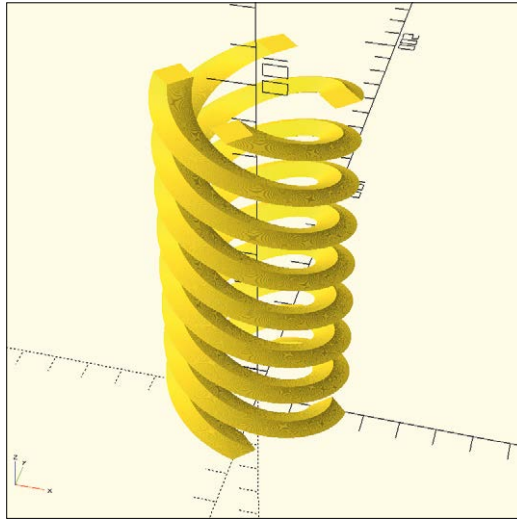


Figure 6-1. This very complicated looking vase...

Whoever tries to assemble this object exclusively from "three-dimensional" primitives is setting himself up for severe and intensive pain. A better approach imagines the object - as shown in figure 6-2 - as a "two-dimensional base", which you then scale upwards and rotate.

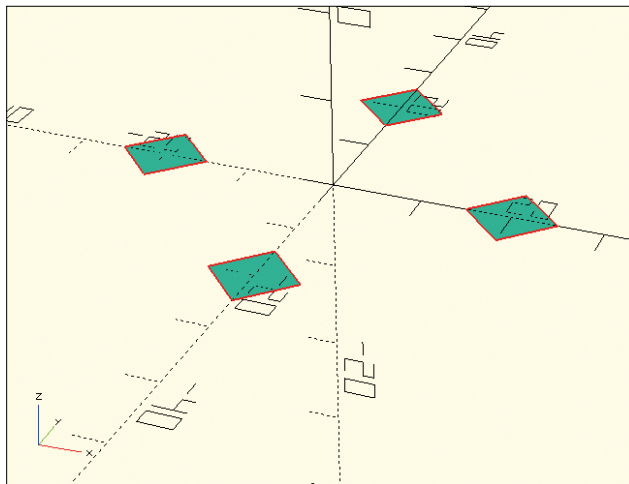


Figure 6-2. ...arises from an absolutely simple base surface.

Example program

If you want to reproduce illustrations one and two, experiment with the following program. The operators used are explained in detail in the course of the chapter:

```
linear_extrude(100, twist = 720, slices = 3200)
  translate([20, 0, 0])
    circle(5, $fn=4);

linear_extrude(100, twist = 720, slices = 3200)
  translate([-20, 0, 0])
    circle(5, $fn=4);

linear_extrude(100, twist = 720, slices = 3200)
  translate([0, 20, 0])
    circle(5, $fn=4);

linear_extrude(100, twist = 720, slices = 3200)
  translate([0, -20, 0])
    circle(5, $fn=4);
```

However, due to the very high complexity of the program, you can expect long rendering times - reduce the slices setting or combine polygons to save resources.

The choice of the "correct" approach is not only important in the field of three-dimensional modeling. Figure 6-3 shows a noise signal in both the time domain and the frequency domain - the signal has a "clear" signature in the frequency domain, whereas it is not possible to detect or even trigger it in the time domain.

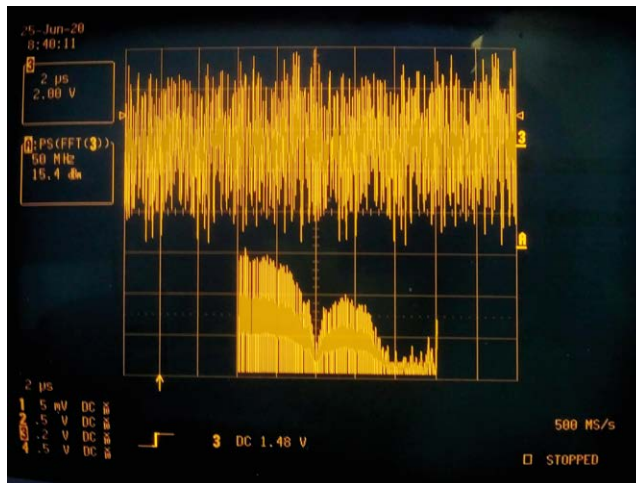


Figure 6-3. Thanks to FFT analysis, we can see more clearly...

The author is fully aware that not every object can be created by "exploding" a 2D plane. In many cases, however, it is the most efficient approach - stay alert in your own interest.

6.1 Theory of construction in the two-dimensional domain

In theory, OpenSCAD can also be used to generate two-dimensional drawings - this is an application that the author would advise against, having done it a few times and finding the results less than satisfactory.

In the world of OpenSCAD, the task of "two-dimensional" elements is to serve as a base for various extrusion operations. The idea behind this is that the two-dimensional object is transformed into three-dimensional space using a transformation.

Analogous to three-dimensional design, the creation of cuboids also serves as basic operation in the 2D area.

The following two commands generate cubes. They behave analogous to the three-dimensional commands we discussed before:

```
square(size = [x, y], center = true/false);
square(size = x      , center = true/false);
```

For a first attempt the following parameterization is useful:

```
square([5,5], true);
```

If you render an OpenSCAD file containing it, you will see the behavior shown in Figure 6-4 after adjusting the perspective.

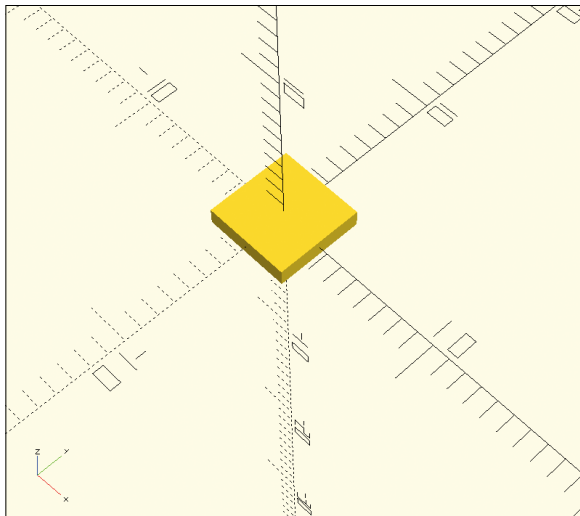


Figure 6-4. Preview renderings show two-dimensional objects with three-dimensional expansion.

The reason for this behaviour is that the display of "perfectly two-dimensional" elements would cause problems for OpenSCAD in 3D mode. An element that is not equipped with "spatial" propagation by other operators is therefore rendered as having a depth of one when a preview is requested.

Interestingly, this behaviour only occurs with preview renderings. If you render a two-dimensional object productively, you will see the behaviour shown in Figure 6-5.

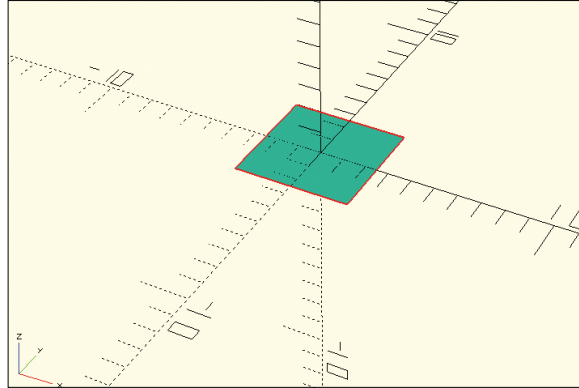


Figure 6-5. Final renderings of two-dimensional scenes look mathematically correct.

OpenSCAD does not allow productive renderings of objects that have both two-dimensional and three-dimensional elements according to the following scheme:

```
$fn=8;
circle(5);
translate([10,0,0])
  sphere(5);
```

Such an element can be rendered as a preview, as shown in chapter five. However, commanding a production rendering run yields the following error message:

```
WARNING: Mixing 2D and 3D objects is not supported, in file , line 3
WARNING: Ignoring 3D child object for 2D operation, in file , line 3
```

The same applies to the circle command, which takes care of creating circles according to the following scheme:

```
circle(r=radius | d=diameter);
```

Note that the OpenSCAD engine considers a single, unnamed parameter as order to set the radius of the circle. When working with the Circle command, there is an exception to the antipattern mentioned in chapter five. If desired, the command accepts a dedicated \$fn parameter, which creates 2D polygons. A good demonstration is the following snippet, taken from the OpenSCAD documentation:

```

translate([-42, 0]){circle(20,$fn=3);%circle(20,$fn=90);}
translate([ 0, 0]) circle(20,$fn=4);
translate([ 42, 0]) circle(20,$fn=5);
translate([-42,-42]) circle(20,$fn=6);
translate([ 0,-42]) circle(20,$fn=8);
translate([ 42,-42]) circle(20,$fn=12);

```

Run the program to see the result shown in figure 6-6.

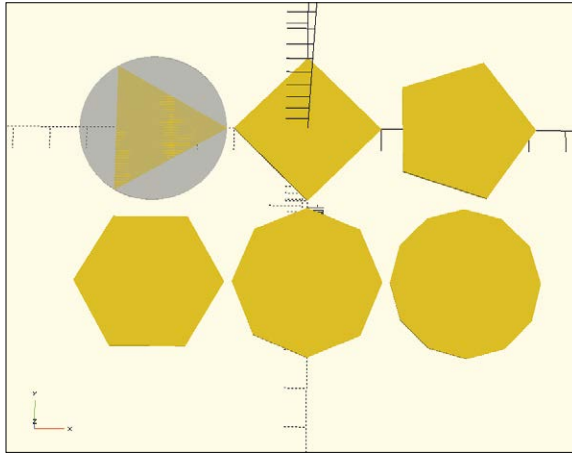


Figure 6-6. The `$fn` parameter creates polygons.

Note that the OpenSCAD team specifies that one edge of the polygon created by the circle method always points in the direction of the positive X-axis. This can be checked by taking a careful look at figure 6-6 - both the polygon conjured with three and the polygon conjured with five sides each have one edge pointing in the direction of the positive X axis.

6.2 Creating Ellipses

Attentive readers of the previous chapters remember that the author promised explanations on the generation of ellipsoidal objects. An ellipse is - geometry experts should not write letters to the editor - a circle, which is deformed "asymmetrically".

The effect responsible for 2D ellipses is known in the 3D area as an axonometry. To generate such operations, OpenSCAD offers a transformation operator that behaves "analogously" to the transform and rotate commands discussed in the three-dimensional domain. The operation is defined as follows:

```

scale(v = [x, y, z]) { ... }

```

Analogous to Transform, the parameter name `v` is "optional". `X`, `Y`, and `Z` supply "scaling values" which describe whether OpenSCAD must enlarge or compress the object in direction of the respective axis. If you want to keep an axis unchanged, pass one. Values `>1` result in the object being stretched, while values `<1` result in compression.

It is important that scale is not commutative. This can be demonstrated in the three-dimensional domain with the following example, which creates two cubes:

```
color("white")
translate([15,0,0])
  scale([0.5,1,2])
    cube(10, center=true);
color("grey")
scale([0.5,1,2])
  translate([15,0,0])
    cube(10, center=true);
```

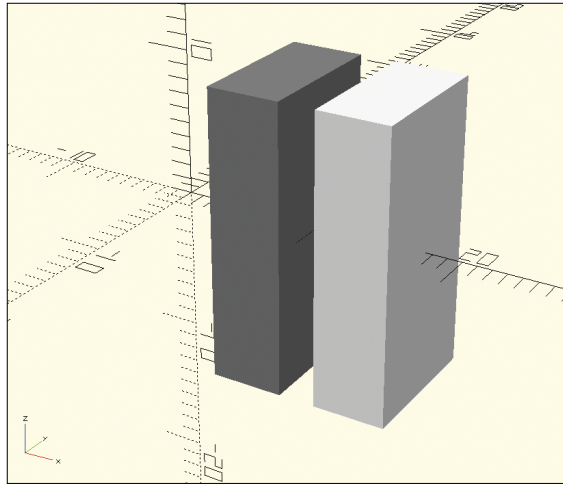


Figure 6-7. The order of translation and scaling is important.

If you look at the two objects shown in figure 6-8 from above, you will notice that scaling after translation also scales the translation value.

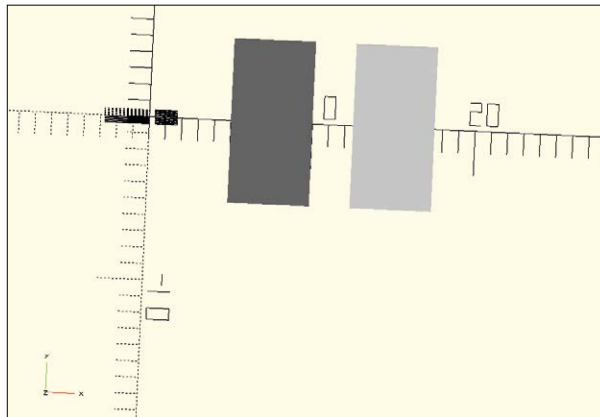


Figure 6-8. A change of perspective creates clarity.

After this short but important excursion into the world of 3D axonometry, we want to return to the two-dimensional plane, where the deformation of a circle into an ellipse is pending. The following code is used for this purpose, the effect of which can be seen as an example rendering in figure 6-9:

```
color("white")
  scale([0.5,1])
    circle(10);
```

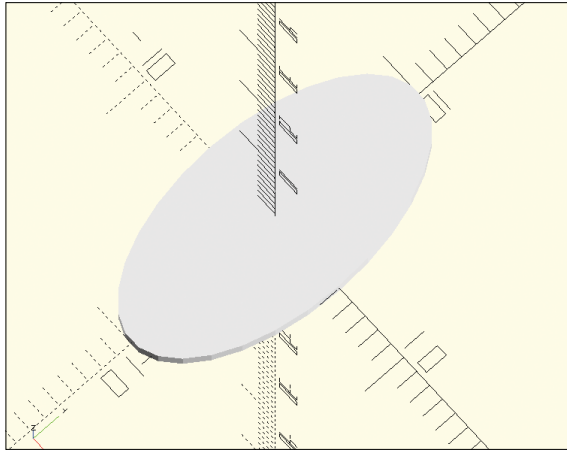


Figure 6-9. The Scale operator generates ellipses.

6.3 Linear extrusion

In the introduction, we stated that the 2D functions of OpenSCAD are not an end in itself. Rather, it is possible to create a three-dimensional object from the two-dimensional "base area" with little effort. The easiest way to do this is linear extrusion. To "illustrate" the procedure, you can imagine a playDOH press which uses a "mould" to bring mass into a specific shape.

The author prefers to visualise the procedure by using a paper cutting machine: Imagine that the two-dimensional floor plan is printed out multiple times, and each of the copies is then cut to shape. The individual sheets are placed on top of each other, similar to a stack of cards, which allows the creation of a three-dimensional object at a certain height specified.

Since the `linear_extrude` command takes a group of parameters, we want to work with the simplest version initially. Anyone who simply calls `linear_extrude(20)` will tell OpenSCAD to make the base element(s) 20 units high – you can, of course, also pass a "convoluted" 2D object made up of multiple primitives:

```
linear_extrude(20)
  circle(10, $fn=7);
```

Executing the program above leads to the result shown in Figure 6-10.

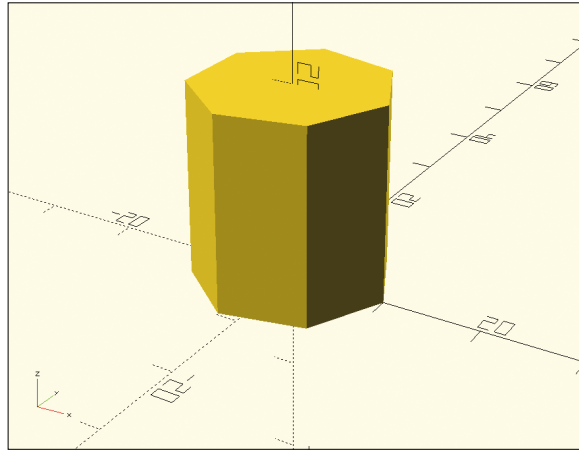


Figure 6-10. *linear_extrude* works as expected.

Creating a "primitive" heptagonal column object may not be a particularly complex task. Nevertheless, it allows for the creation of real-world usable objects, which I want to demonstrate with a small worked example.

6.4 Worked Example: Coat rack a la Tam

Whoever takes over a bunker from a previous owner always finds himself governing over one or more "caverns". If said cavern is small, it is advisable to transform it into a coat rack. Unfortunately, the thought of cloakroom objects seems to trigger a pavlovian greed reflex in providers of pipes and other round elements, which offends the always frugal author.

The solution was to buy two rods of aircraft-grade aluminum from a friend and lengthen them with a saw. This left the problem of wall mounting, which was to be done with dowels, screws, and brackets shown in figure 6-11.



Figure 6-11. *This retaining element is not particularly expensive*

Most important is the dimensioning of the retaining elements, which is shown in figures 6-12 and 6-13.

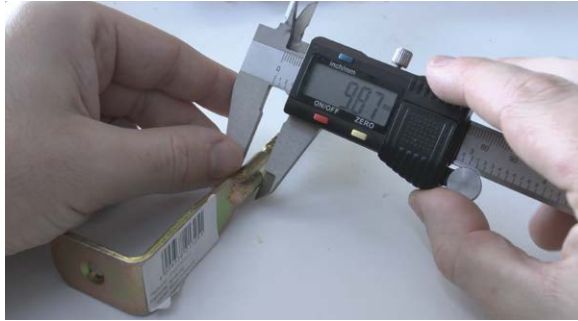


Figure 6-12. Working with a caliper...



Figure 6-13. ...wants to be learned!

In the next step, we construct a "carrier element". It must be pushed "over" the bar but should be as tight as possible. In doing so, we make use of a small special feature of the PLA material used in the FDM process: Wood screws and similarly "aggressively-threaded" screws can be turned into the material with moderate force if the screw channel is dimensioned correctly. Getting this factor perfectly right is both art and science - the parameters used here allowed the bolt to be screwed in (with force).

Let us start with the actual creation of the element - the script used looks like this:

```
$fn=256;
$diam = 25.9/2;
difference() {
  difference() {
    linear_extrude(100)
      circle($diam);
    linear_extrude(20)
      circle(5.2);
  }
}
```

```
linear_extrude(101)
circle(4.7);
}
```

In principle, this is a two-stage creation. In the first step, the "inner" differential operator creates a cylinder that is 100 mm long and has an outer diameter that fits into the aluminium rod.

A 20 mm long cylinder is then subtracted from it, which represents the head of the L-holder element closer to the wall. The object will then be pierced again by a second differential operator - he will blow through a 101 mm long cylinder, which has a radius of $4.7/2$ and will serve as a "holding surface" for the screw.

The next problem we have to address is how to place the object "optimally". Keep in mind that 3D printers have a hard time with "overhangs" – sadly, you find them in our small cylinder (see Figure 6-14).



Figure 6-14. A part of the hole is wider than the rest.

In the case of our object, it is advisable to choose the configuration shown in Figure 6-15. The wider part of the hole points upwards, thereby obviating the need to calculate supports.

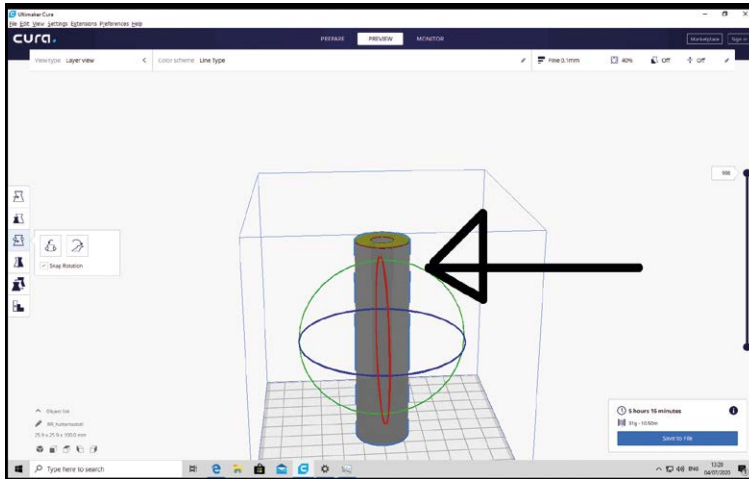


Figure 6-15. If we turn it the right way, we can do without supports - the hole gets wider towards the top, which is no problem for FDM technology.

At this point, you can command a realization of the object. For space reasons alone, this work cannot go into the ideal "parameterization" of your slicer - general information on this can be found in the literature references in the work on functional 3D printing.

6.5 linear_extrude with advanced parameterization

Now that I have shown you the advantages of the 2D-3D combination construction, I would like to return to the "full" definition of the linear_extrude command. According to the documentation, it presents itself as follows:

```
linear_extrude(height = fanwidth, center = true, convexity = 10, twist =
-fanrot, slices = 20, scale = 1.0, $fn = 16) {...}
```

To illustrate the behavior, we want to work with the Twist parameter in the first step:

```
linear_extrude(20, twist = 90)
circle(10, $fn=4);
```

The circle serving as a base now has but four sides, resulting in a square and a "simpler" visualization. By setting twist to 90, we command a rotation of 90°, resulting in the behavior shown in Figure 6-16.

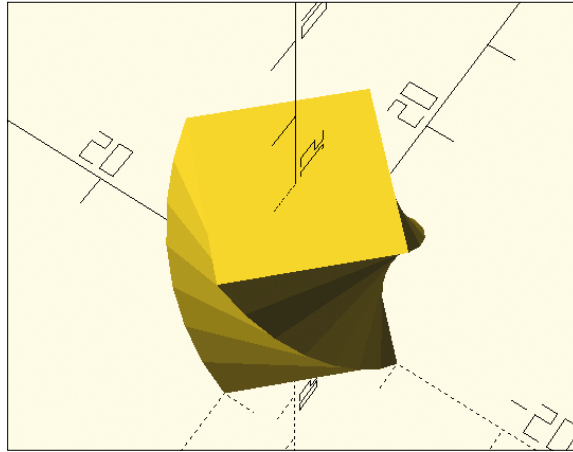


Figure 6-16. The object is now rotated by 90° during its extrusion.

If you look carefully, you will notice that the "outer edges" of the object are comparatively rough - this is because OpenSCAD creates comparatively few layer stops by default, onto which it recalculates the "basic shape".

Fortunately, the program lets you specify the number of intermediate levels via an additional parameter. A significant "improvement" can be achieved if we use the following program example - its output is shown in Figure 6-17:

```
linear_extrude(20, twist = 90, slices = 100)
  circle(10, $fn=4);
```

With the slices value, you command linear_extrude to set a certain number of intermediate layers - the value used here would lead to 100 layers being used.

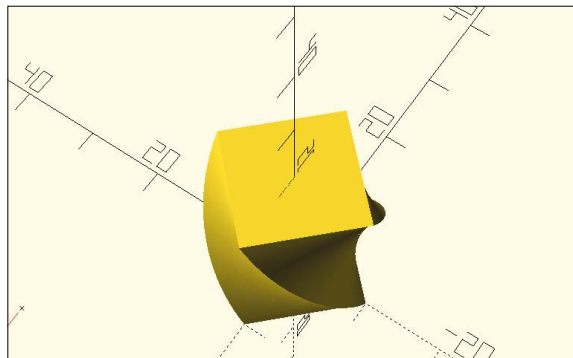


Figure 6-17. Writing in additional layers slows down rendering, but improves the display.

Do not go crazy when choosing the optimal value for slices - setting too high a value will slow down the rendering process and waste processing power needlessly.

A "good" way to determine the slice value is to use the printer's capabilities as a guide. The Renkforce RF100 used by the author has a nominal resolution of 0.1mm according to the datasheet. In Cura - see figure eighteen - this resolution can be improved a bit by using various tricks in stepper motor control.

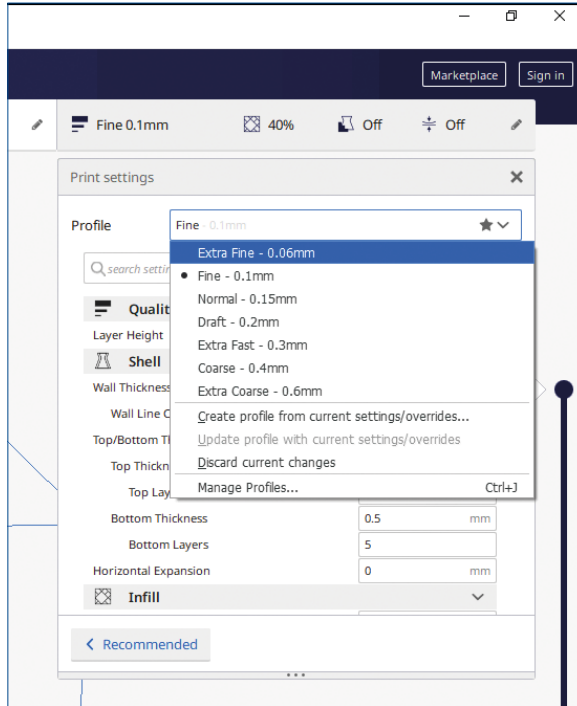


Figure 6-18. According to Cura, the minimum Z-plane thickness is 0.06mm.

Since we have to tell `linear_extrude` how high the object has to be, we can determine an optimal value by division. Alternatively, you can also simply increase the value until the rendering preview looks sufficiently smooth to you.

Next, we need to address the question of how `linear_extrude` performs the "center point determination". Analogous to the previous experiments, we want to proceed playfully here, and move the cuboid away from the center point by a `translate` operation before extrusion:

```
linear_extrude(50, twist = 360, slices = 3200)
  translate([20, 0, 0])
    circle(10, $fn=4);
```

The reward for the effort is the spiral shown in figure 6-19.



Figure 6-19. Correctly parameterized extrusions can look futuristic

Strange surface.

The artifacts on the surface of our spiral, reminiscent of scales, are caused by moiré artifacts during the "smoothing" of the object. Further information about this - optically disturbing but completely harmless - phenomenon can be found at https://en.wikipedia.org/wiki/Moir%C3%A9_pattern.

A close look at the output in the figure shows us that the start point and the endpoint of the spiral are directly above each other. From this, we learn that the "generation" of the object is a two-step process.

In the first step, OpenSCAD "projects" the existing element onto the X-Y plane to obtain a two-dimensional "shadow" of the object. This is then applied along the Z-axis and "stapled" to the origin to allow rotation. It follows that the Translate command in the direction of the x-axis ensures that the resulting spiral will be "20" wide.

Another very interesting aspect of the `linear_extrude` operator is the ability to change the size of the object during extrusion. This can be influenced by the `scale` option:

```
linear_extrude(50, twist = 360, scale=2, slices = 3200)
  translate([20, 0, 0])
    circle(10, $fn=4);
```

The present program would enlarge our coil "from bottom to top". At the end of the scaling, rotation, and extrusion operation, we would have an end block twice as large as shown in Figure 6-20.

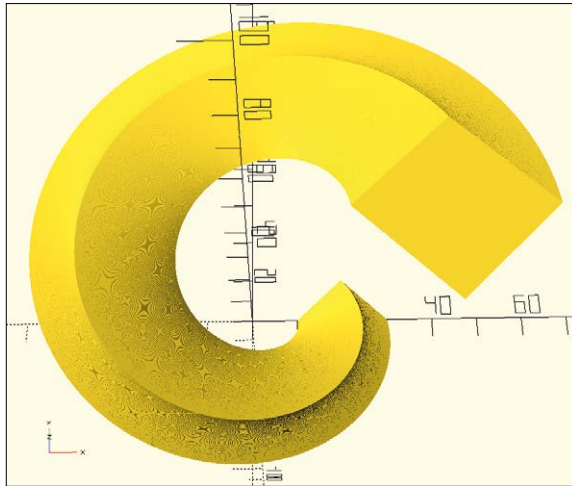


Figure 6-20. The scaling parameter allows for advanced operations.

6.6 Create rotationally symmetrical objects

In addition to "objects created by extrusion", there is a group of objects which are "rotationally symmetrical". This refers to the situation shown in Figure 6-21 - the object would be created by "rotating" it around the axis of rotation.

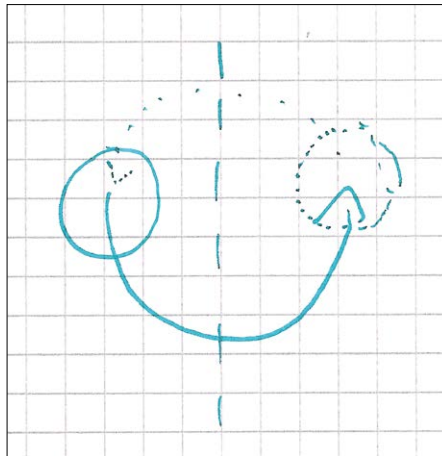


Figure 6-21. Rotating this circle by 360 degrees creates a torus.

This explanation, which at first glance appears somewhat lengthy, is necessary because it is one of those parts of OpenSCAD with which beginners often have trouble - the documentation contains a whole list of objects that cannot be realized with the rotation operator.

GUI design note

In practice, the mentioning of various non-permitted use cases in the documentation indicates that users have problems with this feature and make many customer support requests. More about this, and also about the role of options in general, can be found in the generally recommended blog Joel on Software. A good introduction to this topic would be at the URL <https://www.joelonsoftware.com/2000/04/12/choices/>.

Be that as it may, by far the most important rule is that the object as a whole must lie either entirely on the left or - less popular - entirely on the right of the Y-axis. If this is not the case, OpenSCAD would throw errors, which we will look at in more detail in the following steps. But first, we want to realise a torus, for which the following code is responsible:

```
rotate_extrude(convexity = 10)
  translate([20, 0, 0])
    circle(r = 10);
```

`rotate_extrude` rotates the object 360° by default - the parameter `convexity` defines a certain calculation accuracy and should be increased if the rendering system behaves strangely. The two subsequent `translate`-and-`circle` commands ensure that the rotation command finds an object in the X-Y plane that it can "whirl around". The object shown in Figure 6-22 is the reward for the effort.

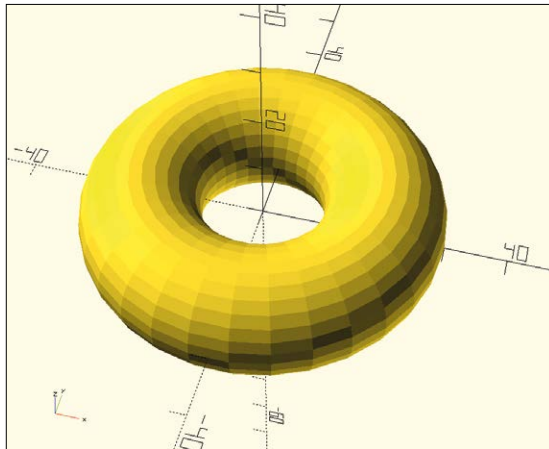


Figure 6-22. `rotate_extrude` facilitates the creation of tori.

Analogous to linear extrusion, you may specify the degree of accuracy for `rotate_extrude`. Here, this is done via the variable `$fn` known from above - we can, for example, use the following scheme to specify that the rotation process must create 100 part planes:

```
rotate_extrude(convexity = 10, $fn = 100)
  translate([20, 0, 0])
    circle(r = 10);
```

As always, the reward for the effort is an improvement in the object structure, which now looks like Figure 6-23.

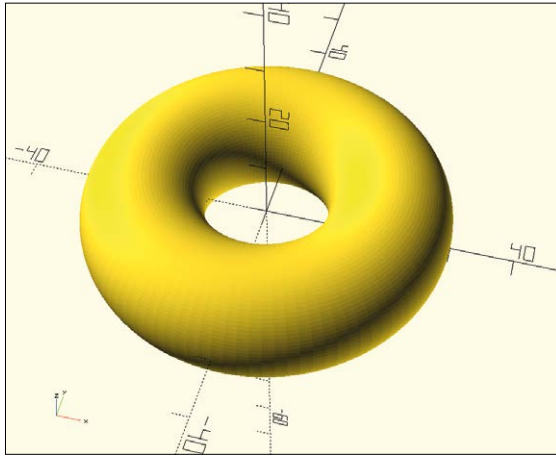


Figure 6-23. Higher values provide smoother tori.

The problems mentioned above can be illustrated by the following code snippet:

```
rotate_extrude(convexity = 10, $fn = 100)
  translate([8, 0, 0])
    circle(r = 10);
```

The most important difference to the two previous commands is that the call to Translate no longer places the circle completely to the left or completely to the right of the X-axis - if the rotate_extrude command were removed, we would see the base object shown in Figure 6-24.

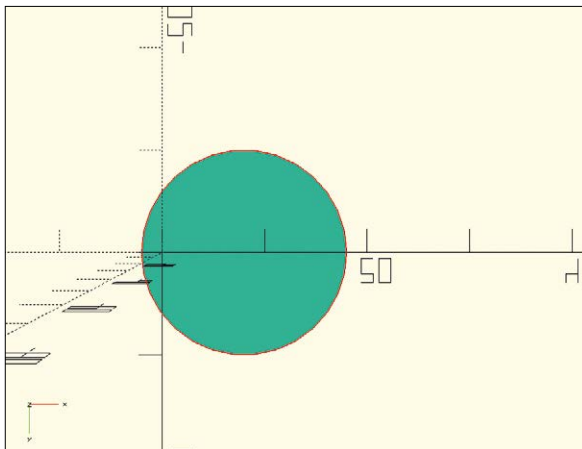


Figure 6-24. This object touches the X-axis.

If you release the program for execution anyway, you will get an error of the following type: ERROR: all points for rotate_extrude() must have the same X coordinate sign (range is -0.09 -> 18.00) is output to the console. Note that the command is then discarded without the creation of any geometry.

Last but not least, let's introduce the Angle parameter. Its name describes the behaviour exactly - by passing Angle you can influence the "rotation path" rotate_extrude takes. If you want to have a "half" torus, the following code would lead to the result shown in Figure 6-25.

```
rotate_extrude(convexity = 10, $fn = 100, angle=180)
  translate([20, 0, 0])
    circle(r = 10);
```

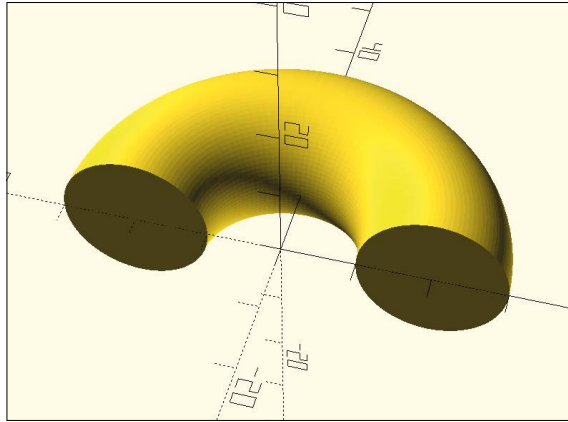


Figure 6-25. The torus is now halved.

6.7 Fully parametric construction from point clouds

Two-dimensional elements can - of course - also be processed with combination operators, which we got to know some time ago. An example of this is a difference that subtracts a smaller circle from a cuboid and yields the result shown in Figure 6-26:

```
difference(){
  square(30);
  circle(r = 10);
}
```

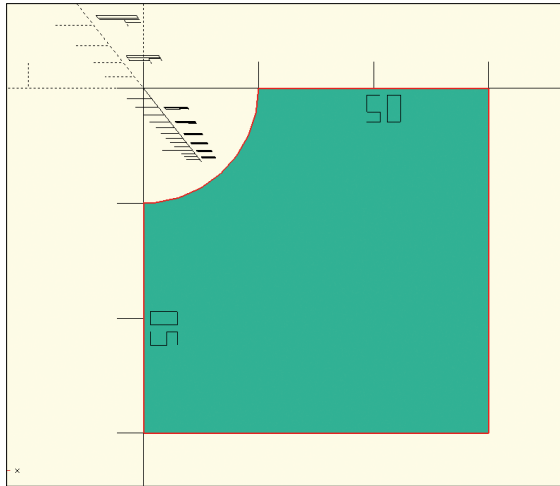


Figure 6-26. Set operators also work on two-dimensional elements.

In practice, there are always objects for which you wish to have a different construction method. Specifically, you often have a point cloud - on quad paper, for example - that you want to shape into a polygon. In OpenSCAD, this can be done using the polygon operator, which is declared as follows

```

polygon(points = [ [x, y], ... ], paths = [ [p1, p2, p3..], ...], convexity
= N);

```

The polygon operator requires a field of points to be supplied as two-dimensional values. If no path is supplied, the polygon consists of the entire contents of the points field.

Alternatively, one or more paths can be supplied - we will see what this does exactly later. For now, it is more important to animate the cloud drawn on checkered paper in Figure 6-27. By the way, it is to be expected that the object has nothing to do with a real cloud - the term point cloud stands for any "collection" of points that describe an object.

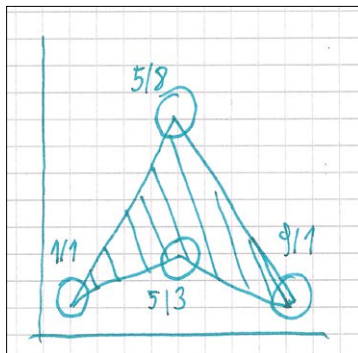


Figure 6-27. Drawing up a point cloud on quad paper provides the needed coordinates for further experimentation.

From the practical experience of the author, it should be noted that the procedure demonstrated here, using chequered paper, is quite "practical" - if you start with a real model, make a hand sketch before hitting OpenSCAD.

Quad paper, self-printing.

The long and bitter life experience of the racial half-Arab teaches that you never have quad paper when you need it. At <https://www.papersnake.de/kariert/> you can find a generator that produces plaid paper with any printer.

Let's now turn to the realisation of the polygon. For now, we want to focus exclusively on the points array, in which we create the individual points of the polygon one after the other:

```
polygon(points = [ [1, 1], [5, 3], [9, 1], [5, 8] ] );
```

At this point the object is ready for rendering - Figure 6-28 shows the result.

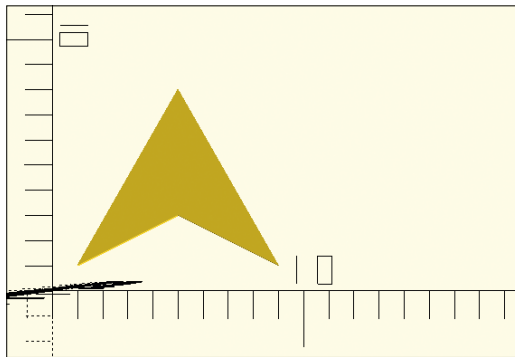


Figure 6-28. The *polygon* operator looks complicated but works without problems.

Next, we want to change the polygon in so far as some points should occur "multiple times". To "motivate" this we want to take a look at the symbol shown in Figure 6-29, which - in principle - is completely described with seven points.

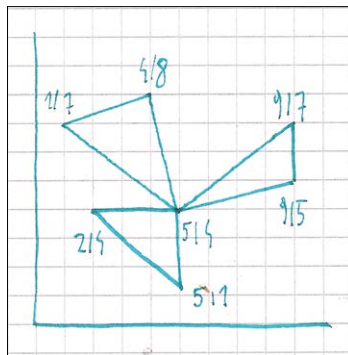


Figure 6-29. This (admittedly somewhat synthetic) demo object allows for more in-depth experiments with OpenSCAD.

The most interesting difference to its predecessor is that the element presented here uses the point in the middle "multiple times". In theory, there is nothing to prevent the polygon from being realized according to the following scheme:

```

polygon(points = [ [5, 1], [2, 4], [5, 4],
                  [1, 7], [4, 8], [5, 4],
                  [9, 7], [9, 5], [5, 4] ]);

```

The problem with the present code is that the origin now occurs "multiple times" in the array. This is inefficient because it leads to considerable additional work when the geometry is to be adapted.

A "nicer" way to describe the polygon chain would assign an ID to each point in the array. The actual description of the polygon then is no longer done "only" via the point list. Instead, in this operating mode, we provide a second field that describes the path to be created via the indices of the points. The reward for the effort is that we can now write to the polygon according to the following scheme:

```

points = [ [5, 1], [2, 4], [5, 4],
          [1, 7], [4, 8],
          [9, 7], [9, 5], ];
paths = [ [0,1,2, 3, 4, 2, 5, 6, 2] ];
polygon(points=points, paths=paths);

```

In the interest of better clarity, the author declares the two vectors in local variables, which then pass as parameters to the actual operator.

Inside the path array, you have to supply sub-arrays - it is explicitly not allowed to create a path consisting of a monovalent array according to the following scheme

```

paths = [0,1,2, 3, 4, 2, 5, 6, 2];

```

The reason for this behaviour, which at first glance does not sound very reasonable, is that the OpenSCAD polygon engine can also process "polygons" consisting of several paths if desired. This allows, among other things, the realization of holes, and will soon be dealt with further in this chapter.

The question as to which procedure is better suited can be debated very well. In practice, however, the prospective OpenSCAD user will quickly discover that he prefers one of the two methods for writing point clouds - this is then used permanently. In any case, the result of the sample rendering is shown in Figure 6-30.

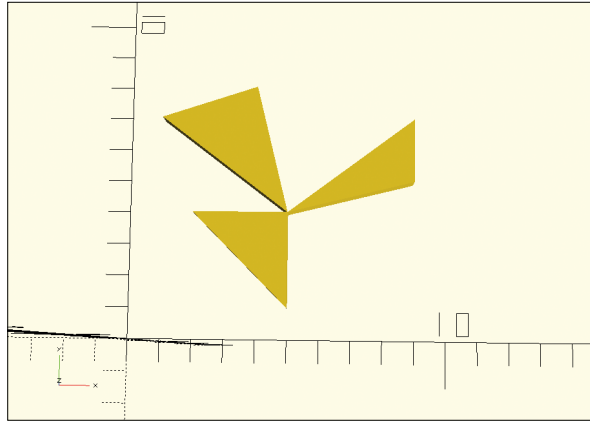


Figure 6-30. The polygon operator manages complex paths without problems.

6.7 Holes in polygons

Two things are interesting at this point - first, the question of whether the "sequence" of points is relevant. For this, we want to return to the test object again, in which we now define the two following paths. Please limit yourself to looking at the direction of the path in Figure 6-31 - the coordinates remain unchanged from Figure 6-27.

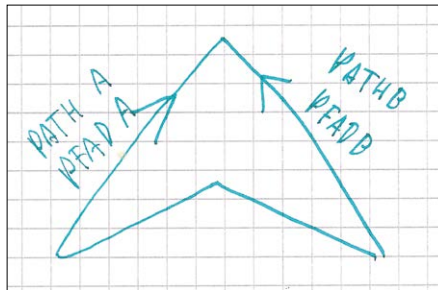


Figure 6-31. The "direction of rotation" of the polygons is different.

Logic dictates that the two following notations must be "identical" from the OpenSCAD engine's point of view:

```

polygon(points = [ [1, 1], [5, 3], [9, 1], [5, 8] ]);
polygon(points = [ [9, 1], [5, 8], [1, 1], [5, 3] ]);

```

If we restrict ourselves to working in two-dimensional space, this is also true. Note that the "direction of rotation" of a three-dimensional polygon is sometimes (!!!) important - a task we will approach in chapter nine.

Problem number two is the creation of polygons "with a hole". In theory, there is nothing to be said against simply creating a positive and negative shape and then connecting them using the difference operator. However, this procedure, which in itself works quite well, is inconvenient, which is why the OpenSCAD development team offers a multi-path function-

ality. To use it, we have to extend the test object for one last time and imagine a hole in its center. Since OpenSCAD also supports non-integer coordinates, we are spoilt for choice.

Incidentally, this is where one of the advantages of polygon-path abstraction becomes apparent. If we had created our polygon by stringing together a series of points, we would have to rearrange the polygon at this point - there is no way to specify that a particular point of the generic points array describes a hole.

In case of working with paths, we just add another path:

```
points = [ [5, 1], [2, 4], [5, 4],
           [1, 7], [4, 8],
           [9, 7], [9, 5],

           //HOLE
           [9.5, 7.5], [9.5, 5.5], [5.5, 4.5],
           ];
paths = [ [0,1,2, 3, 4, 2, 5, 6, 2], [7,8,9] ];

polygon(points=points, paths=paths);
```

The reward of the rendering process is the result shown in Figure 6-32.

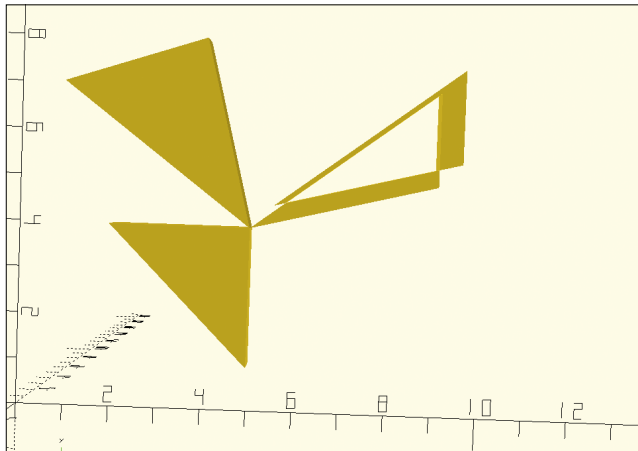


Figure 6-32. The hole in the polygon appears without problems.

At this point, it should be noted that it is allowed to pass "multiple" paths. Analogous to the difference operator, polygon takes the first path as a positive shape and subtracts the solids described in the other paths from it.

6.8 Worked Example: Board holder

As a last example on the topic of polygons, we want to realise a practical example that was used in the author's consulting practice. The circuit board shown in Figure 6-33 was

to be converted - after soldering four jump wires - into a format that would survive postal mailing.

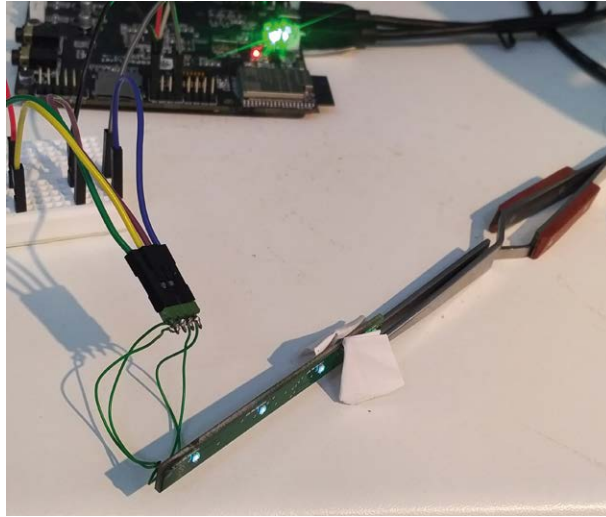


Figure 6-33. If you are not thinking of the cable lug, you have to solder on the PCB...

To make it easier for the customer to use the board as a demonstration object, a kind of holder should be designed. Logically, "shifted" triangles are ideally suited for this purpose, which is why the first step was to create an object consisting of two arms and a foot:

```
union(){
  //ARMS
  rotate([90,0,0])
  {
    linear_extrude(5)
      polygon(points=[[0,0],[30,0],[50,40],[30,40]]);
    translate([0,0,45])
      linear_extrude(5)
        polygon(points=[[0,0],[30,0],[50,40],[30,40]]);
  }
  //Foot
  translate([0,-72,0])
  cube([50,95,4]);
}
```

It is particularly noticeable here that the polygon was converted into a three-dimensional object by using `linear_extrude`. The foot is added by a normal `translate` operation; the reward for the effort is the element shown in Figure 6-34.

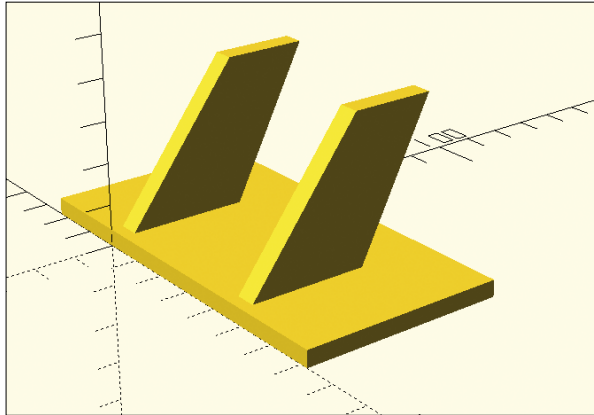


Figure 6-34. The "holder" takes shape.

As the next task, we have to create bulges for the board and holes for the wiring harness. At this point, we could - theoretically - return to the definition of the polygon and use one of the methods discussed above.

When working with OpenSCAD, however, you shouldn't send your brain to the ivory tower. A recess can just as well be generated by a three-dimensional primitive. Therefore, the next task is to generate the primitives to be deducted:

```
union(){
  //Hole for cables
  translate([27,10, 18])
  rotate([90,0,0])
  cylinder(120,4,4);

  translate([13,-60, 18])
  rotate([125,00,90])
  cube([71,8,5]);
}
```

In any case, the results of the rendering of the "negative" elements present themselves as shown in Figure 6-35.

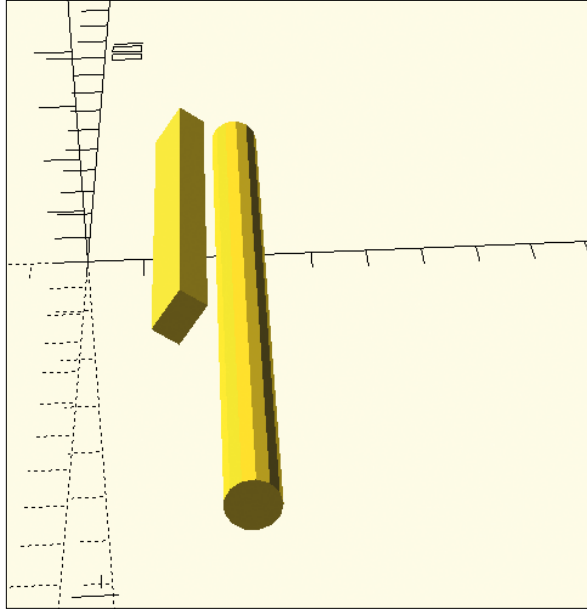


Figure 6-35. The negative forms are "positive" at the beginning.

Our final task is to merge the elements using union and difference operators. The resulting program looks like this:

```
difference(){
  difference(){
    //ADDITIVES
    union(){
      //ARMS
      rotate([90,0,0])
      {
        linear_extrude(5)
        polygon(points=[[0,0],[30,0],[50,40],[30,40]]);
        translate([0,0,45])
        linear_extrude(5)
        polygon(points=[[0,0],[30,0],[50,40],[30,40]]);
      }
      //Foot
      translate([0,-72,0])
      cube([50,95,4]);
    }

    //SUBTRACTIVES
    union(){
      //Hole for cables
      translate([27,10, 18])
```

```

        rotate([90,0,0])
            cylinder(120,4,4);

        translate([13,-60, 18])
            rotate([125,00,90])
            cube([71,8,5]);
    }

}
//MATERIAL SAVER
translate([0,-60, 29])
cube([100,100,100]);
}

```

Last but not least, the author introduced another cube to artificially limit the "height" of the polygons in the interest of more convenient handling. This is a "convenience measure" because the subsequent modification of polygon coordinates is annoying.

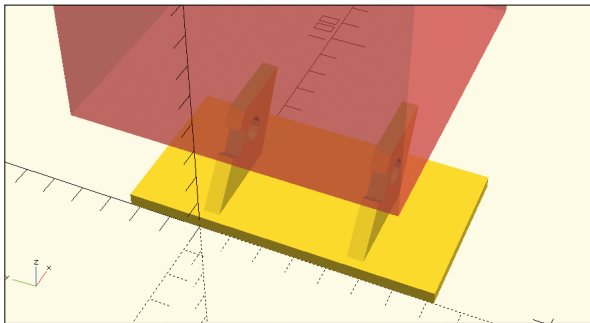


Figure 6-36. The cuboid saves material by trimming the height of the arms.

Be that as it may, we can command a rendering run at this point - it leads to the result shown in Figure 6-37.

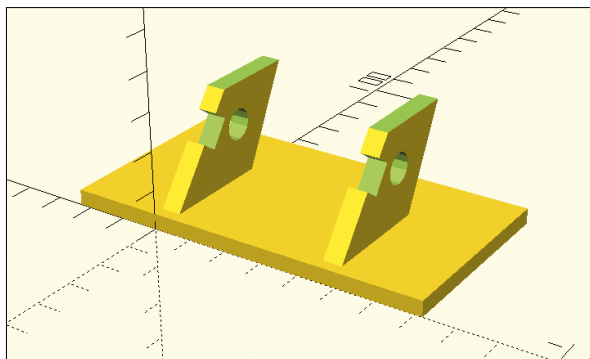


Figure 6-37. The board holder is ready.

For completion, only a 3D printer is required at this point. Afterwards you can use a hot glue gun to make sure the cables in the holes are subject to additional strain relief.

It should be noted that this construction - despite the extremely fragile wire soldering points to 0.8 mm pitch - survived postal delivery in a box with only a little padding. The element, which was posted in Budapest, arrived in Georgia without any problems and has been satisfying the customer perfectly for several months.

6.9 What next?

The use of "two-dimensional" base surfaces makes the creation of some elements much easier. However, we have not yet reached our goal. In the next chapter, we will look at ways of using the functions contained in OpenSCAD to "automatically" generate geometry. This is useful, for example, when geometry is to be assembled from a "group" of identical elements which are moved or rotated to different locations.

Chapter 7 • OpenSCAD as a dynamically reconfigurable modeling system

In the last chapters, we conducted quite impressive experiments with OpenSCAD and created objects that can be used in real life. Until now, except for the excursus with the dreidel, there has always been a one-to-one relationship between the code and the geometries.

If we wanted to recycle a "geometry" in several places, we had to duplicate it in the generator code. This procedure, which is quite manageable for smaller examples, proves to be counterproductive in practice because duplication means that you have to rework several places when changes are made. Experience shows that you reach a point where you can no longer find the changes - the consequence is spaghetti code.

In this chapter, I would like to present some of the possibilities of the OpenSCAD description language for breaking the "one-to-one relationship" between code and geometry. In addition to the possibility of creating modules reminiscent of classes, we will also introduce loops, selections, and variables. These allow flexible parameterisation of the resulting geometry - so stay with us...

7.1 Variable in OpenSCAD

If there is one "aspect" of modern programming that generally needs not be discussed, it is certainly the use of variables - they behave identically in virtually all programming languages. Funnily enough, OpenSCAD is really stepping out of line in this area - please work through the following explanations even if you have been programming for more than 20 years.

The cause of the "problem".

When defining the language, the OpenSCAD development team follows the design paradigm of functional programming. Further information on this can be found in Wikipedia at https://en.wikipedia.org/wiki/Functional_programming. However, understanding of functional programming is explicitly not necessary for working with OpenSCAD - the author is only presenting the link to provide additional background information for those interested in computer science.

For a first taste of the possibilities of OpenSCAD variables, we would like to take a look at the following code, which represents the first part of the wardrobe object realized in the previous chapter:

```
$fn=256;  
$diam = 25.9/2;  
difference() {
```

In addition to the assignment to the variable `$fn`, the author introduces a variable called `diam`, which takes the value `25.9/2`; it describes, as discussed earlier, the "inside diameter" of the aluminum bar.

Once defined, a variable can be used like an ordinary constant - our example passed it to circle according to the following scheme:

```
difference() {  
    linear_extrude(100)  
    circle($diam);  
    . . .  
}
```

For the sake of didactics, it should be noted that prefixing variable names with \$ is only necessary if you want to influence the system variables responsible for rendering accuracy. It is equally legitimate to create a variable with a "normal" name according to the following scheme:

```
$fn=256;  
diam = 25.9/2;  
difference() {  
    difference() {  
        linear_extrude(100)  
        circle(diam);  
    }  
}
```

However, the similarities between OpenSCAD and other programming languages end at this point. The following snippet would be equally valid from the perspective of C, Java and Co:

```
$fn=256;  
diam = 25.9/2;  
diam = diam + 1;
```

If you include it in the header of our previous example, you will get a wrong rendering. The following two messages also appear in the console:

```
WARNING: diam was assigned on line 2 but was overwritten on line 3  
WARNING: Ignoring unknown variable ,diam', in file element.scad, line 3.
```

The reason for this behavior, which at first glance appears confusing, is that an OpenSCAD variable is more of a "constant" than a classic variable. Also, it is in principle not permitted to use circular relationships to the content of a variable for the declaration according to the scheme $a=a+x$.

The error message indicates this in that the line `diam = diam + 1;` represents an invalid assignment, as OpenSCAD cannot access the value of the variable while processing `diam =`. Please note that this procedure only applies to the variable itself. The header taken from one of the following examples presents this by means of a group of mathematical operations - note, for example, the variable `boxmax`, which is generated from an addition against the previously defined value of the variable `boxwidth`:

```
$fn=256;
walldepth=4;
minkDia=2;
boxwidth=72 +2*minkDia;
boxminheight=40;
boxmaxheight=80;
boxmax = boxwidth + 2* walldepth+8;
boxf = boxwidth + 2* walldepth;
```

Another important Gotcha concerns the way OpenSCAD is processed. An OpenSCAD program is not executed dynamically like a C or Visual Basic program but is parsed in the first step and then processed against the construction area. This results in a further quirk, which we want to illustrate with the following snippet:

```
a = 0;
echo(a);
a = 3;
echo(a);
a = 5;
```

With the echo command, OpenSCAD contains a tool that allows the developer to carry the value of a variable to the outside world after or during the creation of the CSG tree. In a normal program, we would expect the output of different values here. However, anyone who executes the snippet in OpenSCAD will, in current versions of the program, receive a group of variables and the following result, which at first glance seems confusing:

```
Parsing design (AST generation)...
Saved backup file: /home/tamhan/.local/share/OpenSCAD/backups/unsaved-
backup-XMTJ7830.scad
WARNING: a was assigned on line 2 but was overwritten on line 4
WARNING: a was assigned on line 4 but was overwritten on line 6
Compiling design (CSG Tree generation)...
ECHO: 5
ECHO: 5
Compiling design (CSG Products generation)...
```

Both the first and second calls to echo work with the final variable value.

To understand this situation, we have to think again of the statement just made about the compilation process. In the first step, the parser processes the program completely. This means that it also includes the various variable value assignments. At the end it creates a syntax tree, in which the variable A has the last assigned value, in this case five.

Since this leads to namespace pollution to a certain extent, OpenSCAD version 2015.03 brought a group of extensions to the programming standard. Specifically, developers can now use a limited version of the block function known from C to generate variables with a

limited visibility period.

To illustrate the possibilities, let's look at the following example, which repeats the multiple assignment to a variable performed above:

```
a=0;
echo("Before the Block", a);
if (a==0)
{
    a=1;
    echo("In the Block", a);
}
echo("After the Block", a);
```

In contrast to the previous example, however, we now use an If selection, which opens a new block via its curved brackets. During program execution, the three echo statements then present themselves as follows:

```
Compiling design (CSG Tree generation)...
ECHO: "Before the Block", 0
ECHO: "In the Block", 1
ECHO: "After the Block", 0
```

Creating a block allows the redefinition of a variable. Within the block, the variable starts a new life cycle, but the defined values expire - as can be seen from the message ECHO:

```
"After the block", 0 - at the end of the block.
```

It should be noted that the rule with multiple assignments also applies within the block - if we were to change the value of A several times within the block, all statements in the block would calculate with the last value.

To dissuade OpenSCAD users from filling the generator files with empty blocks, it should be noted that a block that is not introduced with an operator or a selection does not open a new variable context. This can be checked by removing the if selection according to the following scheme:

```
a=0;
echo("Before the Block", a);
{
    a=1;
    echo("In the Block", a);
}
echo("After the Block", a);
```

If you run the program again at this point, you will notice that all three calls use the value one. However, this is also logical, since its assignment is "last" in the file:

```
ECHO: "Before the Block", 1
ECHO: "In the Block", 1
ECHO: "After the Block", 1
```

However, as in so many areas of computer science, there is no academic rule for which the practical computer scientist cannot find a way around. In the case of OpenSCAD, the means of choice is to create "tautological" selections - a good example of this would be the following block, which always has a true condition:

```
a=0;
echo("Before the Block", a);
if(1==1){
    a=1;
    echo("In the Block", a);
}
echo("After the Block", a);
```

7.2 Modules create geometry

From a certain degree of complexity of the model, one wishes to be able to manage recurring model parts centrally. A good example of this was the experiments on the commutativity of various operators carried out above, which took place using the following small demo object.

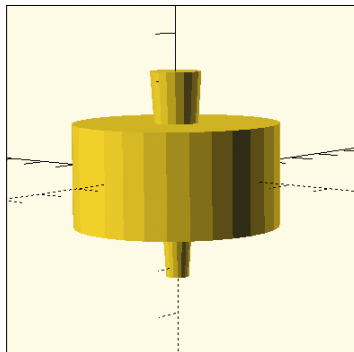


Figure 7-1. This dreidel enabled interesting experiments.

Since the dreidel object was in the form of a module, the actual test code was comparatively compact. Be that as it may, we can turn to the general declaration of a module:

```
module name ( parameters ) { actions }
```

At first glance, our module behaves like a function - next to a name that enables the call, there is a parameter list and a field with the object "to be animated". At this point, the author would like to explicitly point out that modules cannot return values to their callers - instead, a module takes care of creating "geometry". If you want to perform calculations, a function is the means of choice.

In any case, we will turn to the definition of an example module that creates geometry itself:

```
module tamsDreidel(dia=22)
{
    cylinder(dia, dia, dia, true);
    cylinder(dia*2, dia/9, dia/4, true);
}
tamsDreidel();
```

Interesting here is the parameter `dia=22`, which is equipped with a default value. This procedure is not necessary for OpenSCAD - as we would see later, it would also be possible to create a module without default parameters.

At the moment, however, we are quite happy with the default value, because you can simply call the module via `tamsDreidel()`. The reward for the effort is the sample geometry shown above.

From logic, it follows that you may pass other values to `dia`. The syntax used here is based on what we know from 2D or 3D generator functions:

```
module tamsDreidel(dia=22)
{
    cylinder(dia, dia, dia, true);
    cylinder(dia*2, dia/9, dia/4, true);
}
tamsDreidel();
translate([0,-60,0])tamsDreidel(10);
```

Thanks to the translation used here, the two elements now appear side by side, as shown in figure 7-2.

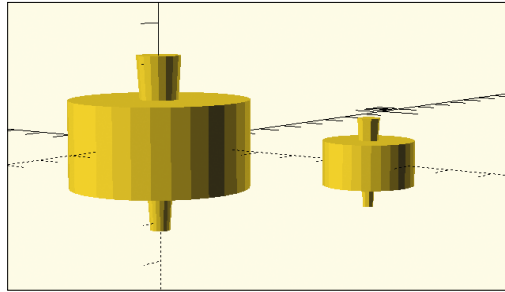


Figure 7-2. Thanks to parameter transfer and translation, we now see one large and one small object.

At this point we would like to venture a small experiment by removing the default parameter:

```
module tamsDreidel(dia)
{
    cylinder(dia, dia, dia, true);
    cylinder(dia*2, dia/9, dia/4, true);
}
tamsDreidel();
```

Funnily enough, running the dreidel without parameters - as shown in Figure 7-3 - leads to the appearance of a "tiny" element near the origin of the coordinate system.

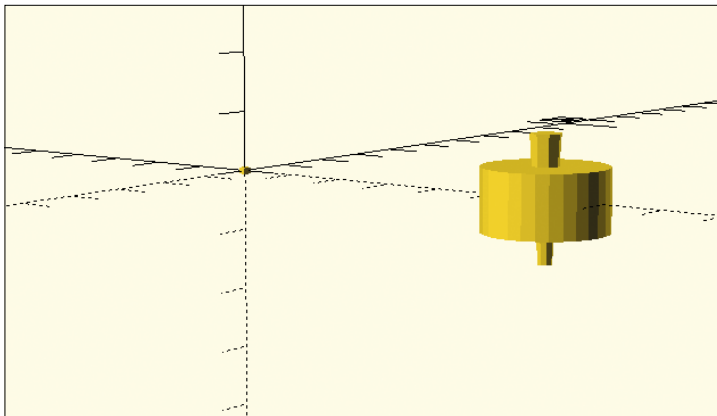


Figure 7-3. An unparameterized dreidel looks strange.

To find out what's happening in the background, we now place a call to the echo function in the module's head:

```
module tamsDreidel(dia)
{
    echo(dia)
```

During the execution of the program, the console will display the following message:

```
ECHO: undef
```

OpenSCAD considers variables that have never been assigned as having the "magic value" `undef`. Funnily enough, `undef` can be used in calculations - according to the following example, we could issue an error message if our dreidel module detects that the value is undefined:

```
module tamsDreidel(dia) {
  if(dia==undef) {
    echo ("ERROR");
  }
  cylinder(dia, dia, dia, true);
  cylinder(dia*2, dia/9, dia/4, true);
}
tamsDreidel();
```

Modules are - logically - not limited to a single parameter. The easiest way is to simply insert two additional parameters according to the following scheme - they will then logically be found in the invocation:

```
module tamsDreidel(dia, top, bot)
{
  echo(dia)
  cylinder(dia, dia, dia, true);
  cylinder(dia*2, top, bot, true);
}
tamsDreidel(22,3,4);
```

The situation becomes more interesting when our module brings along default values in addition to several parameters, but these are not distributed over the entire parameter set. A simple way to test the program behavior would be the following call:

```
module tamsDreidel(dia, top=4, bot)
{
  echo(dia)
  cylinder(dia, dia, dia, true);
  cylinder(dia*2, top, bot, true);
}
tamsDreidel(10, bot=4);
```

The two new parameters `top` and `bottom` influence the width of the pin running through the body. Figure 7-4, therefore, allows us to conclude what is happening in the background.

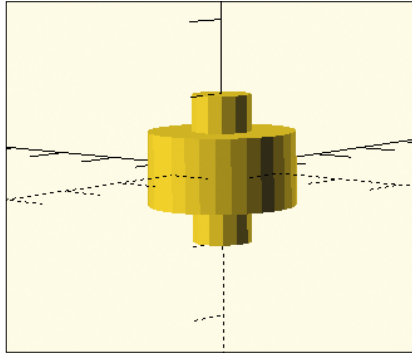


Figure 7-4. Since the two ends of the pin are the same width, the values of top and bottom are identical.

OpenSCAD proceeds "greedily" when filling out the parameter list - in the same way as other programming languages. This means that, as shown in Fig. 7-5, the parameters are used Tour a Tour to fulfil the parameter list of the module. The only "exception" is the use of named parameters according to the scheme bot=4;

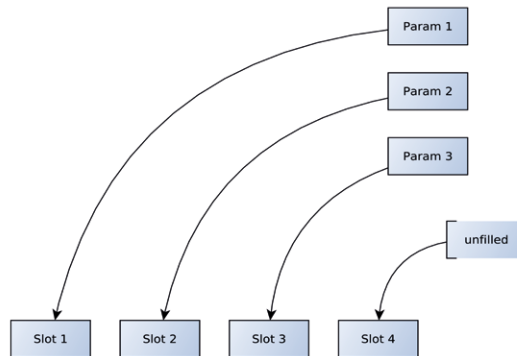


Figure 7-5. Parameter assignment logic is comparatively simple.

In the interest of better maintainability and clarity, you should make sure that parameters with default values are placed "at the end" of the parameter list.

Last but not least - let's temporarily turn to another topic - the following structure is demonstrated:

```
module sphere(){
    square();
}
sphere();
```

The snippet does exactly what you would expect it to do - it replaces the sphere module with a call to square. Modules contained in OpenSCAD can be completely replaced with in-house modules, the overwritten OpenSCAD primitives are then no longer callable.

A legitimate use for this procedure is the use of 2D-3D composite objects, which are in some cases superior to ordinary generated objects. In this case, however, make sure that users are trained - overwriting a primitive contained in the programming language is one of the measures that causes confusion very quickly.

7.3 Selections react to parameter states

In an ideal world, a model is completely parametric - this means that dimensions, coordinates et al can be calculated automatically. In practice, this is not readily achieved - a nice example would be a "drill hole" that has to accommodate a standardized screw.

If one determines its diameter exclusively from the model coordinates, a multitude of "intermediate values" would result, which do not correspond to any valid standard values found at your screw shop next door. In theory, creating custom screws is a workaround - anyone who ever used a CNC lathe in anger will willingly advise you against this approach.

To circumvent this problem, OpenSCAD offers selections. They behave - at first glance - in the same way as their colleagues known from C and Java. In practice, however, there are some "hairs" which we will look at more closely in the following steps. First, however, a look at the structure given in the documentation:

```
if (test) scope1
if (test){scope1}
if (test) scope1 else scope2
if (test){scope1} else {scope2}
```

In principle, the operator behaves as known from other programming languages - besides the possibility to create one block at a time, you can also use single statements.

Of particular interest is the implementation of Else If statement, which is not done using a special keyword. Instead, the first step is an If selection, whose Else block includes another If structure according to the following scheme This procedure, which at first glance appears confusing, works without problems in various other programming languages:

```
if(test1) {scope1}
else if(test2) {scope2}
else if(test3) {scope3}
else if(test4) {scope4}
else {scope5}
```

Before we turn to the practical applications and limitations of If, the following structure is shown. It introduces - analogous to C - the `?:`-operator, which we used to dynamically adjust the value of `$fn` depending on the type of rendering process:

```
a = test ? TrueValue : FalseValue ;
echo( test ? TrueValue : FalseValue );
```

7.4 Selections, modules, children

As the first productive example, we want to return to the dreidel element, and use the following scheme to make sure that the color of the created geometry depends on the value of dia. In theory, we could do this as follows:

```
module tamsDreidel(dia, top=4, bot)
{
    if(dia>12)color("red");
    else color("green");
    cylinder(dia, dia, dia, true);
    cylinder(dia*2, top, bot, true);
}
tamsDreidel(10, bot=4);
```

If you release this version of the program for execution, you will notice that the preview rendering is not executed correctly. The reason for this is the exact type of processing - the table shows what the module generates in the case of the two different scenarios for geometry elements.

If active	Else active
<pre>color("red"); cylinder(dia, dia, dia, true); cylinder(dia*2, top, bot, true);</pre>	<pre>else color("green"); cylinder(dia, dia, dia, true); cylinder(dia*2, top, bot, true);</pre>

Placement of {} does not matter.

Computer scientists love to argue about where the opening curved brackets go in an if selection. OpenSCAD is agnostic in this area - both spellings are equally accepted.

The color operator stands "alone" in both cases. A brutal way to solve the problem would be to provide the dreidel class with a more complicated selection according to the following scheme:

```
module tamsDreidel(dia, top=4, bot)
{
    if(dia>12){
        color("red"){
            cylinder(dia, dia, dia, true);
            cylinder(dia*2, top, bot, true);
        }
    }
    else{
        color("green") {
            cylinder(dia, dia, dia, true);
            cylinder(dia*2, top, bot, true);
        }
    }
}
```

```
    }  
  }  
}  
tamsDreidel(10, bot=4);
```

If selection now has its own geometry block, this program works without problems but does not fulfil its purpose. Each branch of the If loop brings its own geometry block, which is why we have quasi reversed the reduction of the geometry elements by introducing the module.

In the world of OpenSCAD, modules are "cheap" elements that the developer quickly creates and discards - their lifecycles are not accompanied by significant computing power overhead. For this reason, it is perfectly legitimate to create an additional worker module according to the following scheme, which is only called in the main module.

```
module tamsDWorker(dia, top, bot) {  
    cylinder(dia, dia, dia, true);  
    cylinder(dia*2, top, bot, true);  
}  
module tamsDreidel(dia, top=4, bot) {  
    if(dia>12) color("red")tamsDWorker(dia, top, bot);  
    else color("green")tamsDWorker(dia, top, bot);  
}  
tamsDreidel(10, bot=4);
```

7.5 Reject parameters with Assert

The more "rigid" the structure of a library, the more difficult it is for the end-user to cause damage in it. A nice tool to give an OpenSCAD library additional rigidity is the introduction of assertions.

The programming scheme known, for example, from the world of C is a kind of deflector that checks one or more conditions and, if the respective criteria are not met, terminates the processing of the module with an error.

To keep the situation "realistic" at least to a some extent, we want to secure our dreidel module in that it should only work with the OpenSCAD version installed on the author's workstation. To do this, we must - logically - first determine what version is installed on the terminal:

```
echo ( version());  
echo ( version_num());
```

The first version of this command returns a vector with the individual version components, while the second version returns the information as an integer:

```
ECHO: [2020, 6, 16]  
ECHO: 2.02006e+7
```

OpenSCAD has the unpleasant characteristic of switching to the exponential format familiar from scientific calculators when a certain number size is reached. Be that as it may, we want to introduce an assertion according to the following scheme in the next step:

```
module tamsDWorker(dia, top, bot) {
    cylinder(dia, dia, dia, true);
    cylinder(dia*2, top, bot, true);
}

module tamsDreidel(dia, top=4, bot) {
    assert(version()==[2020, 6, 16]);
    if(dia>12)    color("red")tamsDWorker(dia, top, bot);
    else color("green")tamsDWorker(dia, top, bot);
}

tamsDreidel(10, bot=4);
```

If you - like the author - are working with a test version of OpenSCAD compiled around the publication of the book, the condition in Assert is fulfilled, which is why the dreidel appears on the screen. If you change one of the components of the vector or use a different variant of the program, you will see the error message shown in Figure 7-6. Assert informs you "where exactly" the error occurred and which functions were responsible for calling.

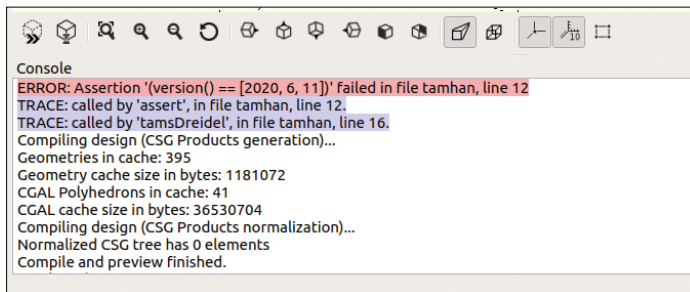


Figure 7-6. OpenSCAD is communicative when an Assert is fired.

It would be even nicer if we could issue a warning message optimised for his error situation. This is also no problem if you deliver an additional error string:

```
module tamsDWorker(dia, top, bot) {
    cylinder(dia, dia, dia, true);
    cylinder(dia*2, top, bot, true);
}

module tamsDreidel(dia, top=4, bot) {
    assert(version()==[2020, 6, 11], "Version falsch");
    if(dia>12)    color("red")tamsDWorker(dia, top, bot);
    else color("green")tamsDWorker(dia, top, bot);
}

tamsDreidel(10, bot=4);
```

The triggering of an assertion in the new version of the program leads to the error shown in Figure 7-7.

```
ERROR: Assertion '(version() == [2020, 6, 11])' failed: "Version falsch" in file tamhan, line 12
TRACE: called by 'assert', in file tamhan, line 12;
TRACE: called by 'tamsDreidel', in file tamhan, line 16.
Compiling design (CSG Products generation)...
Geometries in cache: 395
Geometry cache size in bytes: 1181072
CGAL Polyhedrons in cache: 41
CGAL cache size in bytes: 36530704
Compiling design (CSG Products normalization)...
Normalized CSG tree has 0 elements
Compile and preview finished.
```

Figure 7-7. The error message passed to Assert appears on the screen.

It should be noted that Asserts cannot only be used in modules - the feature can also be used in calculation functions and similar situations where it is a matter of "protecting" against undesired parameters.

7.6 Duplicate geometry with "for"

As a next task, we want to turn to another possibility to realise geometry "programmatically". This time, the oscilloscope shown in Figure 7-8 - Lecroy's of the time are famous for this problem - is to be used as an educational tool. Its buttons fall off over time.



Figure 7-8. The English, uh sorry, Swiss patient.

Some time ago, the author of these lines was asked by a friend to reproduce some of the buttons shown in figure 7-9 and 7-10.



Figure 7-9. On the outside of the button are six grooves...



Figure 7-10. ...while on the inside you find five small grooves and one large groove.

In order to illustrate the "advantages" of procedural geometry, let's first create an ordinary geometry. It starts, as always, with the declaration of the two cylinders that are responsible for the outer structure:

```
$fn=64;
difference(){
  tamsmodule();
  difference() {
    translate([0,0,8.5])cylinder(3, 22/2, 1/2, true);
    translate([0,0,7.6])cylinder(3, 22/2, 1/2, true);
  }
}
module tamsmodule(){
  difference(){
    cylinder(15.5, 12/2, 12/2, true);
    translate([0,0,-2])cylinder(13, 8.8/2, 8.8/2, true);
  }
}
```

The next step is the generation of the "inner" detents. Here it is noticeable that the "rotation" is done by the rotate method, which shows more and more rotation in the direction of the Z-axis tour a tour:

```
//inner detents
rotate([0,0,0])translate([12/2-2.2,0,0])cube([3.1-0.4,1.5,15.5],true);
```

```
    rotate([0,0,360/6])translate([12/2-2.2,0,0])
cube([1.5-0.4,1.5,15.5],true);
    rotate([0,0,360/6*2])translate([12/2-2.2,0,0])
cube([1.5-0.4,1.5,15.5],true);
    rotate([0,0,360/6*3])translate([12/2-2.2,0,0])
cube([1.5-0.4,1.5,15.5],true);
    rotate([0,0,360/6*4])translate([12/2-2.2,0,0])
cube([1.5-0.4,1.5,15.5],true);
    rotate([0,0,360/6*5])translate([12/2-2.2,0,0])
cube([1.5-0.4,1.5,15.5],true);
```

A similar procedure is used for the "outside" parts - here, too, the value of Rotate changes "gradually":

```
//outer detents
rotate([0,0,0])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
rotate([0,0,360/6])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
rotate([0,0,360/6*2])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
rotate([0,0,360/6*3])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
rotate([0,0,360/6*4])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
rotate([0,0,360/6*5])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
```

The only interesting fact here is that in the inner detents the very first object realizes a larger cube than its colleagues - this difference does not exist in the outer detents.

In the interest of a more beautiful appearance, the author then places a small additional cylinder on top:

```
    //some kind of lid
    translate([0,0,8.5])cylinder(2, 12/2, 1/2, true);
}
```

Already here it should be noted that the script worked in the present state - the printed buttons fitted (after a small adjustment of the originally measured geometry) on the LC334 concerned, were posted to Paris and did a great job there.

On the other hand, one has to admit that the program at hand did not win any prizes regarding its maintainability. It would be nicer if we could conjure up similar elements using a loop. The code necessary for this looks like this:

```
for(variable = [start : increment : end])
for(variable = [start : end])
for(variable = [vector])
```

The OpenSCAD language definition knows a total of three different for constructions. In addition to the simplest, which defines but a start and an endpoint, there are also variants

that use either an "in-house" increment or simply process all elements in a vector.

To find out more about the way the for loop is processed, let's iterate over an interval and spit out the values accumulated in the variable to the command line:

```
$fn=64;
for(var = [1:10])
    echo(var);
```

It should be noted that it is legitimate to release a whole block of instructions for interaction. Be that as it may, the results of the execution in the console are as follows:

```
ECHO: 1
ECHO: 2
ECHO: 3
ECHO: 4
ECHO: 5
ECHO: 6
ECHO: 7
ECHO: 8
ECHO: 9
ECHO: 10
```

It is particularly noticeable that OpenSCAD runs the command once for each value - this also applies to the start value and stop value. Armed with this knowledge, we can turn to the simplification of the outer detents. The new structure presents itself as follows:

```
for(var = [0:6])
    rotate([0,0,360/6*var])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
```

Instead of placing a call of the command for each node as before, we now have only one statement that generates the value of the rotation by multiplying the factor with var. That the program still works as expected can be verified by rendering "another" button.

With this, we turn to the inner Detents. The problem here is that "one" of the bridges is bigger than his colleagues. To solve this problem, code based on the following scheme is a good solution:

```
//inner detents
for(var = [0:5])
{
    if(var==0){
        rotate([0,0,0])translate([12/2-2.2,0,0])
        cube([3.1-0.4,1.5,15.5],true);
    }
    else{
```

```

        rotate([0,0,360/6 * var])translate([12/2-2.2,0,0])
cube([1.5-0.4,1.5,15.5],true);
    }

}

```

Besides the loop we also find a selection, which selects the generator command to be executed depending on the value of the variable. This can be checked for problems by looking at the bottom side - it now presents itself as shown in Figure 7-11.

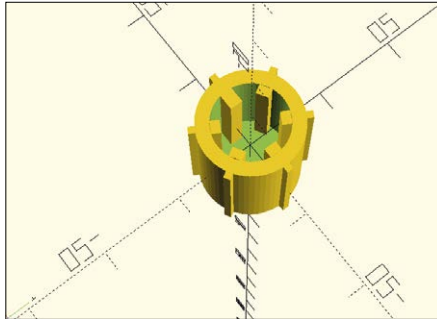


Figure 7-11. Even the complicated structure works without problems.

The question of when and whether the use of a loop is worthwhile can be discussed excellently. The author takes the same view of it as the famous Englishman with the toolbox - if you only have a hammer, you have to consider every problem as a nail to be hammered. If, on the other hand, you have other tools at hand, you can work more flexibly.

At this point, a special version of the for loop is presented, which is called `intersection_for` and illustrated with the following snippet:

```

intersection_for(n = [1 : 6])
{
    rotate([0, 0, n * 60])
    {
        translate([5,0,0])
        sphere(r=12);
    }
}

```

In principle, this is a for loop, the results of which are summarised by the intersection operator known from above. In the case of our small example taken from the OpenSCAD documentation, we would get the rendering shown in Figure 7-12.

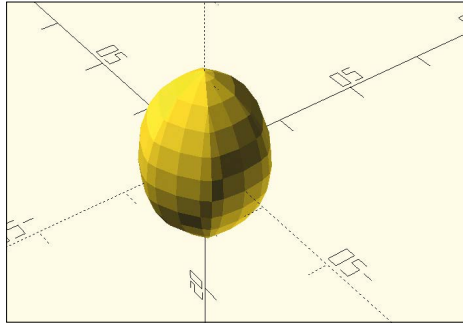


Figure 7-12. The `intersection_for` operator saves some code.

7.7 Functions and calculations

We had noted earlier that a module returns "one geometry". In practice, however, this is not always what you want - in many cases you want to return a number or a string instead. In this case, a function is the tool of choice - OpenSCAD brings along dozens of them, which implement various trigonometric and other mathematical relationships. Alternatively, it is also possible to create your own function according to the following syntax:

```
function name ( parameters ) = value ;
```

The most striking feature of this declaration is the use of the `=` character, which describes the value returned by the function. This also makes clear what role a function plays in OpenSCAD - it is concerned solely with returning a numeric or other value to its caller. The creation of geometry is explicitly not permitted.

To demonstrate what is possible with OpenSCAD functions, I would like to briefly touch on some examples from the documentation. First, a comparatively simple method that simulates a classical linear calculation:

```
function func1(x=3) = 2*x+1;
```

Apart from the syntax with the `=`, which is somewhat unusual for those switching over to the new system, the only interesting thing here is that the parameter `x` is assigned a default value "by default".

The user can thus call the function according to the scheme `func1()`, if he is satisfied with the default value. Another classic application purpose for OpenSCAD functions is the "switching" of discrete values in arrays or vectors. Here are two ready-made examples which are self-explanatory due to their low complexity:

```
function func2() = [1,2,3,4];
function func4(p0,p1,p2,p3) = [p0,p1,p2,p3];
```

The only thing that is important in connection with OpenSCAD functions is that you, as a developer switching from C, C++, or Java, have to rethink how you use them. In Open-

SCAD, a function is not a way to implement a "complicated" process - it is much more a matter of mapping numerical relationships.

This can be checked, for example, by visiting the OpenSCAD documentation at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/The_OpenSCAD_Language#Chapter_6_-_Other_Functions_and_Operators and searching for section Chapter 6 -- Other Functions and Operators. The resulting list of functions confirms this fact.

7.8 Processing lists

We had used vectors earlier, for example as parameters for the Translate operator. Since vectors always play a major role, I would like to briefly discuss some possibilities for their use here. In order not to let this work explode completely, I recommend that people with programming experience consult the syntax available at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/The_OpenSCAD_Language - OpenSCAD contains some (very difficult to use) comfort functions, with which experienced developers can shorten the geometry scripts considerably.

Be that as it may, first a look at the generation of vectors:

```
[1,2,3]
[a,5,b]
[]
[5.643]
["a","b","string"]
```

It is interesting that vectors can also record "mixed" contents or even sub-vectors:

```
[[1,r],[x,y,z,4,5]]
[3, 5, [6,7], [[8,9],[10,[11,12],13], c, "string"]
[4/3, 6*1.5, cos(60)]
```

When working with vectors, also note the difference to Ranges, which are created by using "colons":

```
r1 = [0:10];
r2 = [0.5:2.5:20];
```

A Range does not describe the sum of its elements. Note, however, that outputting a Range with echo leads to "strange" behavior - this is illustrated in the table below.

A range, obviously, describes a set of steps which the program will then iterate through in a tour-a-tour fashion.

Case A	Case B
for(worker=[0:10]) { echo (worker); }	for(worker=[0.5:2.5:20]) { echo (worker); }
ECHO: 0 ECHO: 1 ECHO: 2 ECHO: 3 ECHO: 4 ECHO: 5 ECHO: 6 ECHO: 7 ECHO: 8 ECHO: 9 ECHO: 10	ECHO: 0.5 ECHO: 3 ECHO: 5.5 ECHO: 8 ECHO: 10.5 ECHO: 13 ECHO: 15.5 ECHO: 18

Be that as it may, access to single elements of a vector is done - according to the following scheme - via the procedure known from C and Co:

```
e[5]           // element no 5 (sixth) at 1st nesting level
```

The most interesting thing about vectors is that they can be "grouped together" over a whole group of functions and modified in other ways. This helps, for example, when an existing polygon needs to be supplemented by a group of additional holes - the following snippet shown in the OpenSCAD documentation and the sample rendering shown in Figure 7-13 are good examples of this:

```
a0 = [[0,0],[100,0],[130,50],[30,50]];    // main
b0 = [1,0,3,2];
a1 = [[20,20],[40,20],[30,30]];           // hole 1
b1 = [4,5,6];
a2 = [[50,20],[60,20],[40,30]];           // hole 2
b2 = [7,8,9];
a3 = [[65,10],[80,10],[80,40],[65,40]];   // hole 3
b3 = [10,11,12,13];
a4 = [[98,10],[115,40],[85,40],[85,10]];  // hole 4
b4 = [14,15,16,17];
a  = concat (a0,a1,a2,a3,a4);
b  = [b0,b1,b2,b3,b4];
polygon(a,b);
//alternate
polygon(a,[b0,b1,b2,b3,b4]);
```

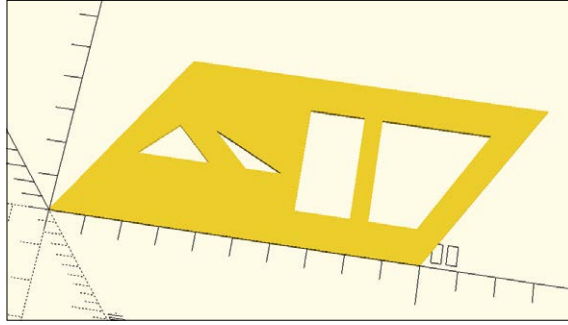


Figure 7-13. Thanks to vector operations, the insertion of the holes went off without any major changes.

The best way to illustrate the "cooperation" between ranges and vectors is to use the diagram rendering example provided by the OpenSCAD team. It leads to the result shown in Figure 7-14, which we will work through in the following steps.

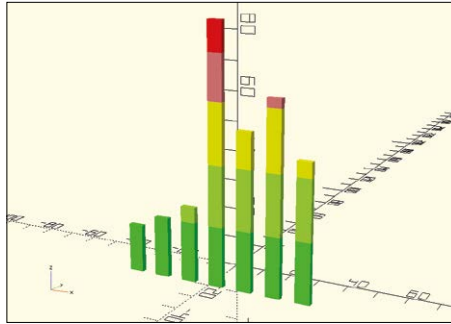


Figure 7-14. Data visualization a la OpenSCAD.

The entry point consists - as is often the case - of a "parent module" which is called in the file header and whose result is then moved to a more accessible location using a translate operation:

```
translate([-30,-20,0])
  ShowColorBars(Expense);
```

As in many other areas of computer science, a good part of the logic is in the form of a look up table. In the case of our diagram plotter, this is the array `ColorBreak`, which connects the respective limit values with color strings:

```
ColorBreak=[[0,"",
              [20,"lime"], // upper limit of color range
              [40,"greenyellow"],
              [60,"yellow"],
              [75,"LightCoral"],
              [200,"red"]];
```

Meanwhile, the Expense field contains the test data to be displayed:

```
Expense=[16,20,25,85,52,63,45];
```

In the next step, the collected information migrates into a For loop, which iterates over two values simultaneously:

```
module ShowColorBars(values){
  for (month = [0:len(values)-1], range = [1:len(ColorBreak)-1])
    ColorBar(values[month],month,range);
}
```

As is so often the case in the world of computer science, we can learn more about the behavior of the code by instrumenting it. For this purpose, an adaptation according to the following scheme is useful to output more information about the encountered values into the command line:

```
module ShowColorBars(values){
  for (month = [0:len(values)-1], range = [1:len(ColorBreak)-1])
  {
    echo(month);
    echo(range);
    echo("----");
    ColorBar(values[month],month,range);
  }
}
```

Whoever runs the program will see that all values of range are processed once for each value of Month:

ECHO: 0	ECHO: 1	ECHO: 2
ECHO: 1	ECHO: 1	ECHO: 1
ECHO: "----"	ECHO: "----"	ECHO: "----"
ECHO: 0	ECHO: 1	ECHO: 2
ECHO: 2	ECHO: 2	ECHO: 2
ECHO: "----"	ECHO: "----"	ECHO: "----"
ECHO: 0	ECHO: 1	ECHO: 2
ECHO: 3	ECHO: 3	ECHO: 3
ECHO: "----"	ECHO: "----"	ECHO: "----"
ECHO: 0	ECHO: 1	ECHO: 2
ECHO: 4	ECHO: 4	ECHO: 4
ECHO: "----"	ECHO: "----"	ECHO: "----"
ECHO: 0	ECHO: 1	ECHO: 2
ECHO: 5	ECHO: 5	ECHO: 5
ECHO: "----"	ECHO: "----"	ECHO: "----"

ECHO: 3	ECHO: "---"	ECHO: 4
ECHO: 1	ECHO: 4	ECHO: "----"
ECHO: "----"	ECHO: 3	ECHO: 5
ECHO: 3	ECHO: "----"	ECHO: 5
ECHO: 2	ECHO: 4	ECHO: "----"
ECHO: "----"	ECHO: 4	ECHO: 6
ECHO: 3	ECHO: "----"	ECHO: 1
ECHO: 3	ECHO: 4	ECHO: "----"
ECHO: "----"	ECHO: 5	ECHO: 6
ECHO: 3	ECHO: "----"	ECHO: 2
ECHO: 4	ECHO: 5	ECHO: "----"
ECHO: "----"	ECHO: 1	ECHO: 6
ECHO: 3	ECHO: "----"	ECHO: 3
ECHO: 5	ECHO: 5	ECHO: "----"
ECHO: "----"	ECHO: 2	ECHO: 6
ECHO: 4	ECHO: "----"	ECHO: 4
ECHO: 1	ECHO: 5	ECHO: "----"
ECHO: "----"	ECHO: 3	ECHO: 6
ECHO: 4	ECHO: "----"	ECHO: 5
ECHO: 2	ECHO: 5	ECHO: "----"

This allows us to turn to the actual rendering, which can be started with the command

```
ColorBar(values[month],month,range);
```

In the first step, it determines which value of ColorBreak needs to be processed - the data is then moved to an if selection, which creates only those Cubes that are marked as required by the requested Expense value:

```
module ColorBar(value,period,range){ // 1 color on 1 bar
    RangeHi = ColorBreak[range][0];
    RangeLo = ColorBreak[range-1][0];
    color( ColorBreak[range][1] )
    translate([10*period,0,RangeLo])
        if (value > RangeHi)      cube([5,2,RangeHi-RangeLo]);
        else if (value > RangeLo) cube([5,2,value-RangeLo]);
}
```

7.9 Tools for troubleshooting

Troubleshooting OpenSCAD can generally be reduced to "divide and conquer" - behind this at first glance flippant sounding formulation lies the idea of simplifying the geometry until the problematic part is in view.

However, in Preview Render mode, OpenSCAD offers a group of additional functions that help you find problems. It should be explicitly noted that these functions are created using the color engine, which - as stated above - is not available in production renderings.

By far the most common candidate is the # symbol, which you simply throw "somewhere" in a geometry tree - according to the following scheme

```
#for(var = [0:6])
  rotate([0,0,360/6*var])translate([12/2,0,0])cube([1.5,0.8,15.5],true);
```

From this point on, the affected elements appear red in the preview rendering - as shown in Figure 7-15 using our test object as an example.

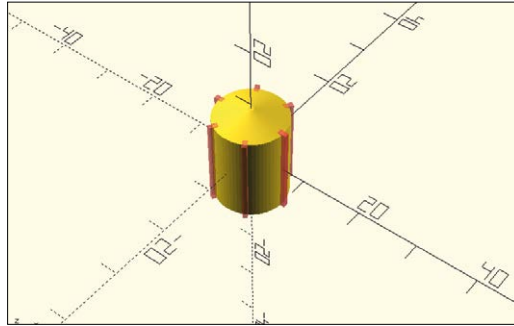


Figure 7-15. If you place the # in front of the For loop, it colors the entire geometry tree.

Of course, there is nothing to be said against addressing "a smaller" part of the geometry according to the following scheme:

```
//some kind of lid
  #translate([0,0,8.5])cylinder(2, 12/2, 1/2, true);
}
```

It should be noted that the # symbol generally works "commutatively". For example, if you place it in an object that is then transformed and scaled, the red color, which is usually clearly visible in the debugger, affects the "final" result of the operation.

Candidate number two is the percentage sign, which can be used analogously in terms of behavior:

```
//some kind of lid
%translate([0,0,8.5])cylinder(2, 12/2, 1/2, true);
}
```

The most interesting thing about it is that the behavior between preview and final rendering is very different. If you command a preview rendering of the structure shown here, you will see that the part became transparent as shown in Figure 7-16.

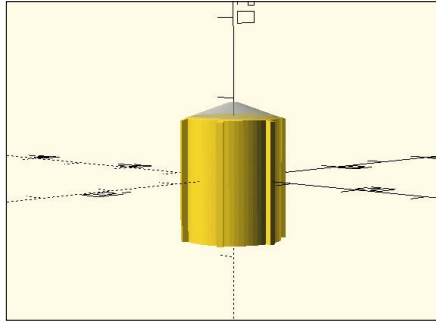


Figure 7-16. The transparency operator works.

Interestingly, it ensures that the geometry in question is not used at all in a production rendering. This can be illustrated with the following small example taken from the OpenSCAD documentation

```
difference() {
  cylinder (h = 12, r=5, center = true, $fn=100);
  // first object to be subtracted
  rotate ([90,0,0]) cylinder (h = 15, r=1, center = true, $fn=100);
  // second object to be subtracted
  %rotate ([0,90,0]) cylinder (h = 15, r=3, center = true, $fn=100);
}
```

Figure 7-17 shows that the geometry with the percentage sign is displayed transparently during the preview rendering, but is not taken into account at all in a productive rendering.

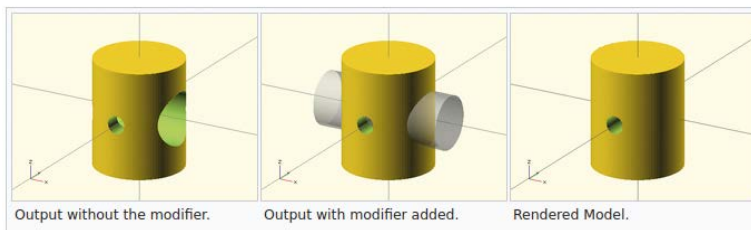


Figure 7-17. The percentage operator can be quite challenging.

In the introduction, we had stated that *divide et impera* is a convenient way to reduce a geometry. This is the life purpose of the `!` operator, which considers its "children" as the source element of the file and hides all remaining geometry. The correct program behavior is most easily illustrated by the following two images, each referring to the "tip" of the LeCroy oscilloscope knob.

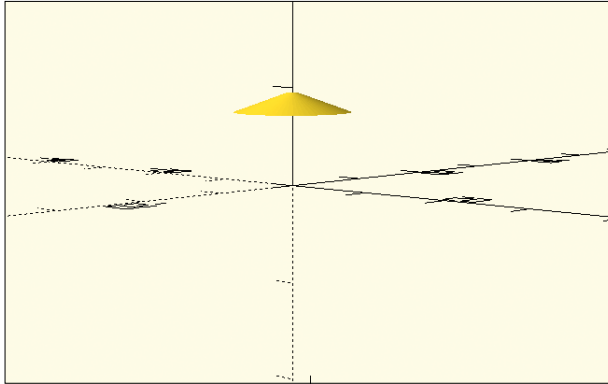


Figure 7-18. `!translate([0,0,8.5]) cylinder(2, 12/2, 1/2, true);`

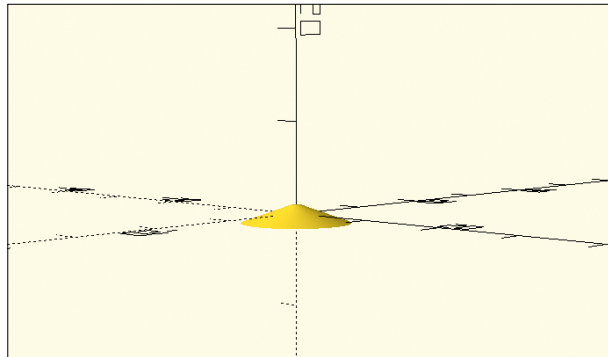


Figure 7-19. `translate([0,0,8.5])!cylinder(2, 12/2, 1/2, true);`

The fourth and last part of the debugger operators then takes care of "sending a piece of geometry to Astoria without replacement". I would also like to use two images to illustrate the star operator.

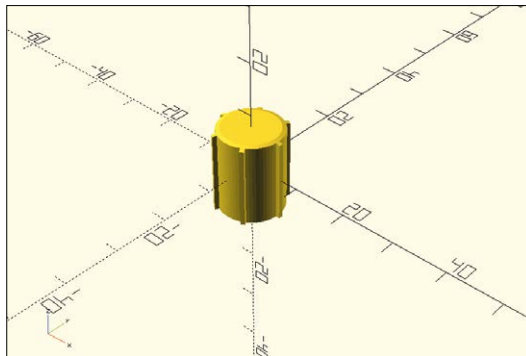


Figure 7-20. `*translate([0,0,8.5])!cylinder(2, 12/2, 1/2, true);`

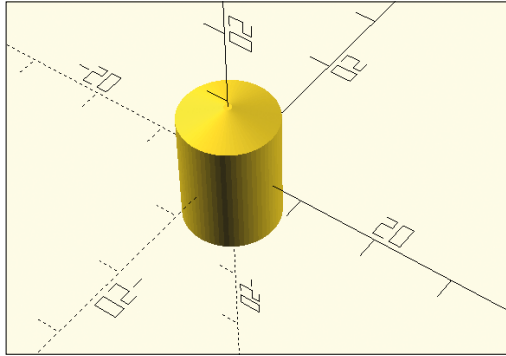


Figure 7-21. `*for(var = [0:6]) rotate([0,0,360/6*var])translate([12/2,0,0])
cube([1.5,0.8,15.5],true);`

Note that OpenSCAD also supports both comment schemes known from the C language standard - you can create both single-line comments per `//` and multi-part comments:

```
// This is a comment

myvar = 10; // The rest of the line is a comment

/*
  Multi-line comments
  can span multiple lines.
*/
```

However, the star operator is often more convenient in practice, because you don't have to worry about opening and closing curved brackets and similar funny things when using it.

7.10 Conclusion

Even though OpenSCAD is not a full-fledged programming language in the conventional sense, valuable time can be saved by intelligently using the various automation and generation options. OpenSCAD can also be integrated into "code generators" - if you write a program using Visual Basic that outputs OpenSCAD code, you can implement even very complex elements comparatively easily. Also, there is now also a Python connection, which I will briefly illustrate in Chapter 11.

The next steps in the world of OpenSCAD, however, are concerned with the output of texts. There is also a lot to see in this area - so stay with us.

Chapter 8 • Texts, projections and bump mapping

After the - sometimes extensive - fights in the Arena Coders against OpenSCAD syntax, we have now reached the lofty heights where most of the work with the OpenSCAD language is behind us. This does not mean, however, that there are no interesting aspects to be seen. The tools presented in this chapter are certainly not required for "every" model. However, since there are always interesting use cases, the author asks for attention once more.

8.1 Render texts

In theory, we could have placed the explanations on displaying texts with OpenSCAD in previous chapters. From the perspective of OpenSCAD, a text or its rendering is a very complicated two-dimensional polygon, but it can otherwise be extruded like any other polygon.

Version your printouts

The author of this work works with the UnME2 group, which packs its Java-enabled school bus trackers (SpotMyBus) into 3D printed and OpenSCAD-designed cases. As the design of the cases changes over time, the UnME2 group prints a version number on the inside of the cases - this has proven to be very helpful in practice more than once and is a recommended course of action.

The actual triggering of the display is done by the command `Text`, which I would like to present to you wrapped in a for-loop:

```
$fn=64;
for ( i = [0 : 5 : 40] ){
    translate([i*5, i*5]) {
        text("Tamoggemon", size = i);
    }
}
```

According to the OpenSCAD documentation, the `Text` module accepts a whole group of different parameters, which are listed in figure 8-1.

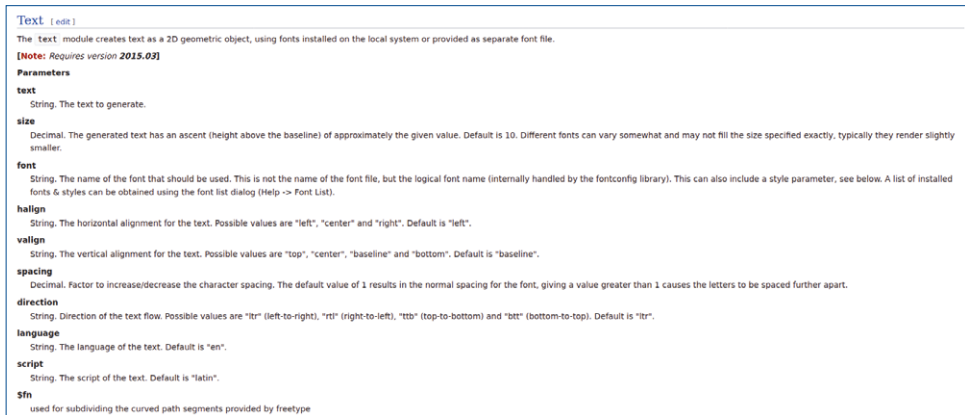


Figure 8-1. Basic typographical knowledge is helpful.

For us, only the size parameter is of interest here, which allows our program to set the font size. The for-loop iterates over a range of values, creates one text each and moves it from the origin to another position using the Translate operator.

In any case, the reward for the effort is the behaviour shown in figures 8-2 and 8-3. Please note that rendering text is a computationally intensive task - the author's workstation needed a few seconds for the rendering of our example.

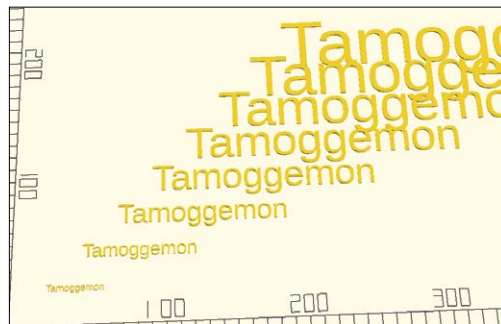


Figure 8-2. In preview mode, texts with geometric extension appear three-dimensional...



Figure 8-3. ...while they are two-dimensional in a production rendering.

8.2 A question of alignment

Our program example has so far proved to be a special case in that it immediately moved the calculated texts. Next, we simply want to render a text according to the following scheme, which leads to the behavior shown in figure 8-4:

```
text("Hallo!");
```

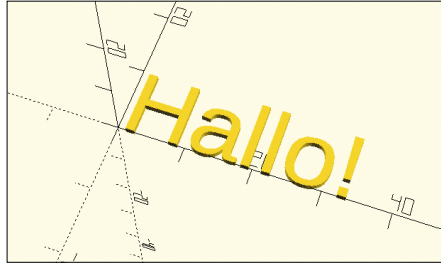


Figure 8-4. The rendering output appears near the origin.

To influence the display, OpenSCAD has two parameters called `valign` and `halign`, which in terms of their behaviour are based on their colleagues known from typography. First, I would like to turn to vertical alignment, using an example taken from the OpenSCAD documentation. To understand the behavior of the program, it is reasonable to first look at its rendering shown in figure 8-5.

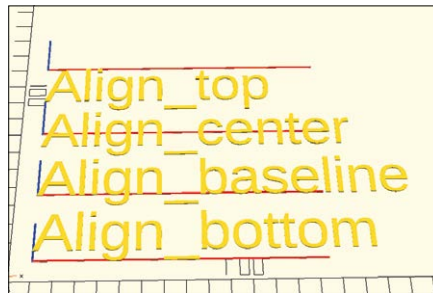


Figure 8-5. The `VALIGN` attribute influences the positioning of the texts in the rendering result.

This program generates four texts which are then moved using the `Translate` operator. For this reason, the code generates two rectangles describing the position of the origin used for rendering the respective texts. The structure around `valign` is an array with a lookup table:

```
text = "Align";  
font = "Liberation Sans";  
  
valign = [  
  [ 0, "top"],  
  [ 40, "center"],  
  [ 75, "baseline"],
```

```

[110, "bottom"]
];

for (a = valign) {
  translate([10, 120 - a[0], 0]) {
    color("red") cube([135, 1, 0.1]);
    color("blue") cube([1, 20, 0.1]);
    linear_extrude(height = 0.5) {
      text(text = str(text,"_",a[1]), font = font, size = 20, valign =
a[1]);
    }
  }
}

```

Besides a method for vertical alignment, there is also a colleague called `halign` or parameter that takes care of the horizontal alignment. Here, we also want to look at the example rendering shown in figure 8-6 as a first step.

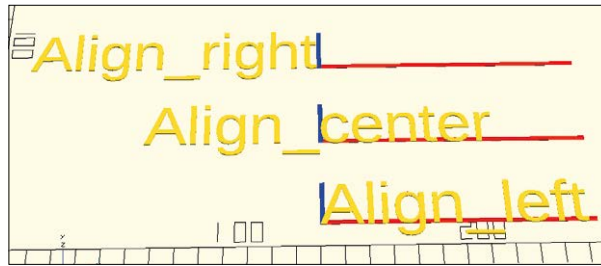


Figure 8-6. `HALIGN` behaves similarly but influences the other axis.

In the program example, also taken from the OpenSCAD documentation, no rocket science can be found. A for loop iterates over the contents known from above and carries the coordinate cross:

```

text = "Align";
font = "Liberation Sans";

halign = [
  [10, "left"],
  [50, "center"],
  [90, "right"]
];

for (a = halign) {
  translate([140, a[0], 0]) {
    color("red") cube([115, 2,0.1]);
    color("blue") cube([2, 20,0.1]);
    linear_extrude(height = 0.5) {

```

```

text(text = str(text,"_",a[1]), font = font, size = 20, halign =
a[1]);
}
}
}

```

8.3 Add and manage fonts

Anyone who has ever dealt with font licensing professionally - whether in an advertising agency or a print shop - knows about the complexity of this problem. Also, the fonts supplied differ from operating system to operating system and from computer to computer. The OpenSCAD development team addresses this problem by delivering the following three fonts, which are available on all supported platforms:

- Liberation Mono
- Liberation Sans
- Liberation Serif

During system startup, OpenSCAD sniffs at the computer's configuration to detect additional font files. The search behaviour varies from platform to platform and should not slow us down any further - more interesting is the font list available under Help > Font List, which is displayed on the author's workstation as shown in Figure 8-7.

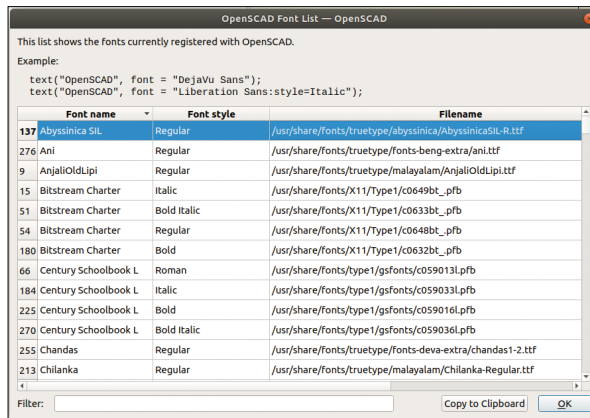


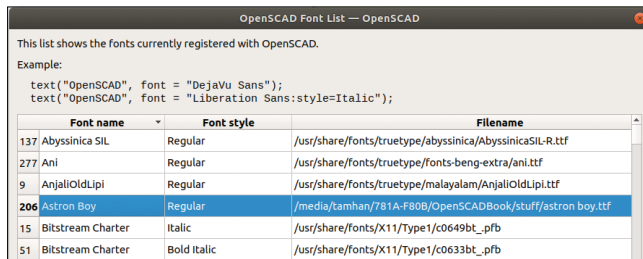
Figure 8-7. Ubuntu provides a lot of fonts.

Interesting here are the names shown in the Font Name column, which are passed to the text operator. It is also important how to add a new font. In the following steps, the author would like to work with the Astron Boy font developed by Raymond Larrabie, which is available for download at the URL <https://www.1001fonts.com/astron-boy-font.html>. Extract the archive to an easily accessible location in the file system.

Our next official act is to inform OpenSCAD of the existence of the new font file. This is done using the following declaration, which you can insert into an empty tab in the first step and execute as if it were a bit of geometry:

```
use </media/tamhan/781A-F80B/OpenSCADBook/stuff/astron boy.ttf>
```

The reward for the effort is that the new font appears in the font list as shown in figure 8-8.



The screenshot shows the 'OpenSCAD Font List' window. It contains a list of fonts with columns for 'Font name', 'Font style', and 'Filename'. The 'Astron Boy' font is highlighted in blue. Above the list, there is an example of how to use the font in a script.

	Font name	Font style	Filename
137	Abyssinica SIL	Regular	/usr/share/fonts/truetype/abyssinica/AbyssinicaSIL-R.ttf
277	Ani	Regular	/usr/share/fonts/truetype/fonts-beng-extra/ani.ttf
9	AnjaliOldLipi	Regular	/usr/share/fonts/truetype/malayalam/AnjaliOldLipi.ttf
206	Astron Boy	Regular	/media/tamhan/781A-F80B/OpenSCADBook/stuff/astron boy.ttf
15	Bitstream Charter	Italic	/usr/share/fonts/X11/Type1/c0649bt_pfb
51	Bitstream Charter	Bold Italic	/usr/share/fonts/X11/Type1/c0633bt_pfb

Figure 8-8. The new font is now available.

For the actual use, not much intelligence is required. A generator script structured according to the following scheme creates a line in the new font, as shown in Figure 8-9:

```
use </media/tamhan/781A-F80B/OpenSCADBook/stuff/astron boy.ttf>

translate([15, 15]) {
  text("Tamoggemon", font = "Astron Boy");
}
```

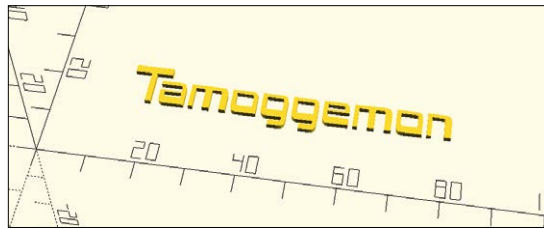


Figure 8-9. Font rendering works without problems.

OpenSCAD must remember fonts added via a use declaration until the program is restarted. This is critical, among other things, because a missing font is simply replaced at runtime by one of the standard fonts contained in the program, which sometimes leads to considerable problems. The author, therefore, recommends to always leave the use declaration in the generator code.

8.4 Exporting 2D Snapshots

OpenSCAD is normally used to create three-dimensional models, which are then produced using 3D printers and similar hardware. In practice, however, there are also cases where basic two-dimensional graphics are required, for example, for sending to customers or for inserting comments.

OpenSCAD supports us in this case with a function known as projection. To understand exactly how a projection works, we want to work with the object shown in figure 8-10.

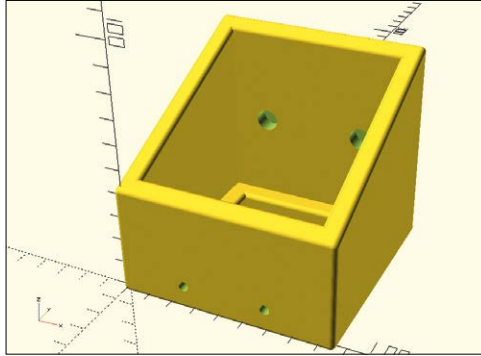


Figure 8-10. This soap dispenser holder should be known from the introduction.

Of special interest are the holes on the front and back. As a "didactically malicious" animal, the author decided to make only two of the holes completely continuous.

Be that as it may, the first task is to create a module containing the geometry. The operator is declared in OpenSCAD according to the following scheme:

```
projection(cut = ?) object();
```

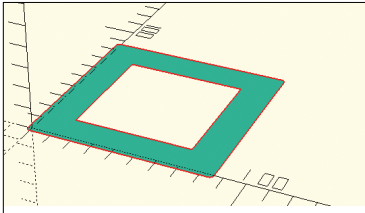
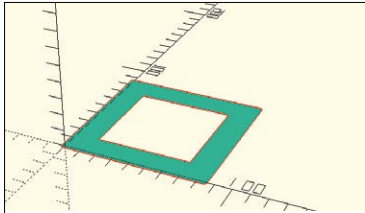
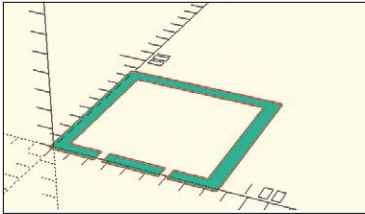
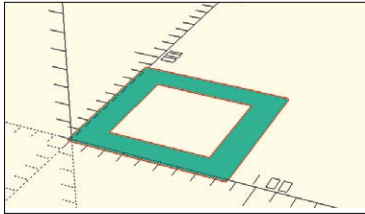
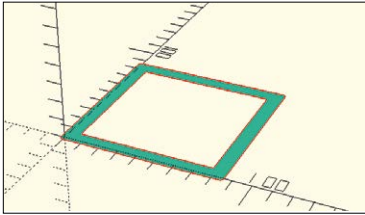
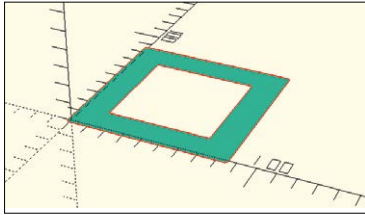
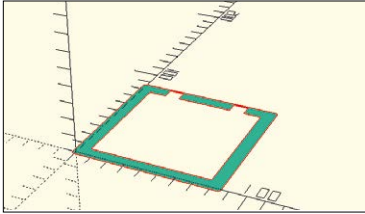
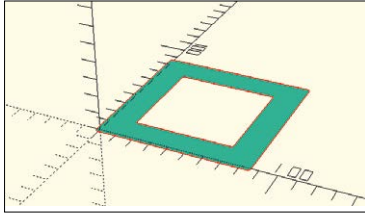
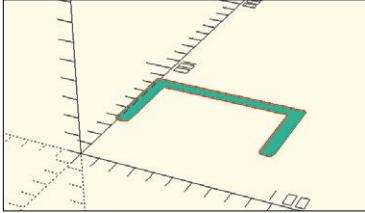
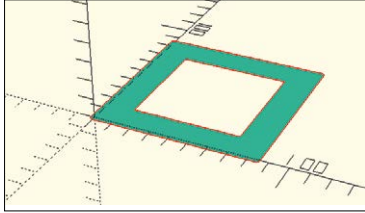
In theory, a projection can also be applied against the result of "ordinary" operators according to the following scheme

```
translate([0,0,25]) rotate([90,0,0]) cube();
```

Normally, more complicated geometries are used, for example, according to the following scheme:

```
projection(cut = true) translate([0,0,-40])TamsCube();
```

Of particular importance here is the cut operator, which describes the type of projection to be performed. For easier understanding, the author now shows a table of cut images that touch the object at different heights.

Command	Cut = true	Cut = false
<code>projection(cut = ?)</code> <code>TamsCube();</code>		
<code>projection(cut = ?)</code> <code>translate([0,0,-8])</code> <code>TamsCube();</code>		
<code>projection(cut = ?)</code> <code>translate([0,0,-18])</code> <code>TamsCube();</code>		
<code>\$fn=64;</code> <code>projection(cut = ?)</code> <code>translate([0,0,-40])</code> <code>TamsCube();</code>		
<code>projection(cut = ?)</code> <code>translate([0,0,-60])</code> <code>TamsCube();</code>		

A skim of the results shown in the table demonstrates how OpenSCAD performs the projection process. The product works with the X-Y plane and normally captures all elements that are located on the plane at a Z coordinate value of zero. This also explains why the individual holes can be transferred "into the projection" by adjusting the values of the translate.

The cut parameter allows you to define the calculation process used to create or execute the projection. If you - like the author - assume cut as true, OpenSCAD "cuts" completely through the model and returns only those pixels which are set on the cut plane. If we set cut to False instead, we get the result of a mirroring.

A good way to imagine this procedure analogously is to use a lamp that illuminates the target object from the Z direction at the top. On the X-Y-plane there is a film that presents the results of the exposure.

From the logic, it follows that there is nothing to be said against rotating or otherwise moving an object before projection. Figure 8-11, for example, shows the result of the following transformation operation:

```
projection(cut = true)
  rotate([-90,0,0])
  TamsCube();
```

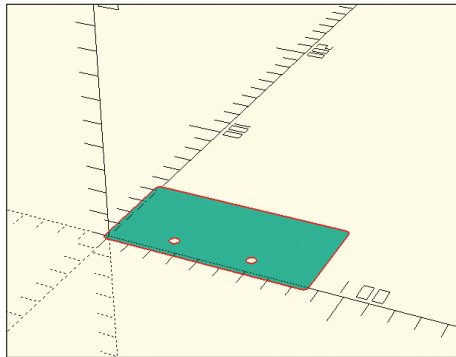


Figure 8-11. Rotations allow you to "adjust" the cut surface used.

Careful consideration of the results in the table informs us that the OpenSCAD projection function is ideally suited to "looking more closely" at critical parts of the model - the holes on the back that are not completely drilled through, for example, are much easier to see in the sectional image than in a three-dimensional rendering.

With this, we want to turn to the second task - the export of the two-dimensional object. Whoever commands a "final" rendering and then clicks on the STL button in the toolbar will be rewarded with the following error message in the console:

```
UI-ERROR: Current top level object is not a 3D object.
```

This means that OpenSCAD checks whether the geometry in memory or the cache is three-dimensional before processing an STL export command.

Be that as it may, the menu shown in figure 8-12 contains a whole group of export commands which enable the generation of frequently used exchange formats in the CAD space.

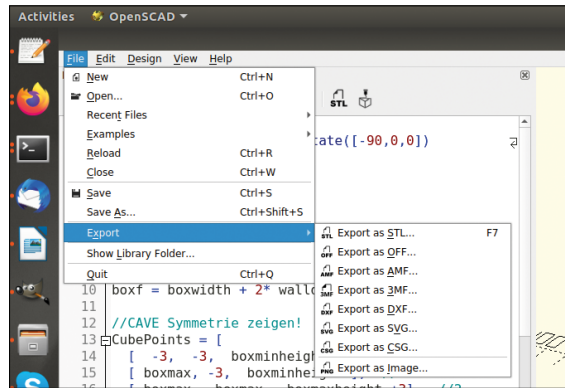


Figure 8-12. These menu options help in communication with pavers and customers.

It should be noted that the menu options shown in Figure twelve also perform checks to verify the "validity" of the information in the rendering buffer. This can be checked, for example, by rendering a three-dimensional model and then commanding the output of a two-dimensional file - the program thanks you with an error:

```
UI-ERROR: Current top level object is not a 2D object.
```

Finally, it should be noted that OpenSCAD can be used to edit "already existing" STL files. A command structured according to the following scheme is used for this purpose:

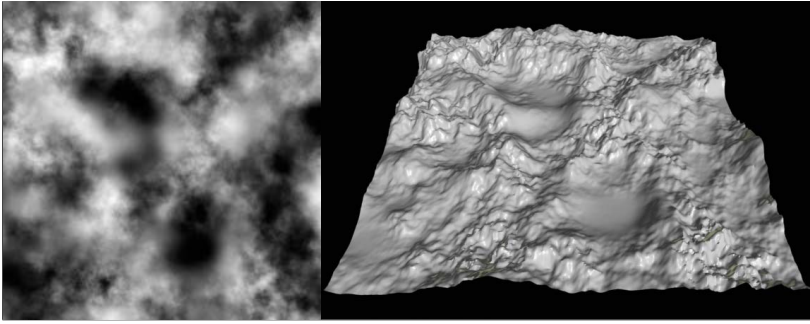
```
projection(cut=false) import_stl("/full/path/to/stl");
```

The `import_stl` command imports the STL file, which OpenSCAD then treats as a three-dimensional object created by generator commands.

8.5 Import geometry item maps

In practice, you often encounter situations in which geometry data is available in the form of images known as bump or height maps. This is based on the idea that the color information shown or stored in the bitmap describes the "height" of the individual pixels.

A nice example is shown in the pictures next - it compares a heightmap and the geometry generated from it.



Picture source: Wikimedia Commons

From a technical point of view, surface behaves similarly to the `import_stl` command we just looked at:

```
surface(file = "surface.dat", center = true, convexity = 5);
surface(file = "smiley.png", center = true);
```

The most interesting feature is that the file parameter can be used to deliver both .dat files, which are popular in the scientific field, and ordinary .png files. A DAT file is usually a number table structured according to the following scheme:

```
#surface.dat
10 9 8 7 6 5 5 5 5 5
9 8 7 6 6 4 3 2 1 0
8 7 6 6 4 3 2 1 0 0
7 6 6 4 3 2 1 0 0 0
6 6 4 3 2 1 1 0 0 0
6 6 3 2 1 1 1 0 0 0
6 6 2 1 1 1 1 0 0 0
6 6 1 0 0 0 0 0 0 0
3 1 0 0 0 0 0 0 0 0
3 0 0 0 0 0 0 0 0 0
```

More interesting is the question of what you can do with a .png file. The author recommends - in your interest - to always request one-dimensional heightmaps in grayscale if possible - if you add color information, you make life harder for yourself.

Color geometry bitmaps are not completely useless!

The presence of full-color geometry bitmaps is not a "decadence" of the designer creating them. In practice, especially in the field of advanced shader programming, there is a whole group of methods that use individual color components to influence various aspects of the resulting model geometry.

However, for OpenSCAD developers, it is sufficient to use the comparison of a colored png and the resulting heightmap shown in Figure 8-13 and 8-14.

It was provided by the OpenSCAD development team and is generally "illustrative".



Figure 8-13. The different colored elements of the PNG...
(Picture source: OpenSCAD / Michael Frey)

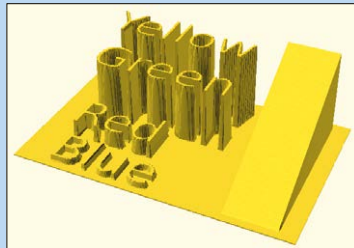


Figure 8-14. ...lead to different geometry heights (Source: OpenSCAD / Michael Frey)

By the way, the actual calculation of the respective values is done by the formula ($Y = 0.2126R + 0.7152G + 0.0722B$), which should appear familiar from other image processing tasks.

At this point, a first test with a 200×200 pixel test image is recommended, which contains the test pattern shown in Figure 8-15.

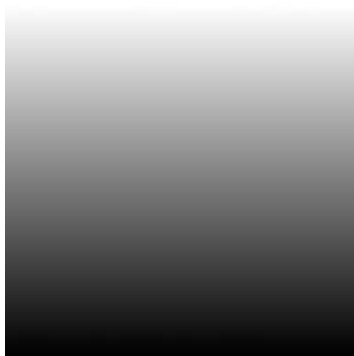


Figure 8-15. The gradient tool included in GIMP makes it easy to create such objects.

Loading the file then follows the well-known scheme – first, the author tries the following command:

```
surface(file = "/media/tamhan/781A-F80B/OpenSCADBook/CH8-DE/gradient200.
png", center = true);
```

The reward for the effort is the geometry shown in figure 8-16, which appears somewhat pixelated due to the comparatively low resolution of the original.

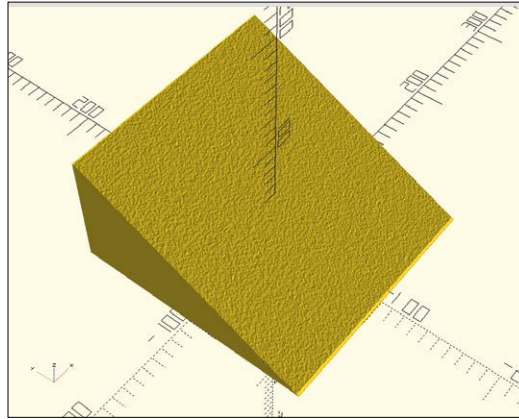


Figure 8-16. The generated heightmap works well.

The most interesting thing is a look at the coordinate crosses - we can see that "one pixel" of the delivered map corresponds to a 1x1 large voxel in the three-dimensional output.

Note that you can adjust this ratio to your needs by using the scale command - if you want to halve it, for example, the following would work:

```
scale([0.5, 0.5, 0.5])
  surface(file = "/media/tamhan/781A-F80B/OpenSCADBook/CH8-DE/gradient200.
png",
        center = true,
        invert = true);
```

If you consult the documentation of the command, you will also see another parameter called `invert`. It can be activated according to the following scheme in our small example program:

```
surface(file = "/media/tamhan/781A-F80B/OpenSCADBook/CH8-DE/gradient200.
png", center = true, invert = true);
```

Its rendering would now lead to the output shown in Figure 8-17.

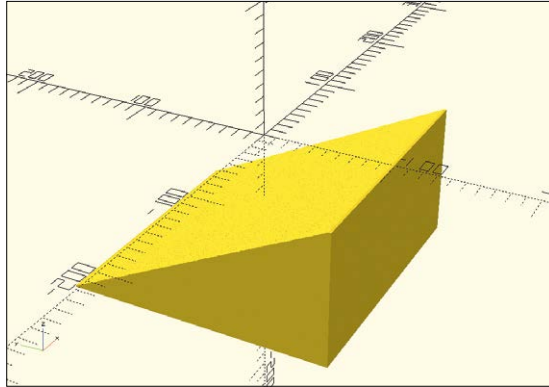


Figure 8-17. Setting invert changes the output.

The next problem is that we do not know how to convert the black and white values into Z-coordinates. To answer this question, the author creates another element, which now - as shown in Figure 8-18 - brings along two gradients. The reward for this change is that we can now see that the "center" of the object is black, while the two edges are white.

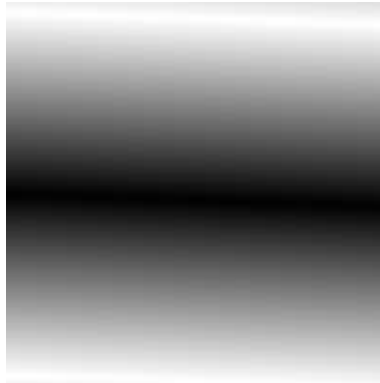


Figure 8-18. A new bitmap makes the situation clearer.

Experimenting with input data is reasonable!

The author's practical experience teaches that many problems - both electronic and otherwise - can be "played to death" by feeding the procedures with constructed input. This is a procedure that should be memorised even when you are not working with OpenSCAD.

In the next step, we have to adapt the file name passed to surface, and vary the settings for invert - rendering yields the results shown in figure 8-19:

```
surface(file = "/media/tamhan/781A-F80B/OpenSCADBook/CH8-DE/gradient200a.
png",
        center = true,
        invert = true);
```

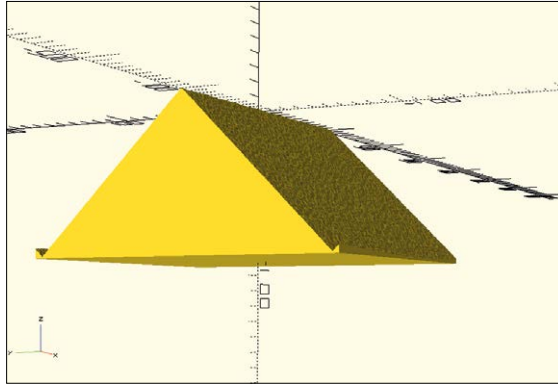


Figure 8-19. If invert is true, black is found at zero and white at -100.

If we set the value to False instead, we see the result shown in figure 8-20.

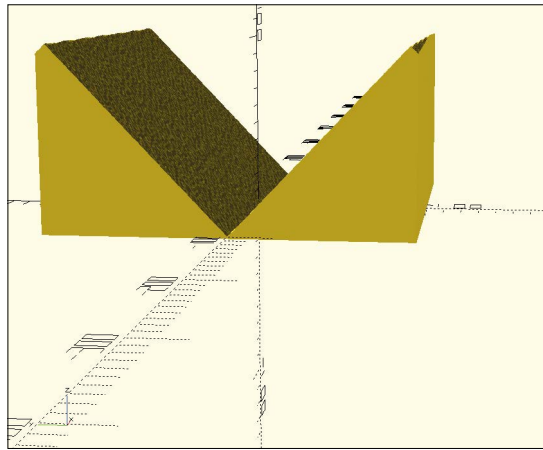


Figure 8-20. When invert is deactivated, black is zero, while white corresponds to +100.

8.6 Conclusion

At this point, we are finished with our little excursion into two-dimensionality. In the following chapter, I will introduce you to operations that significantly enhance the appearance of three-dimensional objects. This is followed by experiments with ready-made part libraries and a brief overview of tools that exist in the OpenSCAD ecosystem.

Chapter 9 • Advanced 3D objects

After our little excursion into the possibilities of 2D-3D-combination, I would like to introduce you to advanced methods for working with purely three-dimensional objects. Apart from the polygon function, which behaves in the three-dimensional area - by and large - like its colleague from the 2D arena, we also want to take a look at combination operations.

This is probably the (most complicated) part of OpenSCAD. However, it's worthwhile dealing with it because the appearance of the resulting objects is improved immensely. For motivation, the figures 9-1 and 9-2 are shown opposite each other - the left one had to do without a Minkowski operator, while the right one is the "final" version of the soap dispenser hanging in the author's bunker.

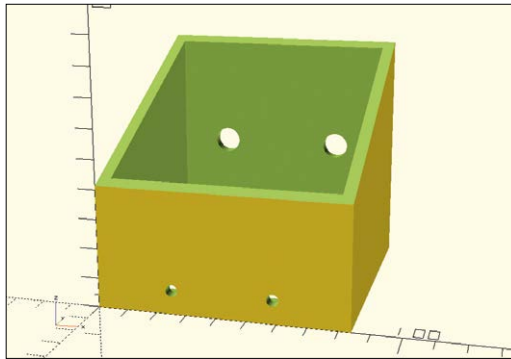


Figure 9-1. Without using the Minkowski operator, edges look "hard".

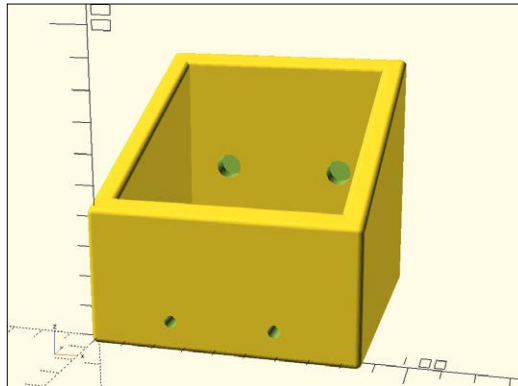


Figure 9-2. The application of a Minkowski combination provides a smoother appearance.

9.1 Create three-dimensional polygons

The previous chapters introduced methods to create - more or less - complex three-dimensional objects "directly", i.e. by using geometric primitives.

This approach works well in practice until we need a complex object. An almost ideal candidate is the - you guessed it - soap dispenser, which accompanies us throughout the book like a little tormentor.

To assemble the soap dispenser we would theoretically need a group of rectangles and triangles. Alternatively, however, we could imagine the part as a rectangle from which an object consisting of a total of six points is subtracted - this (admittedly somewhat innovative) procedure is shown schematically in figure 9-3.

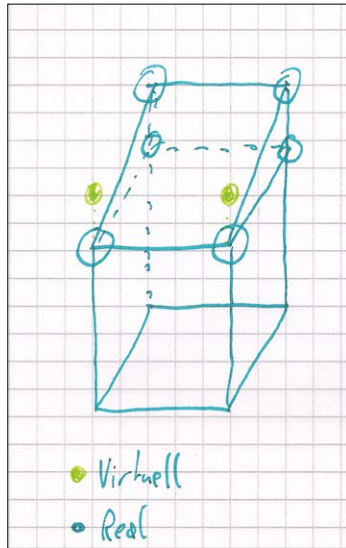


Figure 9-3. You can have a lot of fun with only six points.

The point cloud can be represented in an array - just like our two-dimensional polygons. As the author has taken the precaution of designing this soap holder as a parameterized object, the structure presents itself as follows:

```
CubePoints = [
  [ -3, -3, boxminheight ], //0
  [ boxmax, -3, boxminheight ], //1
  [ boxmax, boxmax, boxmaxheight +3], //2
  [ -3, boxmax, boxmaxheight +3], //3
  [ -3, -3, boxmaxheight +3], //4
  [ boxmax, -3, boxmaxheight +3], //5
  [ boxmax, boxmax, boxmaxheight +3], //6
  [ -3, boxmax, boxmaxheight +3]]; //7
```

The fact that a total of eight points are used here is due to the convenience of the author - two of them are duplicated. This is because the author derived the paths from a "generic" rectangle - the two points shown brighter in the graphic were "merged" with their colleagues further down.

The primary task of the variables is to define the dimensions - the structure looks like this:

```
$fn=64;
walldepth=4;
minkDia=2;
boxwidth=72 +2*minkDia;
boxminheight=40;
boxmaxheight=80;
boxmax = boxwidth + 2* walldepth+8;
boxf = boxwidth + 2* walldepth;
```

Writing the array indices in comments "behind" the respective elements is a small quirk of the author, but it helps to construct the fields required in the next step.

If you think back carefully, you know that we described our polygons using paths. When working with three-dimensional objects, this generally works in the same way - the only difference is that we now describe either triangular or square surfaces. It is important to note that a polygon - according to its definition - does not necessarily have to be rectangular, but can also look like figure 9-4.

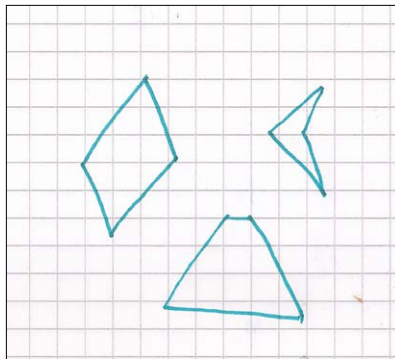


Figure 9-4. Although the angles are not rectangular, these objects are quadrangles.

We can thus create an array that defines the squares according to the following structure:

```
CubeFaces = [
  [0,1,2,3], // bottom
  [4,5,1,0], // front
  [7,6,5,4], // top
  [5,6,2,1], // right
  [6,7,3,2], // back
  [7,4,0,3]] // left
```

It should be noted that OpenSCAD also supports triangular arrays as used in various hardware libraries - in this case, the information supplied in CubeFaces is trivalent. Be that as it may, we can obtain the rendering shown in Figure 9-5 at this point by using the following command:

```
polyhedron( CubePoints, CubeFaces );
```

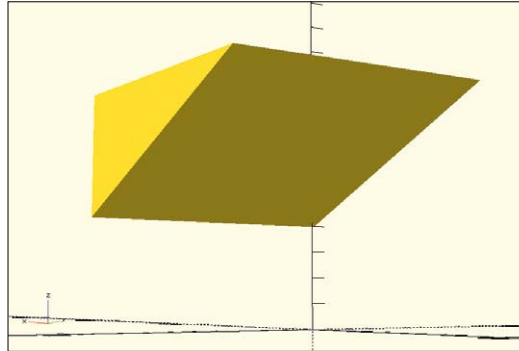


Figure 9-5. The triangular part of the geometry is ready for use.

9.2 Excursus: Soap Dispenser, Part 1

If you didn't grow up programming shaders in your spare time, you are well-advised to keep the polygon structures as simple as possible. Theoretically, the element we have here could have been created "directly" if the points were laid out as shown in figure 9-6.

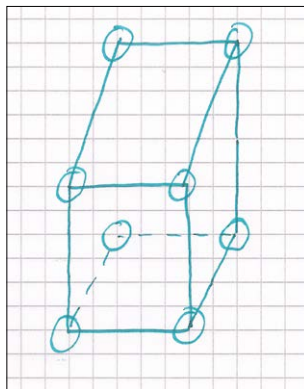


Figure 9-6. This procedure works just as well.

Yours truly, however, selected a slightly different approach for his soap dispenser – we will look at the exact steps of its genesis tour a tour. The first thing we did was to create a "full-surface" container, which was created by the following code and provoked the rendering shown in Figure 9-7.

```

difference()
{
    cube([boxwidth + 2*walldepth,boxwidth + 2*walldepth,boxmaxheight]);

    translate([walldepth, walldepth, walldepth]){
        cube([boxwidth ,boxwidth,boxmaxheight+5]);
    }
}

```

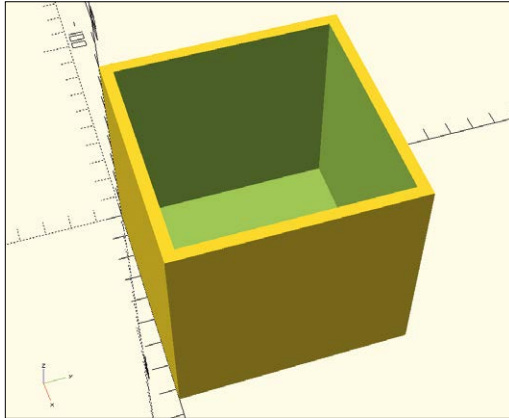


Figure 9-7. The positive mould of the holder is ready for use.

Now we are ready for the next task - it is time to create the parts of the model "to be subtracted". For this purpose, the following snippet is used to create the element shown in figure eight:

```

{
    polyhedron( CubePoints, CubeFaces );
    translate([3*walldepth, 3*walldepth, -walldepth]){
        cube([boxwidth-4*walldepth,
            boxwidth-4*walldepth,boxmaxheight+5]);
    }
}

```

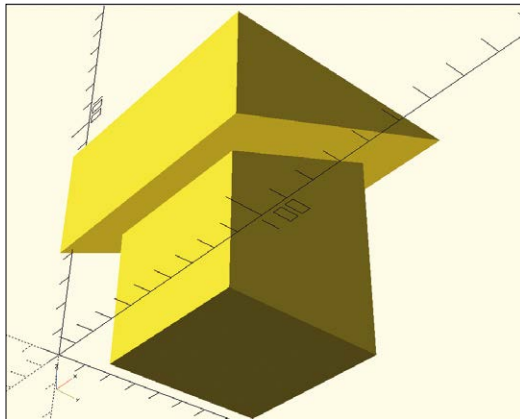


Figure 9-8. Subtracting this part results in a basic holder shape.

Next comes what we would expect - the introduction of a difference operator leads to the result shown in figure 9-9:

```

difference()
{
    difference()
    {
        cube([boxwidth + 2*walldepth,boxwidth + 2*walldepth,boxmaxheight]);

        translate([walldepth, walldepth, walldepth]){
            cube([boxwidth ,boxwidth,boxmaxheight+5]);
        }
    }

    {
        polyhedron( CubePoints, CubeFaces );
        translate([3*walldepth, 3*walldepth, -walldepth]){
            cube([boxwidth-4*walldepth
,boxwidth-4*walldepth,boxmaxheight+5]);
        }
    }
}

```

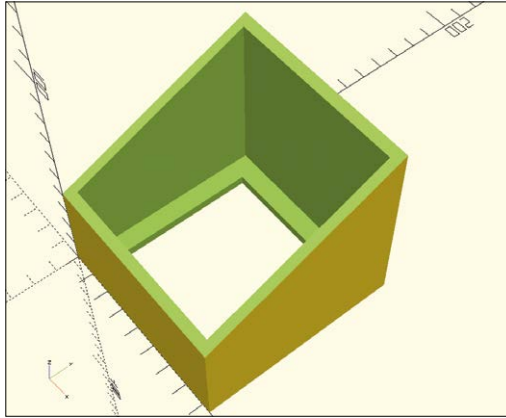


Figure 9-9. The soap dispenser begins to take shape.

To complete the soap dispenser, we need to place drill holes at this point and then "round it off" - a task we will address in the following section. But before we do this, we have to clarify another question.

9.3 Polygon alignment in three-dimensional space

One of the most interesting experiments with our two-dimensional polygons was the "reversal" of the sense of rotation of the paths - this behavior was illustrated in Figure 6-31, which shows a polygon being spawned from "two different directions".

In the world of two-dimensional modeling, we found that it makes no difference from which direction the path was constructed. The question is whether this also applies to three-dimensional objects.

To test how OpenSCAD behaves in 3D mode, we adapt the first square by "turning around" the order of the individual elements:

```
CubeFaces = [
    [3,2,1,0], // bottom
    [4,5,1,0], // front
    [7,6,5,4], // top
    [5,6,2,1], // right
    [6,7,3,2], // back
    [7,4,0,3]]; // left
```

If you run the program again in the present state, you will not see any difference at first - the polyhederon is still produced the way we would expect.

To illustrate potential problems, the first step is to activate the menu option shown in figure ten and then command another preview rendering. Note that the "Thrown Together" option regularly disables itself by default and that "normal" rendering will not display the problem.

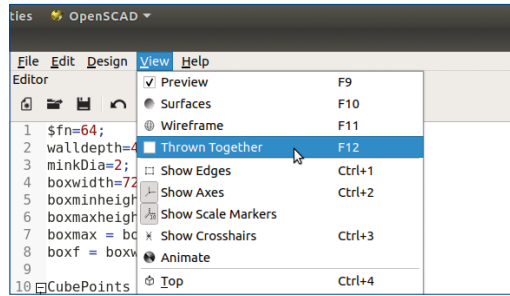


Figure 9-10. The option "Thrown Together" helps to find errors in polygons.

In any case, the reward for the effort is the screen shown in figure eleven. Interesting here is that the bottom side now - this is a bit hard to see in print - appears pink, which indicates a wrong direction of rotation of the polygon.

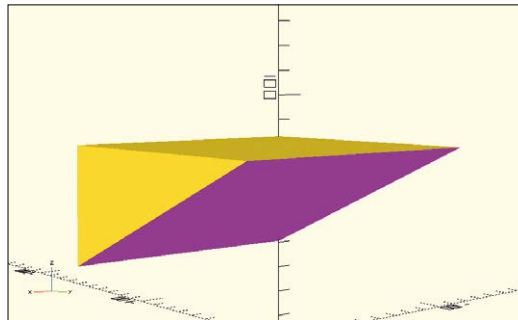


Figure 9-11. This polygon runs "the wrong way round"

OpenSCAD assumes that polygons have a clockwise direction of rotation when viewed "from the front" - this can be most easily illustrated by the hand-drawn sketch shown in Figure 9-12. If you break this rule, nothing happens most of the time - but in some cases, especially when exporting to STL format, strange problems may occur.

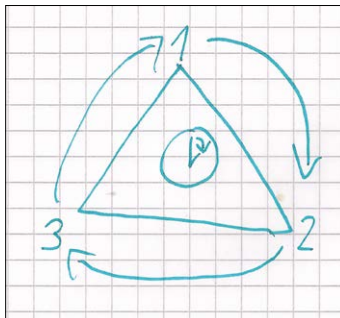


Figure 9-12. The clock finds the correct direction of rotation.

Although the author has never had this problem in his practical experience, we would like to refer you to the documentation available at https://en.wikibooks.org/wiki/OpenSCAD_

[User_Manual/The_OpenSCAD_Language#3D_to_2D_Projection](#) as a reference. Further information on the problem at hand can then be found in the section Debugging polyhedra.

9.4 Combination with Minkowski

Classical 3D modeling programs have a "nice life" in that they map edges and similar boundary structures in software. They can, therefore, be rounded off or otherwise processed with little effort.

In the world of OpenSCAD, the situation is more complicated in that the edges are created during the processing of the program and are therefore not, or at least not readily, addressable.

In practice, one can still find "perfectly" rounded three-dimensional objects. This is due - among other things - to an operation known as Minkowski, which is "mathematically" difficult to understand at first glance, but soon becomes second nature in practice.

In the first steps the author will try at basic mathematical explanations - in your interest, we ask you to skim over these. After that, you will find a group of examples that illustrate the exact procedure.

Background information.

If you want to learn more about the Minkowski operation, find information in the documentation of the OpenSCAD rendering library available at https://doc.cgal.org/latest/Minkowski_sum_3/index.html and https://doc.cgal.org/latest/Minkowski_sum_2/index.html.

The operation is also widely used in the field of "digital art" - at <http://www.math-art.eu/Documents/pdfs/Cagliari2013/Velichova%20revisee7.pdf> you will find a paper which is particularly worth reading.

If you are looking for a "structured" introduction, the following two PowerPoint slide decks are recommended:

https://resources.mpi-inf.mpg.de/departments/d1/teaching/ss10/Seminar_CGGC/Slides/07_Bock_MS.pdf

<http://acg.cs.tau.ac.il/courses/algorithmic-robotics/spring-2011/slides/ms.pdf>

The Minkowski Sum of a polygon is the "collection" of the sum of the two points - mathematically this can be described by the formula shown in figure 9-13.

$$A + B := \{a + b \mid a \in A, b \in B\}$$

Figure 9-13. Set theory helps to understand the Minkowski operation.
(Picture: Wikipedia DE)

We want to keep the task simple in the following steps in that we want to limit ourselves to two-dimensional space and work with rectangular objects.

For the time being, the following element structure should be connected:

```
translate([10, 10])
  square(4, 4, center=true);

square(1,1, center=true);
```

Whoever commands a rendering at this point will see the structure shown in Figure fourteen in 2D mode. Pay particular attention to the coordinates of the elements at the edge to obtain information or conclusions about the actual operation.

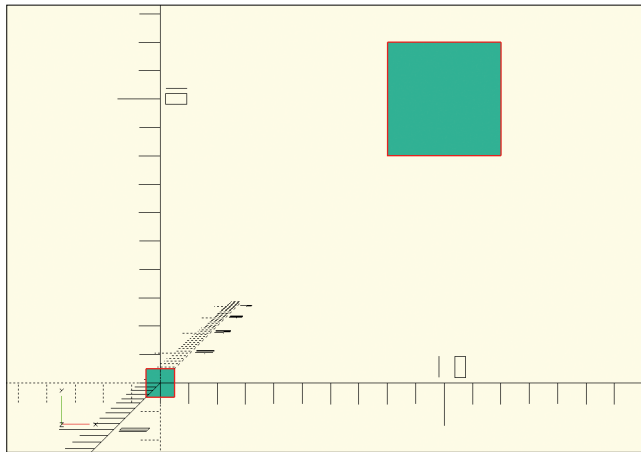


Figure 9-14. Minkowski preparation, two-dimensional.

In the next step, we command a "further" rendering which connects the two elements via the Minkowski operation. Its results are shown in figure 9-15:

```
minkowski(){
  translate([10, 10])
    square(4, 4, center=true);

  square(1,1, center=true);
}
```

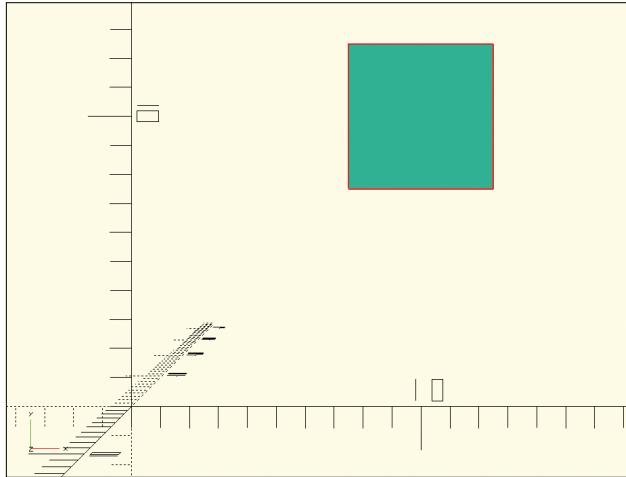


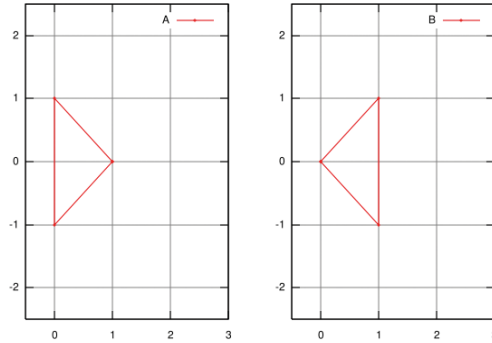
Figure 9-15. The Minkowski operation changed the size of the element.

For further understanding of the operation, we must specify which sum, when applied to any element, returns the element unchanged. In the case of the Minkowski, this "unary element" is an empty sum containing no geometry.

With this knowledge, we can turn to the operation in the background. In principle, the Minkowski Sum is created by combining every element of the "first" polygon with "every" element of the second. On the German Wikipedia, this procedure is defined by the two triangles shown in figure 9-16, which result in the Minkowski Sum shown.

Gegeben A und B mit Elementen aus $\mathbb{R}^2$:

$$A = \{(1, 0), (0, 1), (0, -1)\}, B = \{(0, 0), (1, 1), (1, -1)\}$$



Dann ist die Minkowski-Summe von A und B:

$$A + B = \{(1, 0), (2, 1), (2, -1), (0, 1), (1, 2), (1, 0), (0, -1), (1, 0), (1, -2)\}$$

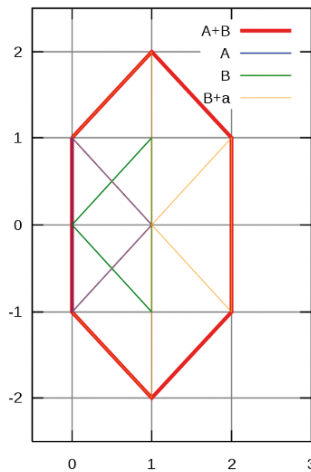


Figure 9-16. This diagram shows the assembly of the point constellation (Source: Wikipedia DE, Stephan Beyer, et al).

In the case of our squares, we would have found a total of "16" points, which, however, simply result in a "larger" square.

A nice alternative way to imagine the procedure is to capture the "second" quadrilateral and then drag it with its origin (i.e. zero over zero) along the edge of the second object.

This can be verified by looking at Figure 9-15, where we see that the larger of the two squares has grown by "0.5" in each axis direction. The 0.5 growth itself can then be justified as shown in Figure 9-17.

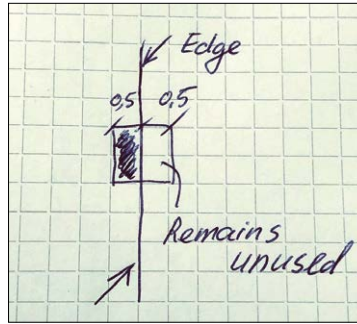


Figure 9-17. Since the inner rectangle moves along the edge with its center of mass, it can only add 0.5.

Be that as it may, we can make a "practical" attempt in the next step:

```
$fn=64;
minkowski(){
  cube(4);
  sphere(r=1);
}
```

For this, we use the following snippet, which creates the Minkowski Sum of a circle with the radius - explicitly radius, not diameter - 1 and a cuboid with a side length of four. The reward of the effort is a "round combinatorial object" which is shown in figures 9-18 and 9-19.

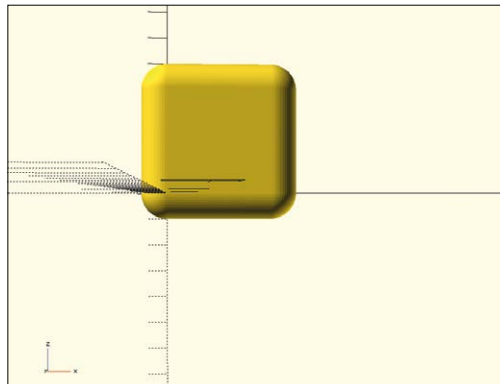


Figure 9-18. The cube looks rounded both from the front...

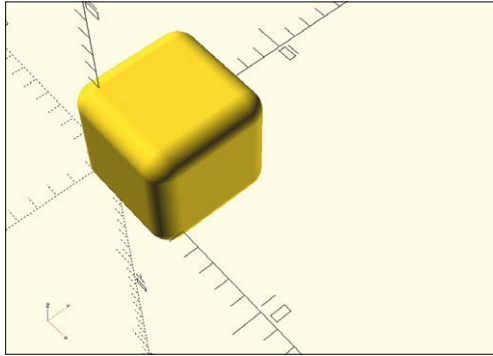


Figure 9-19. ...and from the side.

This illustrates the most frequent purpose of the Minkowski operation – rounding off corners. Another question concerns what happens if you move the sphere away from the origin like in the following snippet:

```
$fn=64;

minkowski(){
  cube(4);
  translate([2,2,2])
  sphere(r=1);
}
```

A further rendering run informs us that the position of the cuboids has shifted. In practice, it is recommended to always place the "center of mass" of the second object in the middle of the coordinate system.

9.5 Effect of the Minkowski operator on boreholes

By far the most common application is the "rounding" of edges of objects. This works well as long as an object consists only of the edges that we want to round off. Next, let's look at the following object, which has an inner channel as well as outer edges:

```
$fn=64;

difference(){
  cube(6,6);
  translate([3,3,-3])
  cylinder(20,2);
}
```

As an experienced OpenSCAD user, this should not come as a surprise - Figure 9-20 shows what we can expect.

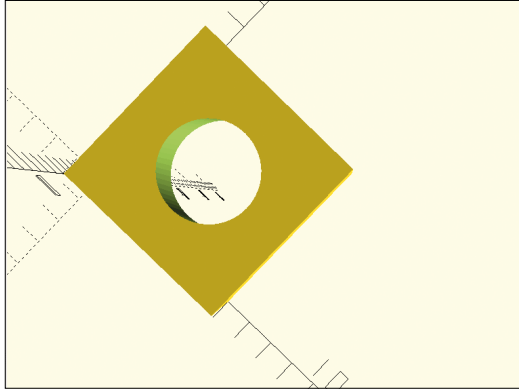


Figure 9-20. Pearl necklace, ante portas.

Next, we use the Minkowski operator to command a rounding:

```
$fn=64;

minkowski(){
difference(){
  cube(6,6);
  translate([3,3,-3])
    cylinder(20,2);
}
sphere(r=1);
}
```

A look at the rendering result shown in Figure 9-21 informs us that the channel has become much smaller.

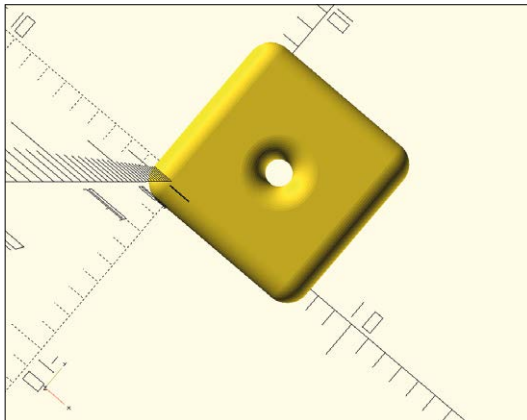


Figure 9-21. The Minkowski operation greatly reduced the dimensions of the channel.

The extremely long rendering run is to be expected - keep in mind that a sum increases by a factor of 64 times 64. We are dealing with two circles - the Minkowski operator is a quite "expensive" calculation.

There are several possibilities to circumvent the problems associated with the Minkowski operation. First of all, it is possible to place boreholes and other things only after the application of the operator - in this case, the calculation becomes much easier, because the circle does not have to "drive" along the entire "borehole". Also, the Minkowski operation cannot change the borehole diameter in this case, because the borehole has not yet become part of the geometry at the time of the application of the Minkowski Transformation.

Workaround number two is to perform some kind of "intelligent placement" of round objects using for loops or similar programmatic constructs.

With a little skill, a "round" edge can be constructed from spherical parts, cylinders, etc., without Minkowski. Funnily enough, there is a whole group of libraries for this purpose, which can easily be found in Google.

9.6 Excursus: Soap dispenser, part 2

After this little digression into the role of the Minkowski operator, we want to return to our soap dispenser, who is still waiting for his finishing touches. First of all, it is advisable to pack the above structure into a module - if only because the overall program is more "pleasant" to handle that way.

In the interest of compactness, we show but a small part - the author places the two elements CubePoints and CubePaths within the module but leaves out the other variables:

```
boxf = boxwidth + 2* walldepth;
superstructure();
module superstructure(){
    CubePoints = [
```

Next, we take care of the introduction of a sphere that "rounds off" the geometry returned by superstructure:

```
minkowski()
{
    superstructure();
    sphere(r=1);
}
```

At this point, you can command another rendering run, which will take some time and finally return the result shown in Figure 9-22.

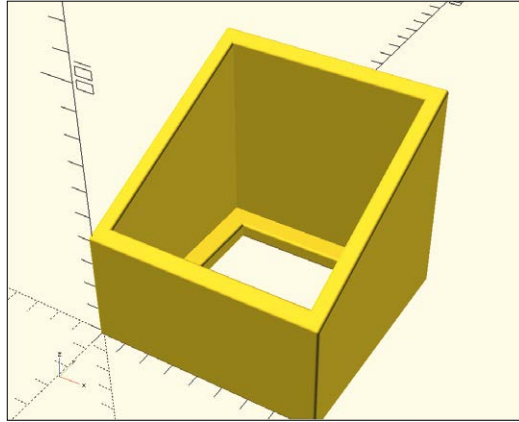


Figure 9-22. The "rounding off" of the geometry worked without problems.

Rescuing artifacts.

From a certain level of complexity, the "rendering process" of an element can take a long time and in some cases even fail, forcing you to quit the entire OpenSCAD application. Any information that may be in the editor is not lost in this case. While the rendering command is being processed in the OpenSCAD console, you will find messages structured according to the scheme "Saved backup file: /home/tamhan/.local/share/OpenSCAD/backups/unsaved-backup-MTJ10916.scad", which informs you of the location of the temporary files. Simply harvest them before you restart the OpenSCAD application.

Logic dictates what is necessary at this point - the following code generates the "negative form", whose rendering is shown in Figure 23:

```
union(){
  translate([boxf*0.25,boxwidth+walldepth*2.2, boxminheight]){
    rotate([90,0,0]){
      cylinder(h=walldepth*3, r=4.5);
    }
  }

  translate([boxf*0.75,boxwidth + walldepth*2.2, boxminheight]){
    rotate([90,0,0]){
      cylinder(h=walldepth*3, r=4.5);
    }
  }
}

union(){
  translate([boxf*0.3,walldepth+10, boxminheight / 5]){
    rotate([90,0,0]){
      cylinder(h=walldepth*8, r=2);
    }
  }
}
```

```

    }
  }
  translate([boxf*0.70,walldepth+10, boxminheight / 5]){
    rotate([90,0,0]){
      cylinder(h=walldepth*8, r=2);
    }
  }
}

```

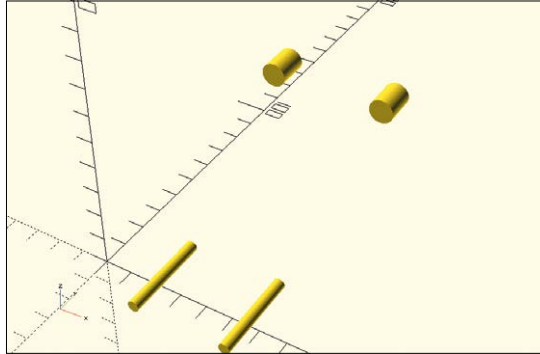


Figure 9-23. The negative form provides the necessary drill holes.

To complete the holder, you have to pack the negative form into another module and subtract it from the already existing and rounded element using the difference operator. The reward for your efforts is the finished object, which we have shown you in the introduction.

9.7 The Hull operator

Anyone looking for operators for connecting geometries in the OpenSCAD documentation will find, in addition to the Minkowski operator just discussed, a simpler candidate known as Hull operator. Anyone who fears another mathematical tour de force at this point can sit back and take a deep breath - this version is much easier to handle.

The Hull operator realises - in two-dimensional as well as in three-dimensional space - a kind of "solid" that encompasses all points that are affected by it.

It follows from the logic that this feature can also be misused to create round cuboids. For this, we need a structure consisting of a total of eight spheres in space:

```

translate([-10,-10,-10])
  sphere(r=2);
translate([10,-10,-10])
  sphere(r=2);
translate([-10,10,-10])
  sphere(r=2);
translate([10,10,-10])
  sphere(r=2);

```

```

translate([-10,-10,10])
  sphere(r=2);
translate([10,-10,10])
  sphere(r=2);
translate([-10,10,10])
  sphere(r=2);
translate([10,10,10])
  sphere(r=2);

```

If you execute a rendering command against the object at hand, you will get the spherical framework structure shown in Figure 9-24.

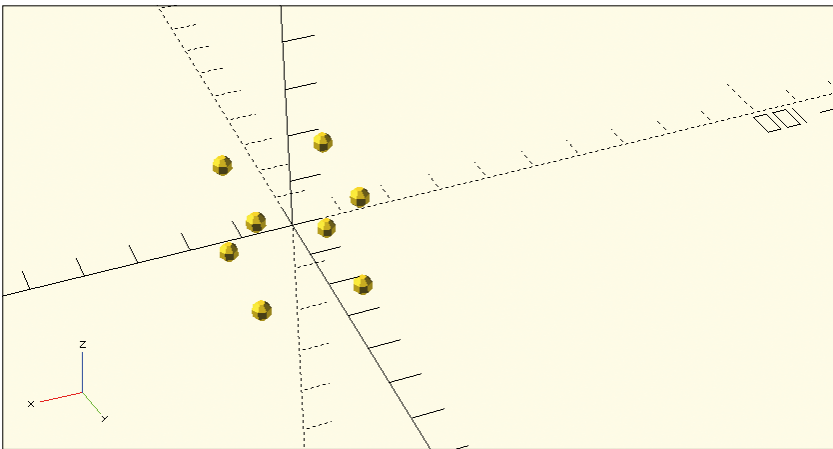


Figure 9-24. The framework of our object is ready for use.

At this point we have to imagine that the duty officer waddles by with an old nylon stocking and uses it to "span" the whole object. This can be done by using the Hull operator, which is thrown around the entire enchillada:

```

hull(){
  translate([-10,-10,-10])
    sphere(r=2);
  translate([10,-10,-10])
    sphere(r=2);
  translate([-10,10,-10])
    sphere(r=2);
  translate([10,10,-10])
    sphere(r=2);

  translate([-10,-10,10])
    sphere(r=2);
  translate([10,-10,10])

```

```

    sphere(r=2);
    translate([-10,10,10])
    sphere(r=2);
    translate([10,10,10])
    sphere(r=2);

}

```

With the default value for `$fn`, the result shown in Figure 9-25 is obtained.

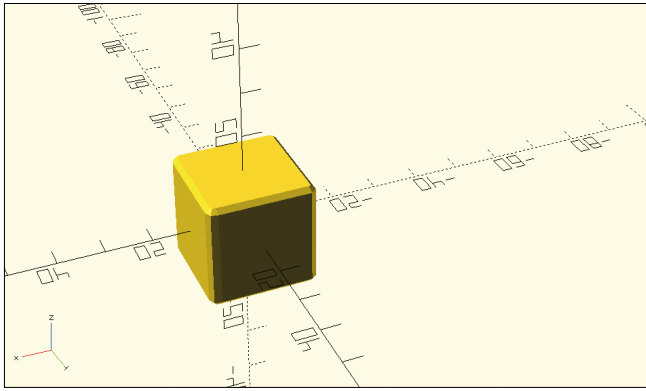


Figure 9-25. The box based on the Hull operator takes shape.

Next, we want to increase the value of `$fn` to 64 to generate a round object - it now presents itself as shown in Figure 9-26.

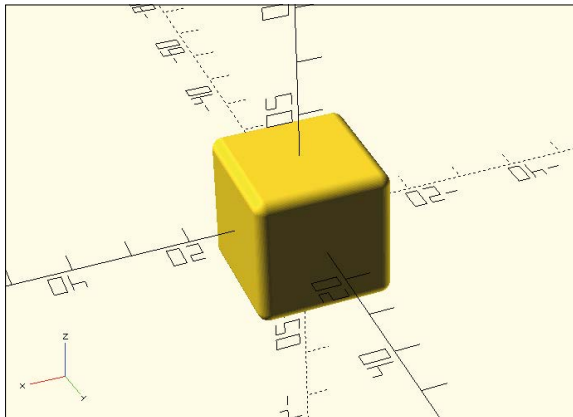


Figure 9-26. The Hull-based round cuboid also looks excellent.

9.8 Conclusion

At this point the author must congratulate you - we have, by and large, discussed all the important aspects of OpenSCAD. Unfortunately, we have not yet reached our goal - as in practically every other market, there is a whole group of libraries which fill "pains". Of particular importance is the product's standard library known as MCAD, which implements a "multitude" of frequently used geometric objects. It is the subject of our explanations in the following chapter.

Chapter 10 • MCAD - technical primitives for OpenSCAD

OpenSCAD has established itself in the field of "mechanical" design. Housings, screws, and similar things are often created with OpenSCAD, especially in smaller companies.

As noted in the conclusion of the last chapter, when working with housings and other geometric shapes, there are some structures that you encounter over and over again. Anyone familiar with the standard library known as MCAD can save a lot of time in this case.

10.1 Providing the library

The term standard library makes experienced developers assume that the library is available on every OpenSCAD installation. As is so often the case in the world of technology, however, an industry-standard is primarily a challenge for innovative people to create "as many" incompatible implementations as possible that meet the requirements set on paper. It follows that not every OpenSCAD installation is equipped with MCAD.

To determine whether your terminal comes with it, compile the following program:

```
use <MCAD/involute_gears.scad>
test_gears();
```

After including the MCAD gear library, we activate the `involute_gears.scad` module, whose task is to generate a gear test pattern (see figure).



Figure 10-1. If the gears appear, the MCAD installation is OK.

If the library is "missing" on your computer, the menu item highlighted in Figure 10-2 will help. It opens the OpenSCAD library directory in the file browser of your workstation; you can place libraries there.

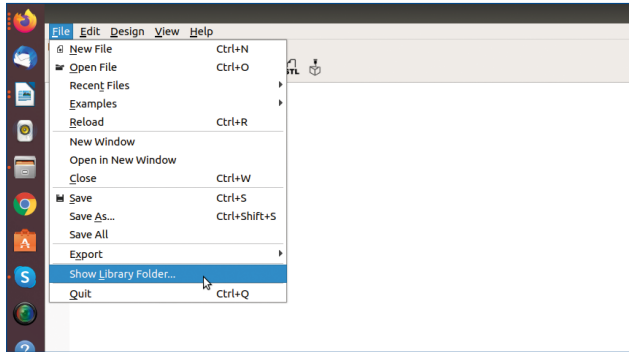


Figure 10-2. This option opens the file browser in the library directory.

A nice way to "download" MCAD is to use the GitHub repository available at <https://github.com/openscad/MCAD>, which you can download as a .zip file using the option shown in figure 10-3. Then extract the files contained in the archive into the directory to put our little example program to work.

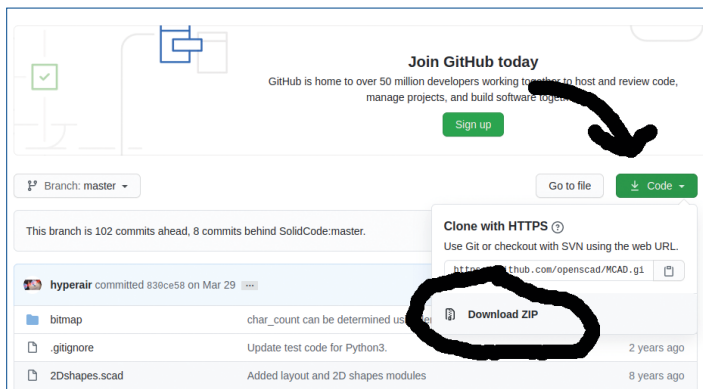


Figure 10-3. The use of the Git command-line client is not necessary here because the repository is not recursively structured.

Note that the absence of a library in the directory opened after clicking the menu item does not necessarily mean that it is missing. OpenSCAD has - see the documentation available at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Libraries - a whole group of further directories which also contain libraries which can be searched by the Use command.

10.2 Generation of gears, analysis of the library structure.

As with many other projects, the best way to get started with the MCAD library is to open the GitHub repository and view the individual files. They are generally extensively commented, and the file names are self-explanatory.

As a first small task, the author of these lines wants to create a gear system. There are two files available for this purpose - MCAD/gears.scad and MCAD/involute_gears.scad.

The two libraries differ mainly in the structure of the individual gear teeth. For tasks that are not mechanically demanding, you can pick any of the two. Logic, however, dictates that all elements belonging to a gear should be generated using the same algorithm.

Another critical point is that the two libraries contain many "identically" named elements and should therefore never be included together in a script. Those who do so nevertheless risk undefined behavior.

Be that as it may, we want to implement a small gearbox. The "simpler" library gears should be used, which leads to the following code:

```
$fn=64;  
use <MCAD/gears.scad>  
  
linear_extrude(height = 10, center = true, convexity = 10)  
  gear(number_of_teeth=51,circular_pitch=200);  
linear_extrude(height = 10, center = true, convexity = 10)  
  translate([0, 50])gear(number_of_teeth=17,circular_pitch=200);
```

The Gears library is idiosyncratic in that it generates its gears in the form of a two-dimensional object. This must then be converted into a three-dimensional form, for example by `linear_extrude`. It follows from logic that "twisting angles" let you create advanced gear-wheel topologies.

Most importantly, "matching" gears must have an identical value for `circular_pitch` in the case of the one before this library. Be that as it may, we can command a rendering at this point that leads to the result shown in figure 10-4.

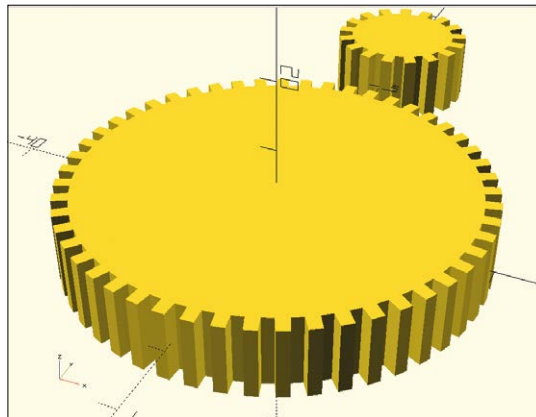


Figure 10-4. The gears are in place.

To get "productive" gears, you would have to place axle cutouts - a task that should not cause you any difficulties at this point.

10.3 Conjuring screws and nuts

Many years ago, a private militia in Eastern Europe used its expensive and highly qualified model carpentry workshop for the production of hexagon nuts. This created a laugh that is still enjoyed by those involved 20 years later, albeit now over a cigar.

Since the use of 3D printers to produce screws - benchmarks are explicitly excluded - is a similarly moronic task, the library is limited to providing cut-outs for hex nuts or a specific type of bolt (round head). This can also be confirmed by the functions created in the library, whose declaration looks like this:

```
module nutHole(size, units=MM, tolerance = +0.0001, proj = -1)
{
  module boltHole(size, units=MM, length, tolerance = +0.0001, proj = -1)
  {
```

A nice demonstration object can be created via the following command, which leads to the result shown in figure 10-5:

```
difference()
{
  translate([0,0,7.5])
    cube(30, center=true);
  boltHole(8, units="MM", length= 30);
}
```

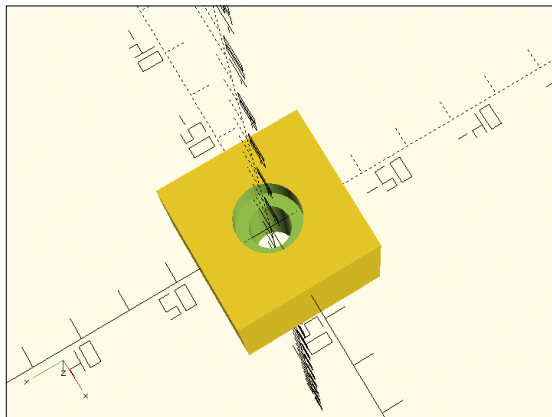


Figure 10-5. The hole for the round-headed bolt in dimension M8 is part of the cuboid.

If a hexagon nut is to be countersunk, the relevant code is as follows:

```
$fn=64;
use <MCAD/nuts_and_bolts.scad>
difference(){
  difference()
```

```
{
  translate([0,0,7.5])
    cube(30, center=true);
  boltHole(8, units="MM", length= 30);
}

translate([0,0,16.5])
  nutHole(8, units="MM", tolerance = +0.0001, proj = -1);
}
```

When using the `nuts_and_bolts` library, the most important thing is that you should name all parameters - this is especially true for the length.

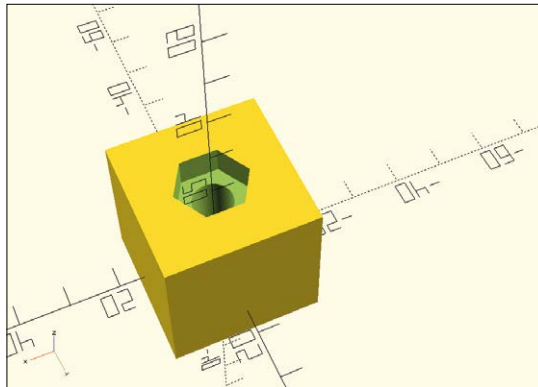


Figure 10-6. Sinking the M8 nut works without issues.

In the interest of humor, the author "realized" the test object created here, and then confronted it with locally purchased M8 parts. Figures 10-7 and 10-8 show the reward for the effort.

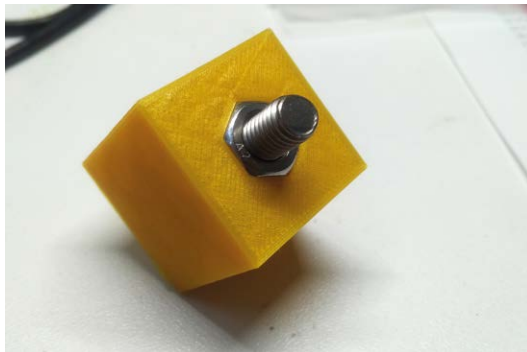


Figure 10-7. The hexagon nut fits perfectly into its recess...

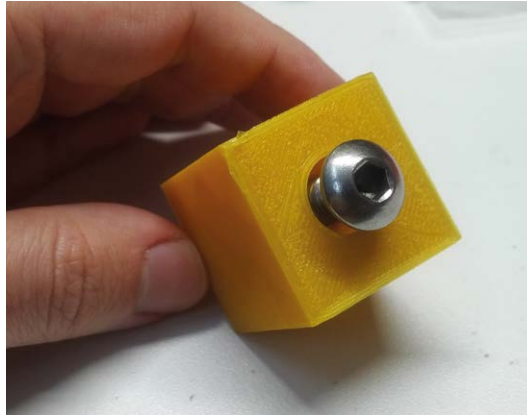


Figure 10-8. while the pan-head screw is slightly too large.

It is interesting to note that the round head bolt that was intended was not locally available because it is not very common and is expensive and thus not stocked by the local hardware store next to yours truly's office. By the way, the thread of the screw could not be "thrown" into the recess, but had to be screwed in and then withstood at least slight manual pressure.

10.4 Lego emulation

There is no question that LEGO has created a borderline ingenious system for the rapid construction of "small" systems with its bricks. But: there also is no question that using Lego-like footprints in other products is a great way to get into legal disputes.

A Cat!

Amateur lawyers can find a list of relevant legal disputes at https://en.wikipedia.org/wiki/Lego_clone. The crux of the matter is that LEGO usually loses lawsuits based on patents and footprints, while "more extensive cloning, especially of designs or figures" or using the LEGO brand name is usually won.

Be that as it may, the CAD library also offers the possibility to create Lego compatible elements. The easiest way to do this is to conjure up a "brick", which looks like the one shown in Figure 10-9 and is created by the following command

```
use <MCAD/lego_compatibility.scad>
block(1,2,5,reinforcement=false,hollow_knob=true);
```

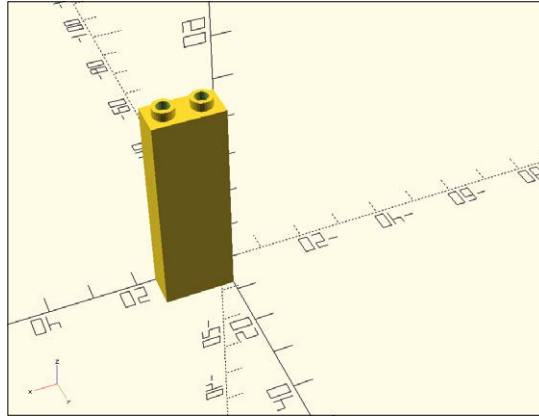


Figure 10-9. Similarities to children's toys are purely coincidental.

If you want a "simpler" layout instead, the following command is recommended:

```
use <MCAD/lego_compatibility.scad>
block(1,2,1/3,reinforcement=false,flat_top=true);
```

The reward for the effort is the "pronounced" lower end of the block shown in Figure 10-10.

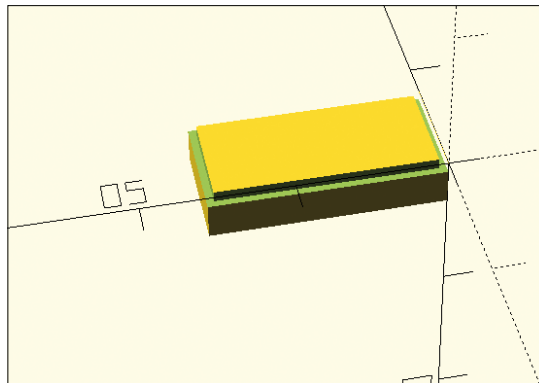


Figure 10-10. With sufficient printing accuracy, another Lego tile could be "stuck" on here.

10.5 Generate outlines of stepper motors

Anyone using a stepper motor in a non-military project would be well advised to follow the NEMA rules. They specify a group of "common" form factors - if the design fits a NEMA stepper motor, motors from other manufacturers will fit.

It should be noted that the NEMA standard is limited to the physical format only - within a format family, different motors can be found that have both different forces and different operating voltages. It is therefore not allowed under any circumstances to randomly use "any" NEMA motor element as a substitute for any other.

Be that as it may, the easiest way to demonstrate the stepper library is to call up the example program contained in it:

```
use <MCAD/stepper.scad>
nema_demo();
```

On the author's computer, it presented the stepper motor field shown in figure 11 – like most other parts of the MCAD library, the stepper motor module also comes with a demo program.

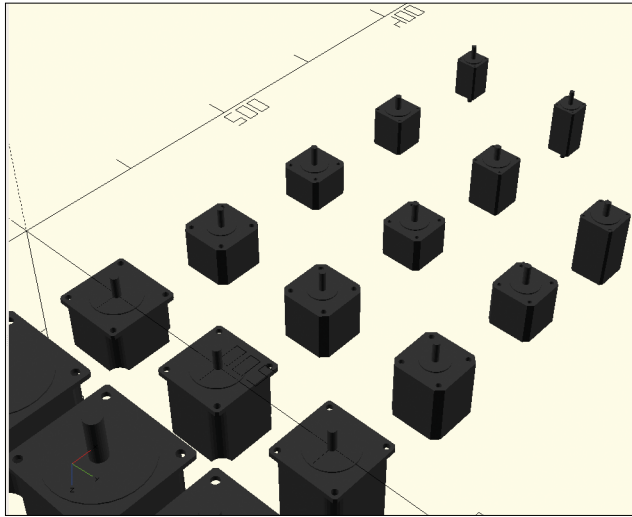


Figure 10-11. These stepper motor outlines can save time in an emergency.

Meanwhile, the structure of the software is self-explanatory. The field shown was created by the following code:

```
module nema_demo(){
  for (size = [NemaShort, NemaMedium, NemaLong]) {
    translate([-100,size*100,0]) motor(Nema34, size, dualAxis=true);
    translate([0,size*100,0])      motor(Nema23, size, dualAxis=true);
    translate([100,size*100,0])    motor(Nema17, size, dualAxis=true);
    translate([200,size*100,0])    motor(Nema14, size, dualAxis=true);
    translate([300,size*100,0])    motor(Nema11, size, dualAxis=true);
    translate([400,size*100,0])    motor(Nema08, size, dualAxis=true);
  }
}
```

10.6 Conclusion

Due to the immense range of functions of MCAD, it is illusory to give a complete overview of the various possibilities of the product in this textbook. The author hopes, however, that the experiments carried out here will arouse interest in continuing to deal with MCAD and other libraries on one's authority.

In the following final chapter we will free OpenSCAD from the "prison" of our workstation. A whole ecosystem of auxiliary products has developed around OpenSCAD, some of which I will briefly describe.

Chapter 11 • Value-added OpenSCAD

As a program deeply rooted in the Open Source movement, the OpenSCAD community has always been deeply committed to democratising access to technology. This is gratifying for us as there is a whole group of Value Added Services in the ecosystem. In this chapter, I would like to present some of them to you, and briefly touch on other offerings.

These products are not necessarily needed every day. But if you don't know one of the services when you need it, you can lose thousands of man-hours – so keep tagging along!

11.1 Model to measure

The option of creating variables in OpenSCAD has been talked all the way to P'Yong Yang. Things get funny the moment a non-programmer plops himself down in front of the terminal. In this case, offering a graphical configuration interface exposing settings without the possibility of damaging the rest of the file is the method of choice.

Thingiverse has established itself as a quasi-standard for sharing 3D models with friends and colleagues. For some time there is an extension that allows basic adjustments to be performed in an exposed STL file. In the first step, logically, a Thingiverse account is required, in which you then upload an OpenSCAD file.

To be used with caution.

The Thingiverse customiser is an extremely convenient tool. Note, however, that the web service has been discussed quite controversially in the past. In 2012, for example - see <https://hackaday.com/2012/09/20/makerbot-occupy-thingiverse-and-the-reality-of-selling-open-hardware/> - there were rumors that the parent company secured various rights to the models uploaded by the user.

A - short - review of the EULA available at <https://www.thingiverse.com/legal> did not give the author this impression, but he is a hobby lawyer at best. Also, there are repeated references, for example at <https://www.thingiverse.com/groups/thingiverse/forums/site-issues-and-feedback/topic:31378>, to the fact that Thingiverse is not open-hearted towards "conservatives".

This is not intended to be a "general" warning against the service. However, keep in mind that they sometimes lose all rights to uploaded items and do not use the service for politically sensitive content or products that compete with the company's parent company MakerBot. Furthermore, as with all cloud services, the fun can come to an abrupt end at any time...

If you open the documentation at <https://customizer.makerbot.com/docs>, you will see the following declaration. It describes how a variable visible to the customizer must be structured:

```
// variable description
variable name = default value; // possible values
```

The Thingiverse Customizer searches the header of the OpenSCAD file to grab variable declarations located there - note that this does not spread within the modules, but only analyzes the file headers of the uploaded file.

The individual variables that the user should be able to set via the user interface must follow a formal declaration - in addition to a descriptive comment, you must assign a default value. It is important to note that the default value can only be assigned by writing in a "discrete" value - if you place a calculation here according to the following scheme, you will create a variable that the customizer will not accept:

```
hole_radius = hole_diameter / 2;
height = 4 * 25.4;
function foo(x) = pow((1-x), 3);
```

The comment following the variable declaration informs the customizer of the type of control to be provided for "value input" and also determines which values the user may insert.

This procedure, which at first glance sounds a little abstract, becomes apparent the moment we visit <https://www.thingiverse.com/> and register for an account. In tests by the author, there were reproducible errors concerning the captcha when using the Firefox browser. Surprisingly, login with Google Chrome under Linux worked without problems.

After successfully logging in, click on the + in the menu on the upper right-hand corner to select the "Upload a thing" option in the context menu. The website will then display a window in which you can upload an OpenSCAD file. For now, we would like to work with the following code:

```
$fn=32;

//How big should it be
cube_size = 20; // [10:Small,20:Medium,30:Large]

//How wide should the diameter be
hole_diameter = 2.5;

difference()
{
    cube(cube_size, center=true);
    cylinder(d=hole_diameter, h=cube_size*2, center = true);
}
```

It is important to note that the recognition of the uploaded data type is based solely on the file names - so make sure to use a file name ending with .scad.

Test code faster

If you want to speed up the uploading-rendering-dance, visit the documentation available at <https://customizer.makerbot.com/docs> and scroll down until you reach the header Example. It contains a large text box where you may enter scripts - after clicking the blue button below you will see what the customizer would show a user when presented with the uploaded OpenSCAD snippet.

The code is discussed in the next step. First, save the file to a convenient location and upload it to the Thingiverse backend by dragging and dropping it or by clicking the My Computer option.

Note that the website "normally" accepts multiple SCAD files, but this feature is disabled, so all elements (including any modules) must be in one OpenSCAD file. The only exception to this rule is the following group of very frequently used libraries:

```
Build Plate use <utils/build_plate.scad>
MCAD use <MCAD/filename.scad>
Pin Connectors use <pins/pins.scad>
Write.scad use <write/Write.scad>
Includes the following fonts:
    write/Letters.dxf
    write/BlackRose.dxf
    write/orbitron.dxf
    write/knewave.dxf
    write/braille.dxf
The text() function supports all the fonts from Google as well as some
basic fonts such as Helvetica
3DVector use <utils/3dvector.scad>
hsvToRGB use <utils/hsvtorgb.scad>
```

Next, we scroll down to the Basic Information window. There, mark the checkbox "This is a customizer". Last but not least, it is recommended to check the checkbox shown in figure 11-1 - this informs the system that changes are in progress.

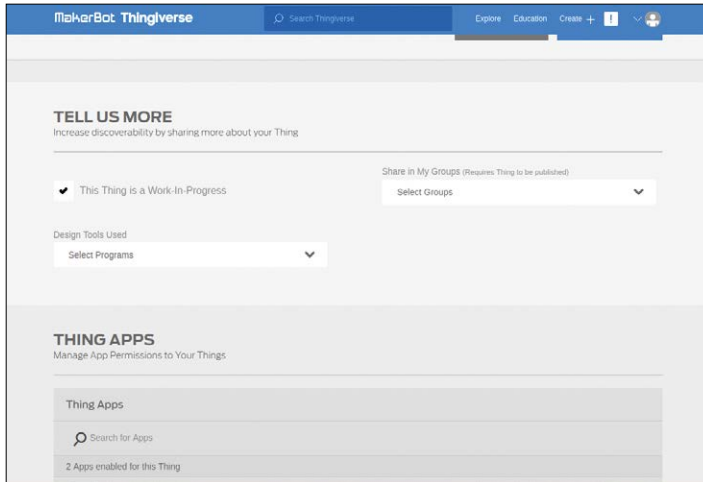


Figure 11-1. This option marks the element as Work in Progress.

In the next step, click on the button displayed in the upper right corner to find yourself in the editing page. There is a button Open in Customizer, which we click on next. The program reacts to this by opening a "new" browser tab, which is shown in figure 11-2.

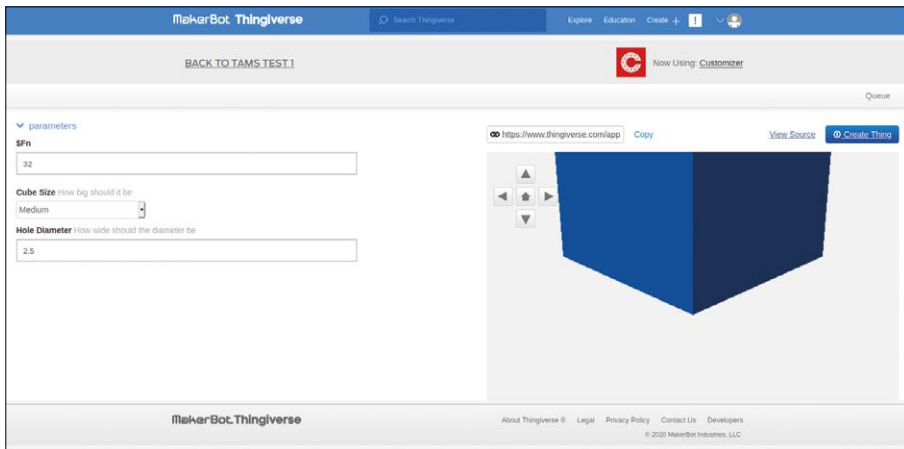


Figure 11-2. The Customizer has summarized our settings in the Parameters section.

What is most noticeable is that we have received an input box for all three options - including \$fn. Both hole_diameter and \$fn are text boxes in which the user may enter arbitrary values.

A click on the option hole_diameter is more interesting - the Thingiverse Customizer reacts to this by opening a combo box in which the three values defined in the file are available for selection. After adjusting the values, a new rendering run starts - it usually takes some time.

To check the results, you can click View Source to view the file contents. By clicking the Create Thing option, you can then command the creation of a finished STL file. This process may take a minute or two, depending on the Thingiverse server load and the complexity of the model.

Next, we want to return to editing the file. To do this, we close the customizer and click on Edit Thing. We then find ourselves in the editing window known from above, where we now upload a new file with identical names.

Note that uploading a file "multiple times" does not delete the elements already in the cloud - you have to do this, as shown in figure 11-3, using the delete icon, which the website displays when you mouse over it.



Figure 11-3. The option to delete an existing file is well hidden.

Next, we want to make sure that the user can no longer influence the \$fn option. The documentation contains the following procedure, but it does not work in practice:

```
//Hide this
// $fn=32;
```

Commenting out a variable as shown here firstly prevents rendering in the local SCAD parser. Secondly, in tests by the author, there were problems in which the values held in the variable could not be found even by the Thingiverse rendering system. A "more correct" approach, which almost always works at least on the local machine, looks like this:

```
/* [Hidden] */
$fn=32;

/* [Global] */
//How big should it be
cube_size = 20; // [10:Small,20:Medium,30:Large]

//How wide should the diameter be
hole_diameter = 2.5;

difference() . . .
```

The comments in square brackets have the task of implementing parameter tabs. These are control elements based on the behavior of the TabView control known from Windows and help the user to handle the model files by "grouping" the options offered.

Our example used here is limited to the Global and Hidden options. Global is a kind of meta-tab that is always displayed in the user interface of the Customizer. The Hidden section contains all those elements that the developer does not want the user to see under any circumstances.

In any case, using the preview renderer leads to the behavior shown in figure 11-4.

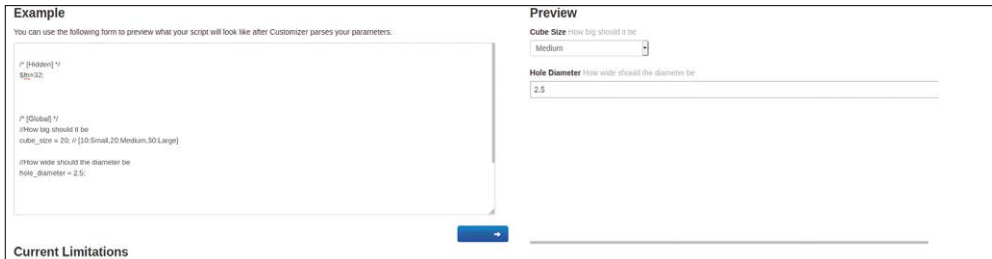


Figure 11-4. The variable `$FN` is no longer visible.

Not very stable!

In tests by the author, changes to the tab structure or the introduction of a tab comment caused the customizer to stop working for some time. In this case, use the preview functionality mentioned in the other box, or switch to the customiser integrated in OpenSCAD.

Be that as it may, we would like to introduce two more tabs at this point, each containing an option. For the sake of fun, we will also create a "global" element, which will result in the following structure:

```
/* [Global] */
tams_var=42;

/* [Size] */
//How big should it be
cube_size = 20; // [10:Small,20:Medium,30:Large]

/* [Hole settings] */
//How wide should the diameter be
hole_diameter = 2.5;

/* [Hidden] */
$fn=32;

difference() . . .
```

There is no rocket science here - the respective variables are listed in separate sections. Unfortunately, we have reached a point at which the customizer - at the time of going to press - is no longer working reliably.

Neither the preview renderer in the documentation nor the "actual" application evaluates the contents of the tabs. However, this is probably a "temporary" problem – especially as it does not occur when using the local customiser discussed next.

11.2 Controls of the Customizer

Be that as it may, we need to turn to the definition of the various available controls. The author introduces one type at a time and briefly illustrates it. Candidate number one is a normal text box - for this purpose, it is sufficient to simply place a documentation comment in front of the variable:

```
//How wide should the diameter be
hole_diameter = 2.5;
```

The problem with this procedure is that the user is not prevented from entering "incorrect" data. In the case of our holding object, for example, he could pick a 1-meter soap holder, which is not in the inventor's interest.

If we are dealing with purely numerical parameters, we can use the slider control shown in Figure 11-5 in use.



Figure 11-5. Sliders prevent the user from entering "idiotic" values.

In principle, sliders are created by placing square brackets behind them, which hold either a number or a group of values separated by a colon. Our small example was created by entering the following command:

```
//How wide should the diameter be
hole_diameter = 2.5; // [1:200]
```

Since developers have different requirements for the values delivered, the customizer offers several parameter sets. To simplify the display, here is a table that parameterizes the different operating modes.

Parameter syntax	Effect
[max]	Determines the maximum slider value. The minimum value is assumed to be zero.
[min:max]	Determines maximum and minimum values for the slider.
[min:step:max]	Determines maximum and minimum values for the slider. The step parameter determines the granularity the user can use for adjusting the value.

Candidate number three is the combo box used above, which limits the user to selecting from a group of values. Again, there are several parameter variants - the most important difference to the slider is that the individual values are separated by commas (see table).

Parameter syntax	Effect
[0, 1, 2, 3]	Entries in the combobox show the passed numbers. These numbers are also used as variable content in the program.
[foo, bar, baz]	Entries in the combobox show the passed strings. These strings are also used as variable content in the program.
[10:Small, 20:Medium, 30:Large]	Special regimen, where the combobox will show strings. The program, however, will be provided with the numerical values passed in as KV pairs.

Last but not least, the customiser offers the possibility to upload heightmaps and other information - a function that would go beyond the scope of this book.

11.3 Custom model, second

An old proverb claims that there are no rules that come without an exception. In the case of the Thingiverse Customiser, this exception is the use of a version of OpenSCAD greater than or equal to 2019.05 - in which case, as shown in Figure 11-6, there is an inherently disabled option in the menu. If you deactivate the element shown, OpenSCAD will display a small customizer window.

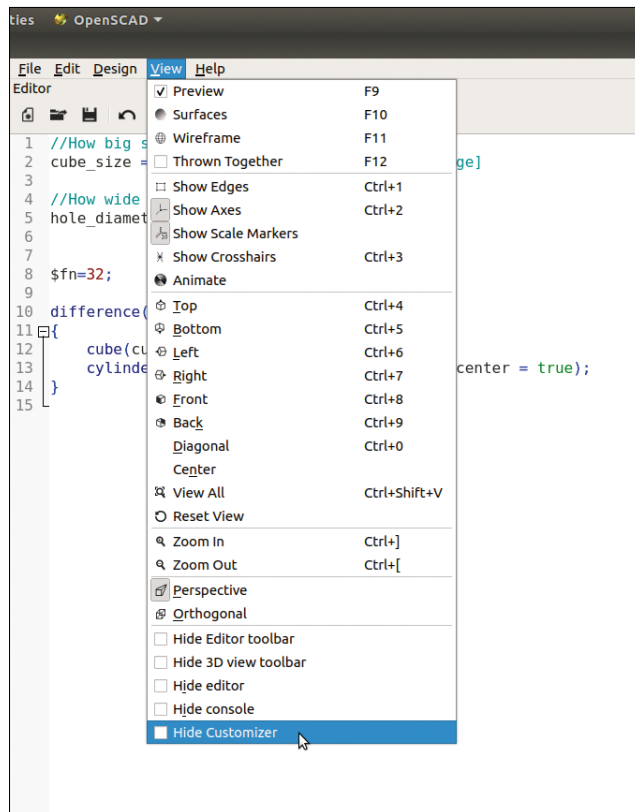


Figure 11-6. The Hide Customizer option must be disabled to activate the customizer.

More information!

<https://www.dr-lex.be/3d-printing/customizer.html> contains a detailed discussion of the differences between the version implemented in Thingiverse and the version implemented in OpenSCAD.

After deactivating the checkbox, OpenSCAD reacts by displaying an initially empty window. To "provide" the variables, you must first command a preview rendering by pressing F5: the parser generates the control element list only after this special invitation. In any case, the example program used above would generate the screen shown in figure 11-7.

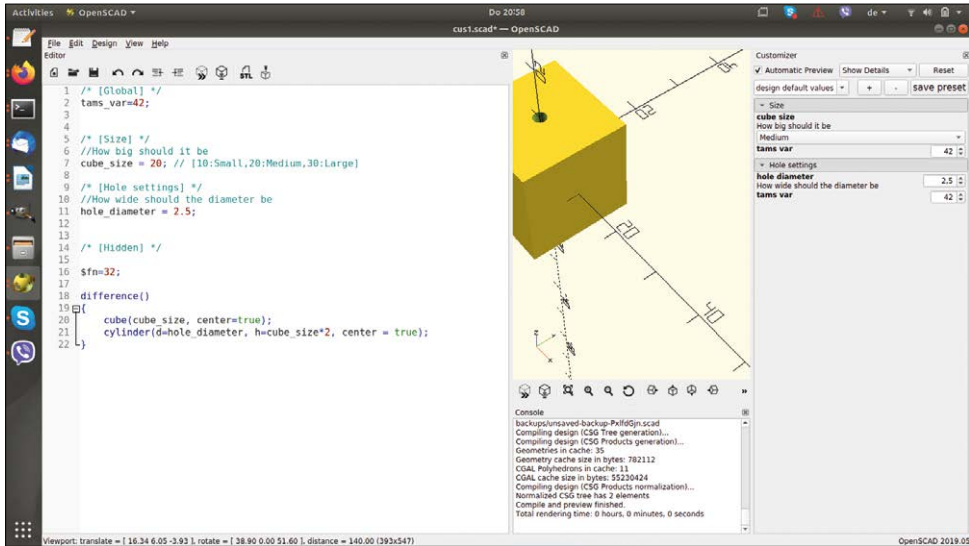


Figure 11-7. Tab grouping works without problems in the local customizer.

The combo box labeled Show Details lets you customize the appearance of the controls. The Automatic Preview checkbox is important because the customizer usually renders a new preview immediately after a control loses focus (and blocks the user interface in complex models).

Last but not least, it should be noted that the controls next to the Default Parameter Set option implement a kind of "profile system" - it allows you to keep several parameter sets available to conveniently manage variants of the object.

11.4 OpenSCAD without App

The advent of "high-performance Web browsers with WebGL support enabled repeated experiments to render OpenSCAD models in the browser. As these are currently still in active development, we would like to refer to the URL <https://openjscad.org/> - anyone who calls it up will see the start screen shown in Figure 11-8.

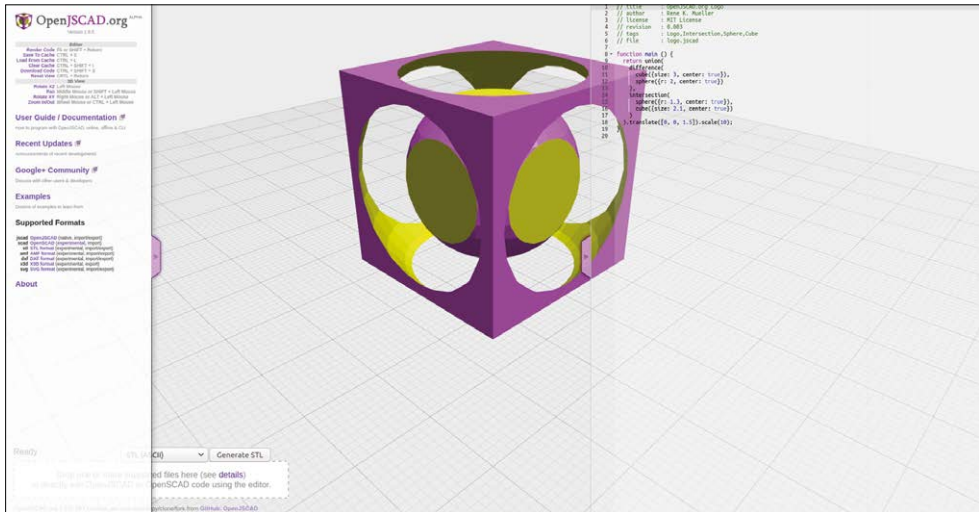


Figure 11-8. The browser version of OpenSCAD is ready for use.

Another interesting aspect in this context is that the product does not support pure OpenSCAD files at the time of going to press. Instead, the program works with a Javascript wrapper, which processes code structured according to the following scheme.

```
function main () {
  return union(
    difference(
      cube({size: 3, center: true}),
      sphere({r: 2, center: true})
    ),
    intersection(
      sphere({r: 1.3, center: true}),
      cube({size: 2.1, center: true})
    )
  ).translate([0, 0, 1.5]).scale(10);
}
```

In theory, it is possible to store ordinary scad files via the work area displayed in the lower-left corner, which the program then displays. Unfortunately, at the time of printing, this often fails with an error.

In other cases, however, the product works without problems. The use of the comment `//!OpenSCAD` seems to be of particular importance here - if it is on the first line of the file, the renderer does not work in JavaScript mode:

```
//!OpenSCAD
//How big should it be
cube_size = 20; // [10:Small,20:Medium,30:Large]
```

```
//How wide should the diameter be
hole_diameter = 2.5; // [1:200]

$fn=32;

difference()
{
    cube(cube_size, center=true);
    cylinder(d=hole_diameter, h=cube_size*2, center = true);
}
```

At this point, it should be noted that the experience and thought processes gained with "ordinary" OpenSCAD generally also work in the Javascript variant. A more detailed description of the product is available for download at https://en.wikibooks.org/wiki/OpenJSCAD_User_Guide.

11.5 OpenSCAD meets Python

Developers of parsers and similar systems are magically attracted to OpenSCAD application as it has a "front end" and a "back end", which together create a powerful pipeline for generating three-dimensional objects.

By far the most "radical" OpenSCAD extension is called SolidPython and can be found in GitHub at the URL <https://github.com/SolidCode/SolidPython>. It is a library that converts Python code into OpenSCAD commands.

The idea behind this is that the model developer writes Python code and then executes it. The methods contained in the library generate OpenSCAD output which can be reused more or less as desired.

Working with Python is a bit hairy under Windows, which is why the author relies on the Python3 interpreter included in Ubuntu 18.04 in the following steps. During the following experiments, the version status is as follows:

```
tamhan@TAMHAN18:~$ python3 --version
Python 3.6.9
```

Be that as it may, our first official act is to download the support libraries via the package manager PIP3 included in Python. Be careful not to inadvertently activate the "older" version pip - it would also run through, but load a library that the Python 3 interpreter cannot find:

```
tamhan@TAMHAN18:~$ pip3 install solidpython
Collecting solidpython
. . .
```

```
Successfully installed PrettyTable-0.7.2 euclid3-0.1 pypng-0.0.19
regex-2019.12.20 solidpython-0.4.2
```

We can then create a .py file in which we import the solid library and the corresponding utilities module like any other Python library:

```
from solid import *
from solid.utils import *
```

In the next step we create an object that is the difference between a cube and a sphere:

```
d = difference()(
    cube(10),
    sphere(15)
)
```

Because of the use of Python, a slightly different syntax is used. The difference operator is created as a function that can take "any number" of parameters. We call both cube and sphere, both methods generate one 3D object each. The "results" of the difference operator then represent an object that is temporarily stored in the variable d.

At this point, we only have to make sure that the code contained in d is addressable. The easiest way to solve this problem is to print it out to the command line:

```
print(scad_render(d))
```

At this point, we can venture a first program test run. Navigate to the location of the .py file in your terminal and release it for execution in the Python3 interpreter according to the following scheme

```
tamhan@TAMHAN18:/media/tamhan/781A-F80B/OpenSCADBook/CHWEB-DE$ python3
worker.py
```

```
difference() {
    cube(size = 10);
    sphere(r = 15);
}
```

The library will thank you by returning OpenSCAD code, as shown in the example output. You can then use the clipboard to transfer this code to another location.

In practice, it is often more convenient to be able to store this code "directly" in a working file. This is also possible without any problems if you adapt the code according to the following scheme for use with the `scad_render_to_file` method:

```
from solid import *
from solid.utils import *

d = difference()(
    cube(10),
    sphere(15)
)

scad_render_to_file(d, "filepath.scad")
```

If you run this version of the program and analyze the contents of the working directory, you will see the creation of a new file, which is located in the same directory as the code file:

```
tamhan@TAMHAN18:/media/tamhan/781A-F80B/OpenSCADBook/CHWEB-DE$ python3
worker.py
tamhan@TAMHAN18:/media/tamhan/781A-F80B/OpenSCADBook/CHWEB-DE$ ls
filepath.scad  worker.py
```

If you do not want this, you can pass in a "full" file path in the parameter. In any case, the file can then be loaded into OpenSCAD as usual, to interact with it.

OpenSCAD on command line.

Anyone using OpenSCAD's Python API to "automate" a workflow is dealing with a manual step at this point. However, there is also a command line tool that allows you to "remote control" the GUI - more information on this admittedly very advanced use case can be found at https://en.wikibooks.org/wiki/OpenSCAD_User_Manual/Using_OpenSCAD_in_a_command_line_environment#Command_line_usage.

At this point, we would like to point out a comfort feature of the OpenSCAD user interface shown in Figure 11-9. Activating the Automatic Reload checkbox instructs the IDE to permanently monitor the contents of the open file and to automatically load changes into the code editor. In this way, you save yourself clicks after each execution of the Python program.

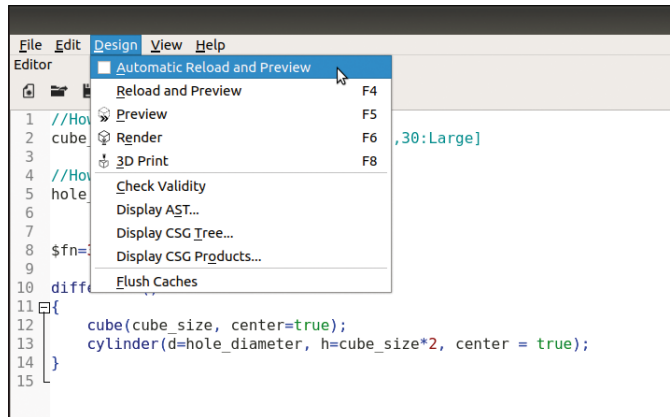


Figure 11-9. The Automatic Reload option facilitates work in "combinatorial" scenarios.

In addition to the obvious advantages of "programmatic" execution of Python code, its use also brings a group of special syntactical functions. By far the funniest feature is the ability to simply "add" or "subtract" objects as shown in the table.

Python-Program	Output
<pre>d = cube(10) + sphere(15) print(scad_render(d))</pre>	<pre>union() { cube(size = 10); sphere(r = 15); }</pre>
<pre>d = cube(10) - sphere(15) print(scad_render(d))</pre>	<pre>difference() { cube(size = 10); sphere(r = 15); }</pre>

Another interesting aspect is the option of replacing translation commands with "more conveniently understandable" operands. Again, look at a small table comparing the commands and the generated OpenSCAD code:

Python-Program	Output
<code>d = up(10)(cylinder())</code>	<code>translate(v = [0, 0, 10]) { cylinder(); }</code>
<code>d = down(10)(cylinder())</code>	<code>translate(v = [0, 0, -10]) { cylinder(); }</code>
<code>d = left(10)(cylinder())</code>	<code>translate(v = [-10, 0, 0]) { cylinder(); }</code>
<code>d = right(10)(cylinder())</code>	<code>translate(v = [10, 0, 0]) { cylinder(); }</code>
<code>d = forward(10)(cylinder())</code>	<code>translate(v = [0, 10, 0]) { cylinder(); }</code>
<code>d = back(10)(cylinder())</code>	<code>translate(v = [0, -10, 0]) { cylinder(); }</code>

Please refer to the readme and code samples available on GitHub for further information - a complete discussion of the possibilities is beyond the scope of this chapter.

11.6 Conclusion

The tools presented here enable you to use OpenSCAD outside of the usual OpenSCAD application. Our journey through the world of OpenSCAD is almost over at this point - last but not least, I have a list of recommended literature and general tips for you to get "more" out of your OpenSCAD installation.

Appendix • Quo vadis?

Congratulations – we have reached the "end" of our journey through the world of OpenSCAD. The only remaining question concerns how you can learn more.

First and foremost, it should be pointed out that construction - very much like cooking or electronics, for example - is a skill that can best be improved through its application. Find a nuisance and try to transfer it to the solution set - if it works, firstly you have one nuisance less, and secondly you usually have learned something along the way. Your nuisances may well be "moronic" tasks, such as constructing a (see figure) holder for a shower head - there is hardly anything that is not somehow instructive.



Figure A-1. This element was created in OpenSCAD.

Contact the development team

At some point, you are bound to have a problem where you don't know what to do. Readers of my electronics textbooks should know that it is advisable to get a little "distance" from the situation. If you haven't gotten to know them yet, you will find a summary of the most important hints in the box.

Medical break!

The term "operational blindness" is unpopular, especially in the IT sector, but it is widespread. /me could solve more than one problem by grabbing a rehydrating drink and walking around the block or through the lab.

Especially in combination with fresh air, one quickly comes up with new ideas. It should be noted that the energy drinks that are particularly popular in films and cyberlore are not ideally suited to engineering work - they contain too much sugar and too little water. If this is not enough, a little exercise can sometimes help - my practical experience shows that two to three medical strength training sessions per week greatly increase mental performance.

In the case of OpenSCAD, the situation is more complicated in that development has "stagnated" over the last few years. At <http://forum.openscad.org/>, you will find the forum shown in figure A-2, whose strange design is mainly because it is also run as a kind of mailing list.

Topics (204)	Replies	Last Post	Views
Registration and Subscription Info (READ THIS FIRST)	0	Jun 09, 2017 by MichaelAOz	2052
Reporting Bugs (not just general queries)	0	Jul 07, 2017 by MichaelAOz	997
OpenSCAD Merchandise - sales help support development	1	Nov 28, 2018 by MichaelAOz	395
Issue when inserting code in a .scd file	2	11-29-20 by new	4
Trying to difference text block from cube	0	10-30-20 by jordanthompson	7
2D preview connectivity issue	3	Jul 06 by Withouthat	5
Need help creating a hole in a solid	14	Jul 04 by mrgreath	40
Polyhedron generation from two parallel polygons?	15	Jul 03 by OpenSCAD mailing li...	56
Lib for linear extrusion with curved profiles?	7	Jul 03 by Alexander Prusa	58
Inconsistent interpretation of \$fs	25	Jun 29 by mrgreath	40
Also output currently required	7	Jun 28 by bawards	30
Issue with simple 2D vector addition	6	Jun 26 by amundsen	27
SVG import questions	9	Jun 25 by mrgreath	40
How to solve without classic variables?	11	Jun 25 by acwrest	37
Normal float direction vector for extruding geometry by Leo_Sief	12	Jun 25 by Parkincoe	49
A stack of polyhedrons - getting 2D slices instead of 3D	2	Jun 23 by toli	14
Approximate a 3D object by an ellipsoid	3	Jun 24 by Ekinola	10
WHAT IN GODS NAME MUST I DO TO BE REMOVED FROM THIS MAILING LIST??	4	Jun 23 by thoose	48
Exporting information from a model	11	Jun 22 by JordanBrown	45
2D cross section	2	Jun 22 by thoose	23

Figure A-2. The motto of the OpenSCAD forum could very well be: old-fashioned is us!

The question of whether it is better to join the mailing list or the forum can be discussed very well. Less room for discussion arises from the question of whether it is sensible to consult Google when technical problems occur. OpenSCAD enjoys an immense community of users, which is why information can be found in a large number of blog posts, Instagram images, and the like.

It should be noted that many model-specific 3D printer forums also have areas where OpenSCAD users can exchange information. Anyone who operates one or more printers should join a user forum for these devices and at least observe it peripherally. Incidentally, this also applies to measurement technology - especially for older devices, the owner's mailing lists contain very valuable service information.

Contact the author

Whoever buys a textbook of mine may of course contact me. The easiest way is to send your request by e-mail to tamhan@tamoggemon.com.

Another recommended source of information is the Instagram account shown in figure three and available at <https://www.instagram.com/tam.hanna/>. You can find out about almost everything interesting for an electronics designer there - OpenSCAD is also mentioned often.

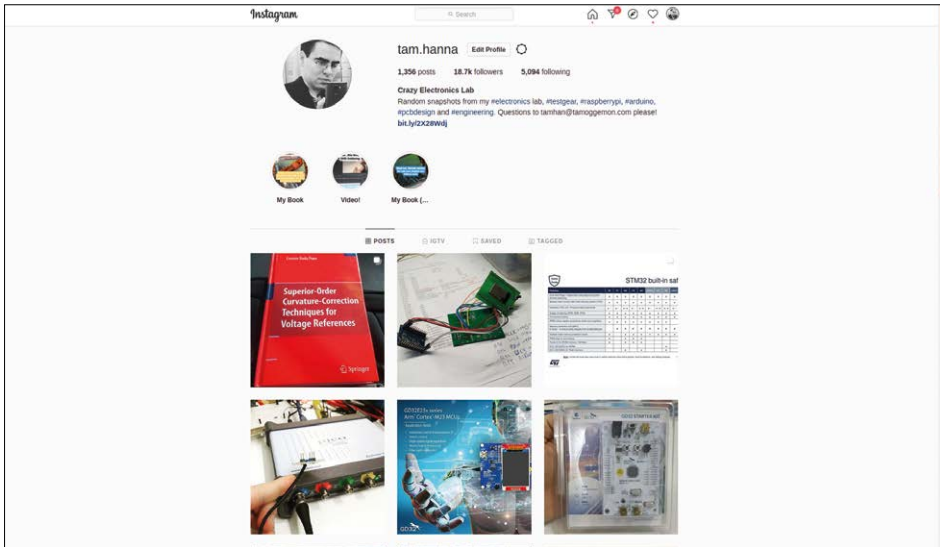


Figure A3. Electronics, Metrology, and Cigars - 18700 subscribers can't be wrong.

Read more

Finally, it should be noted that the shop where you bought this book likely offers a whole group of other books. The author believes that reading a good textbook remains one of the most efficient ways to become familiar with topics.

In the field of "classic" hard work with 3D printers based on the FDM process, Clifford Smyth's "Functional Design for 3D Printing: Designing 3D printed things for everyday use" has established itself as a classic. The textbook deals with the "functional" aspects of 3D printing: the author addresses how to print or create objects which have to pass certain requirements.

If there is a lack of "general" mechanical understanding, the English language "Mechanical Engineering for Makers" by Bunnell and Najia is recommended. Although the authors have a very "flappy" style, they present just about everything the budding mechanic wants to learn, from gears to extruders, to screws. Moreover, the book is richly illustrated.

For "more in-depth" considerations about machine elements, we recommend the classic Decker's Maschinenelemente by Hanser Verlag, unfortunately only available in German. The book is not an easy read - but after reading it, there should be hardly any open questions left.

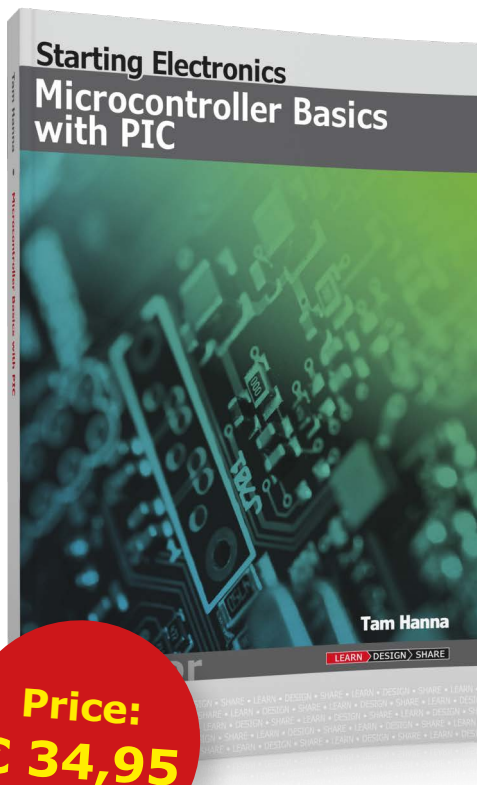
In general, it should also be pointed out that technical universities generally have well-stocked libraries, which are usually accessible to non-students against payment of a small fee and friendly (!!!) treatment of the staff. A little "phoning around" can bring extreme dividends in this area.

Index

2D modeling	64	E	
2D polygons	67	echo command	94
3D modeler	34	Edges	55
3D printers	11	Ellipses	44
3D View section	31	F	
.gcode file	59	fonts	124
.STL file	28	Font Size	31
		Functions	110
A		G	
AppImage files	16	GIMS	57
array	36	GitHub	23
Assert	104	GitHub repository	157
Automatic Preview	174	Git repository	17
		GUI design	79
B		H	
BNF form	39	HALIGN	123
boreholes	148	heptagonal column	71
		I	
C		import/export libraries	19
C++	36	Installing OpenSCAD	15
calculations	110	intersection	52
Circle	57	intersection operator	52
Circle command	67	L	
CNC	33	LEGO	161
Color operator	47	linear_extrude	87
CommonDialog	25	linear_extrude command	74
compiler	19	Linear extrusion	70
corners	55	Linux	16
cube	33	M	
CubeFaces	138	MCAD	156
Cube function	36	MCAD library	157
CubePoints	150	Minkowski operator	135
cuboid	33	N	
Customizer	170	numerical values	37
cylinder	27, 39	nuts_and_bolts library	160
D			
data types	36		
Debian packages	21		
Degrees	46		
Detents	108		
Development Snapshots	23		
difference operator	52		

O		Union	51
Open command	29		
OpenSCAD documentation	37	V	
P		VALIGN	122
polygon operator	82	vector	34
polygon structures	138	View menu	31
polyhedron	141	Viewport	29
prefix	41	Visual Studio	22
Preview	40	voxel	36
projection	126	W	
Python	13	WebGL support	174
Q		Windows	22
qmake	19	Wireframe	31
qmake command	19	X	
R		X-Y plane	127
Radiants	46		
Render button	40		
rendering	27		
Render texts	120		
rotate	45		
Rotation	44		
S			
scale	69		
Slicer	59		
Snapshots	23, 125		
SolidPython	176		
Sphere	39		
stepper motors	162		
stereolithography	59		
STL file	60		
superstructure	150		
Surface mode	31		
T			
Text Editor	31		
Thingiverse	165		
Thingiverse Customiser	172		
Translate	43		
Troubleshooting	115		
U			
Ubuntu	15		

Microcontroller Basics with PIC



Description

In this book the author presents all essential aspects of microcontroller programming, without overloading the reader with unnecessary or quasi-relevant bits of information. Having read the book, you should be able to understand as well as program, 8-bit microcontrollers.

The introduction to microcontroller programming is worked out using microcontrollers from the PIC series. Not exactly state-of-the-art with just 8 bits, the PIC micro has the advantage of being easy to comprehend. It is offered in a DIP enclosure, widely available and not overly complex. The entire datasheet of the PIC micro is shorter by decades than the description of the architecture outlining the processor section of an advanced microcontroller. Simplicity has its advantages here. Having mastered the fundamental operation of a microcontroller, you can easily enter into the realms of advanced softcores later.

Having placed assembly code as the executive programming language in the foreground in the first part of the book, the author reaches a deeper level with 'C' in the second part. Cheerfully alongside the official subject matter, the book presents tips & tricks, interesting measurement technology, practical aspects of microcontroller programming, as well as hands-on options for easier working, debugging and faultfinding.

- **Author:** Tam Hanna
- **Pages:** 318 pages
- **Format:** 14 x 21 cm (Paperback)
- **ISBN:** 978-1-907920-81-3
- **www.elektor.nl/microcontroller-basics-with-pic**

Technical Modeling with OpenSCAD

Create Models for 3D Printing, CNC Milling,
Process Communication and Documentation

Engineers dread designing 3D models using traditional modeling software. OpenSCAD takes a refreshing and completely different approach. Create your models by arranging geometric solids in a JavaScript-like language, and use them with your 3D printer, CNC mill, or process communication.

OpenSCAD differs from other design systems in that it uses programmatic modeling. Your model is made up of primitives that are invoked using a C-, Java- or Python-like language. This approach to model design is close to the “mechanical work” done in the real world and appeals to engineers and others who are not a member of the traditional creative class.

OpenSCAD also provides a wide variety of comfort functions that break the 1:1 relationship between code and geometry. This book demonstrates the various features of the programming language using practical examples such as a replacement knob for a LeCroy oscilloscope, a wardrobe hanger, a container for soap dispensers, and various other real-life examples.

Written by an engineer with over 15 years of experience, this book is intended for Linux and Windows users alike. If you have programming experience in any language, this book will have you producing practical three-dimensional objects in short order!



Tam Hanna has been programming microcontrollers and microprocessors systems for over 15 years aimed at different areas. Besides his consulting activities he gives lectures congresses, writes specialist articles for various magazines and tutors. His popular Instagram channel supplies background information on measurement technology and electronics.

Elektor International Media BV
www.elektor.com